

Data Structures -- Comprehensive Exam Preparation Guide

ASE Bucharest -- Prof. Mihai Doinea -- English Edition

Table of Contents

Section	Title	Page
1	Pointers & the Memory Model	--
2	Complexity & Big-O Notation	--
3	Arrays & Dynamic Arrays	--
4	Linked Lists	--
5	Stacks & Queues	--
6	Hash Tables	--
7	Binary Heaps	--
8	Recursion	--
9	Binary Search Trees	--
10	AVL Trees	--
11	B-Trees	--
12	Graphs	--
13	Sorting Algorithms	--
14	Advanced Topics & Integration	--
App A	Master Big-O Cheat Sheet	--
App B	MCQ Coverage Matrix	--
App C	C Memory Glossary	--

How to Use This Guide

This guide is organized into 14 chapters, each covering one core topic from the Data Structures curriculum. Every chapter follows a consistent 13-section structure designed to take you from zero knowledge to exam readiness: learning objectives, a mental model, formal definitions, step-by-step operations, trace-through examples, Big-O tables, C code references, ASCII diagrams, exam hot spots, common traps, and a quick-recall card. This uniformity lets you build a study rhythm -- once you know how one chapter works, every other chapter follows the same pattern.

MCQ coverage markers appear throughout. Each chapter's "Exam Hot Spots" section lists the specific question IDs from the 339-question corpus that test that topic. Use these markers to gauge where the exam focuses most heavily. Chapters with

more assigned questions deserve proportionally more study time. Appendix B provides the full coverage matrix so you can verify coverage at a glance and identify any gaps.

Quick-Recall Cards at the end of each chapter are condensed summaries of the most-tested facts. Cut them out (physically or mentally) and review them in the final days before the exam. They are written as bare-bones reference points -- no prose, no examples, just the facts you need to have memorized.

The three appendices are reference material. Appendix A is a Big-O cheat sheet covering every data structure and sorting algorithm. Appendix B maps all 339 exam questions to their chapters. Appendix C is a glossary of C memory concepts that appear across all chapters. Keep these appendices open during practice sessions and close them during mock exams to simulate test conditions.

\newpage

Chapter 1 — Pointers & the Memory Model

Foundation chapter. Pointers and the Stack-vs-Heap memory model are the prerequisite for every data structure in this course. This chapter is also the source of the most-tested C code-analysis questions in the exam (120 MCQ corpus questions are assigned to this chapter).

1. Learning Objectives

After reading this chapter, you will be able to:

- Explain the difference between Stack and Heap memory regions, with a concrete map of where each variable lives.
- Declare, initialize, dereference, and pass pointers in C without producing undefined behavior.
- Trace through `malloc` / `free` C code and predict the output, including the canonical "what happens after `free(p)`" question.
- Identify and fix the three most common pointer bugs: uninitialized pointer, dangling pointer, and memory leak.
- Explain why list nodes are heap-allocated and why this matters for pointer-based data structures.
- Distinguish between static (stack) and dynamic (heap) array allocation, including the `free` responsibility.

2. Mental Model & Analogy

Think of memory as a giant city of post-office boxes, each one byte wide. Every box has a numbered address (e.g., box #1000, box #1001, ...). A **pointer** is a slip of paper that holds the *number* of a box — the slip doesn't *contain* the thing, it contains *where to find* the thing.

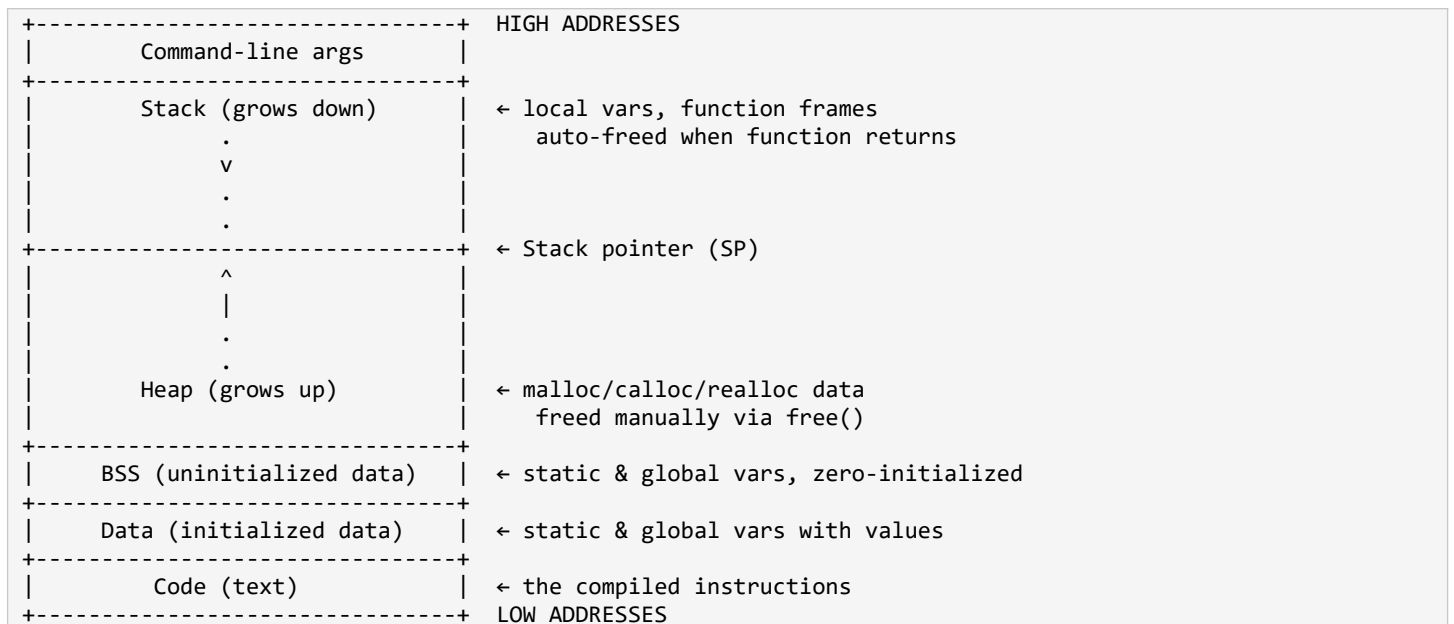
Three rules to internalize:

1. **A pointer is not the data; it's the address of the data.** Looking at the slip tells you the box number; to read what's inside, you have to *go to* that box (that's dereferencing).
2. **A pointer has a type.** A slip saying "go to box #1000 and read 4 bytes" is different from "go to box #1000 and read 1 byte." The type tells the compiler how to interpret what you find.
3. **A pointer is either valid or it is not.** A valid pointer holds the address of a real, currently-allocated object. An invalid pointer holds either garbage (uninitialized), a freed address (dangling), or zero (`NULL`).

When you `malloc`, you're asking the operating system: "give me a new box (or several), here is how big it needs to be, and tell me its number so I can write it on a slip." When you `free`, you're returning that box to the OS — but the slip you wrote is still in your hand, still has the same number on it, and it is now a **lie** unless you scribble `NULL` on it.

3. Memory Layout

Every C program's memory is divided into named regions. The two you must know cold are **Stack** and **Heap**.



Key facts the exam will test:

- **Stack:** allocation is automatic. When a function is called, a new "frame" is pushed on top of the stack containing the function's local variables. When the function returns, the frame is popped and all those locals are gone. Stack memory is fast, but limited in size (typically a few MB). **You do NOT call `free()` on stack memory** — it is reclaimed automatically.
- **Heap:** allocation is manual via `malloc` / `calloc` / `realloc`. Memory stays allocated until you explicitly `free()` it (or the program ends). Heap is much larger than the stack but allocation/deallocation is slower. The OS does not know when you're "done" with heap memory — that's your job.
- **The terms "stack" and "heap" are NOT the same as the data structure terms "stack" (LIFO) and "heap" (priority queue).** The memory-region "heap" is just a name for the free-store region; it has nothing to do with the binary-heap data structure of Chapter 7. The exam sometimes deliberately conflates them in wrong-answer options.
- **BSS / Data / Code** are the other three regions; they hold global/static variables and the program instructions. You won't be tested on them in detail, but you should know they exist.

What lives where — for the exam:

Variable	Region	Lifetime	Who frees it?
<code>int x;</code> inside a function	Stack	Until function returns	Automatic
<code>int arr[10];</code> inside a function	Stack	Until function returns	Automatic
<code>static int counter;</code> in a function	BSS/Data	Entire program	Never (frees at program exit)
<code>int* p = malloc(sizeof(int));</code>	Heap	Until you call <code>free(p)</code>	You
<code>struct Node* n = malloc(sizeof(struct Node));</code>	Heap	Until you call <code>free(n)</code>	You

4. Definition & Properties

Pointer — a variable whose value is a memory address. Declared with a `*` between type and name: `int* p;` means "p is a pointer to an `int`."

Properties:

- A pointer is **typed** (`int*`, `char*`, `void*`, ...). The type tells the compiler how to interpret the bytes at that address and how big a step to take on pointer arithmetic.
- A pointer can be **dereferenced** with `*p` to access the object it points to.
- A pointer can be the operand of `&` (`&x` gives the address of `x`) and `*` (`*p` gives the value at the address held in `p`).
- A pointer can be `NULL` (a special "no valid object" sentinel). Dereferencing `NULL` is undefined behavior (on most systems: a segmentation fault).
- A pointer can be **uninitialized** (e.g., `int* p;` with no assignment). Its value is **garbage** — whatever bits happened to be in that stack slot. Dereferencing it is undefined behavior.
- A pointer can be **dangling** (e.g., after `free(p)`, the value of `p` is unchanged but no longer points to a valid object). Dereferencing it is undefined behavior.
- Pointer arithmetic (`p + 1`, `p - p2`) is valid only within arrays (including one-past-the-end) and only on pointers to the same type. `p + 1` advances the address by `sizeof(*p)` bytes.
- `void*` is a "pointer to anything" — used by `malloc` because it doesn't know what type you want. You must cast it to a real type before dereferencing.

Casts and pointer manipulation:

- `(int*)malloc(sizeof(int))` — explicit cast from `void*` to `int*`. In C this is optional (implicit conversion is allowed), but in C++ it is required. The exam corpus uses the explicit form.
- `int* p = (int*)1000;` — assigning a literal address. Almost always wrong in exam code; do not do this.

5. Operations — Step-by-Step Mechanics

5.1 Declaring a pointer

```
int* p;
```

Step 1. The compiler reserves `sizeof(int*)` bytes (typically 4 or 8) on the **stack** in the current function's frame. The name `p` is bound to those bytes.

Step 2. The bytes are **uninitialized** — they contain whatever was left in that stack slot from a previous function call. `p` is *not* `NULL` by default; it is *garbage*.

Step 3. Reading `p` before writing to it is undefined behavior. Always initialize a pointer: either to `NULL` (`int* p = NULL;`) or to the address of a real object (`int* p = &x;`).

5.2 Taking the address with &

```
int x = 42;
int* p = &x;
```

Step 1. `&x` computes the address of `x` (a constant at compile time for stack variables).

Step 2. That address is written into `p`. Now `*p` and `x` refer to the same memory.

5.3 Dereferencing with *

```
#include <stdio.h>
int main() {
    int x = 42;
    int* p = &x;
    printf("%d\n", *p); // prints 42
    *p = 100;           // changes x to 100
}
```

```
    return 0;
}
```

Step 1. `*p` means "go to the address held in `p` and treat what's there as an `int`." Reading `*p` reads 4 bytes starting at that address and interprets them as an `int`.

Step 2. Writing `*p = 100` puts the bytes for 100 at that address. Since `p` points to `x`, this changes `x` itself.

5.4 Allocating with `malloc`

```
#include <stdlib.h>
int main() {
    int* p = (int*)malloc(sizeof(int));

    return 0;
}
```

Step 1. `sizeof(int)` is evaluated at compile time — on most modern systems it is 4.

Step 2. `malloc(4)` asks the OS for a 4-byte block of heap memory. It returns the address of that block as a `void*`.

Step 3. The cast `(int*)` converts the `void*` to `int*`. (Required in C++, optional in C; the corpus uses the explicit form.)

Step 4. That address is stored in `p`. The 4 bytes at that address are **uninitialized** — they contain whatever happened to be in that heap region.

Step 5. You can now write to that memory: `*p = -10;` or `p[0] = -10;` (these are equivalent for index 0).

5.5 Freeing with `free`

```
#include <stdlib.h>
int main() {
    int* p = (int*)malloc(sizeof(int));
    free(p);
    return 0;
}
```

Step 1. The 4 bytes at the address held in `p` are returned to the heap's free list. The OS marks them as available for future `malloc` calls.

Step 2. `p` itself is not changed. The local variable `p` still holds the same address. Reading `p` will still return that address — but the object it points to no longer exists.

Step 3. This is the source of the **dangling pointer** bug. The standard pattern to avoid it:

```
#include <stdlib.h>
int main() {
    int* p = (int*)malloc(sizeof(int));
    free(p);
    p = NULL;
    return 0;
}
```

After this, reading `p` gives `NULL` and dereferencing it crashes cleanly (rather than silently corrupting some other heap allocation that may have been placed at the same address by a subsequent `malloc`).

5.6 Passing pointers to functions

Two ways to "modify a variable inside a function":

Pass by value (no effect on caller):

```

void f(int x) { x = 100; }
int main() {
    int a = 5;
    f(a);
    // a is still 5 - f got a copy

    return 0;
}

```

Pass by pointer (modifies caller):

```

void f(int* p) { *p = 100; }
int main() {
    int a = 5;
    f(&a);
    // a is now 100 - f wrote through the pointer

    return 0;
}

```

Exam trap: C does NOT have true pass-by-reference. When you "pass by reference" in C, you are really passing the address of the variable. The function receives a pointer; it dereferences that pointer to modify the original.

6. Trace-Through Example

This is the most-tested C code in the entire corpus. Trace it carefully.

```

#include <stdio.h>
#include <malloc.h>

int main()
{
    int* pc, c, d;
    pc = (int*)malloc(sizeof(int));
    pc[0] = -10;
    c = *pc + 11;
    d = c + 1;
    printf("c = %d, d = %d, *pc = %d\n", c, d, *pc);
    free(pc);
    return 0;
}

```

Line-by-line trace:

Line	Effect	State after
int* pc, c, d;	Reserve stack space for 3 vars: pc (uninit), c (uninit), d (uninit)	pc = ?, c = ?, d = ?
pc = (int*)malloc(sizeof(int));	Allocate 4 bytes on heap, store address in pc	pc = 0x???? (some heap address), c = ?, d = ?
pc[0] = -10;	Write -10 (as 4 bytes) into the heap block	*pc = -10, c = ?, d = ?
c = *pc + 11;	Read *pc (-10), add 11 (= 1), store in c	*pc = -10, c = 1, d = ?
d = c + 1;	Read c (1), add 1 (= 2), store in d	*pc = -10, c = 1, d = 2
printf(...);	Output: c = 1, d = 2, *pc = -10	(unchanged)
free(pc);	Return heap block to OS. `pc` is not changed.	pc still = the (now-freed) address

Output: c = 1, d = 2, *pc = -10

Exam trap: A very common wrong answer is $c = 1$, $d = 1$, $*pc = -10$ (assuming $d = c$ instead of $d = c + 1$).
Re-read the line carefully.

7. Operations + Big-O Table

Operation	Complexity	Notes
Declare a pointer	$O(1)$	One stack-slot allocation
Take address with <code>&</code>	$O(1)$	Computed at compile time for stack vars
Dereference with <code>*</code>	$O(1)$	One memory read or write
Pointer comparison (<code>==</code> , <code>!=</code>)	$O(1)$	Just compares two addresses
Pointer arithmetic (<code>p + n</code>)	$O(1)$	Just adds $n * \text{sizeof}(*p)$ to the address
<code>malloc(n)</code>	$O(1)$ amortized	Usually constant; occasionally $O(n)$ for big allocations
<code>free(p)</code>	$O(1)$ amortized	Usually constant
<code>sizeof</code>	$O(1)$	Compile-time constant
<code>memcpy(dst, src, n)</code>	$O(n)$	n bytes to copy
<code>strlen(s)</code>	$O(n)$	Walks the string until <code>\0</code>

Note: This chapter is about the memory model rather than a single data structure, so the Big-O rows above are about pointer operations, not data-structure operations. The data-structure Big-O values are in the respective chapters (Ch 3 Arrays, Ch 4 Linked Lists, etc.).

8. C Code Reference

8.1 The canonical malloc/free example (most-tested)

```
#include <stdio.h>
#include <malloc.h>

int main()
{
    int* pc, c, d;
    pc = (int*)malloc(sizeof(int));
    pc[0] = -10;
    c = *pc + 11;
    d = c + 1;
    printf("c = %d, d = %d, *pc = %d\n", c, d, *pc);
    free(pc);
    return 0;
}
/* Output: c = 1, d = 2, *pc = -10 */
```

8.2 The uninitialized-pointer bug

```
#include <stdio.h>

int main()
{
    int* p;
    *p = 10;          /* BUG: p is uninitialized, holds a random address */
    return 0;
}
```

What goes wrong: `p` contains a garbage address. Writing `*p = 10` overwrites whatever happens to be at that random address — could be another program's memory, your own stack, etc. The behavior is **undefined**. The exam answer: "Memory is not allocated before use" (i.e., you forgot to call `malloc`).

8.3 The free-and-NULL pattern

```
#include <stdlib.h>
int main() {
    int* v = malloc(10 * sizeof(int));
    /* ... use v ... */
    free(v); /* returns the heap block to the OS */
    v = NULL; /* defensively clear the pointer so a stray deref crashes cleanly */

    return 0;
}
```

Exam trap: The exam often asks "what is true about the pointer variable `p` after `free(p)`?" The correct answer is "it still holds the heap address it had at allocation" — NOT "it is reset to NULL" (free does not change the pointer's value) and NOT "it is deallocated" (only the heap block is deallocated, the stack variable still exists).

8.4 Array name as pointer decay

```
#include <stdio.h>
int main() {
    int v[] = {10, 20};
    int* pv = v; /* array name decays to pointer to first element */
    printf("%d %d\n", v[0], pv[0]); /* both print 10 */

    return 0;
}
```

Key fact: In most contexts, the array name `v` "decays" to a `int*` pointing to `v[0]`. So `v[0]` and `pv[0]` access the same memory. The array name is NOT a modifiable lvalue — you cannot write `v = NULL;` — but it can be assigned to a pointer variable.

8.5 Static vs dynamic array

```
#include <stdlib.h>
int main() {
    /* Static (stack) array – size must be a compile-time constant */
    int arr[100];
    /* no malloc, no free – the compiler handles it; lifetime = enclosing function */

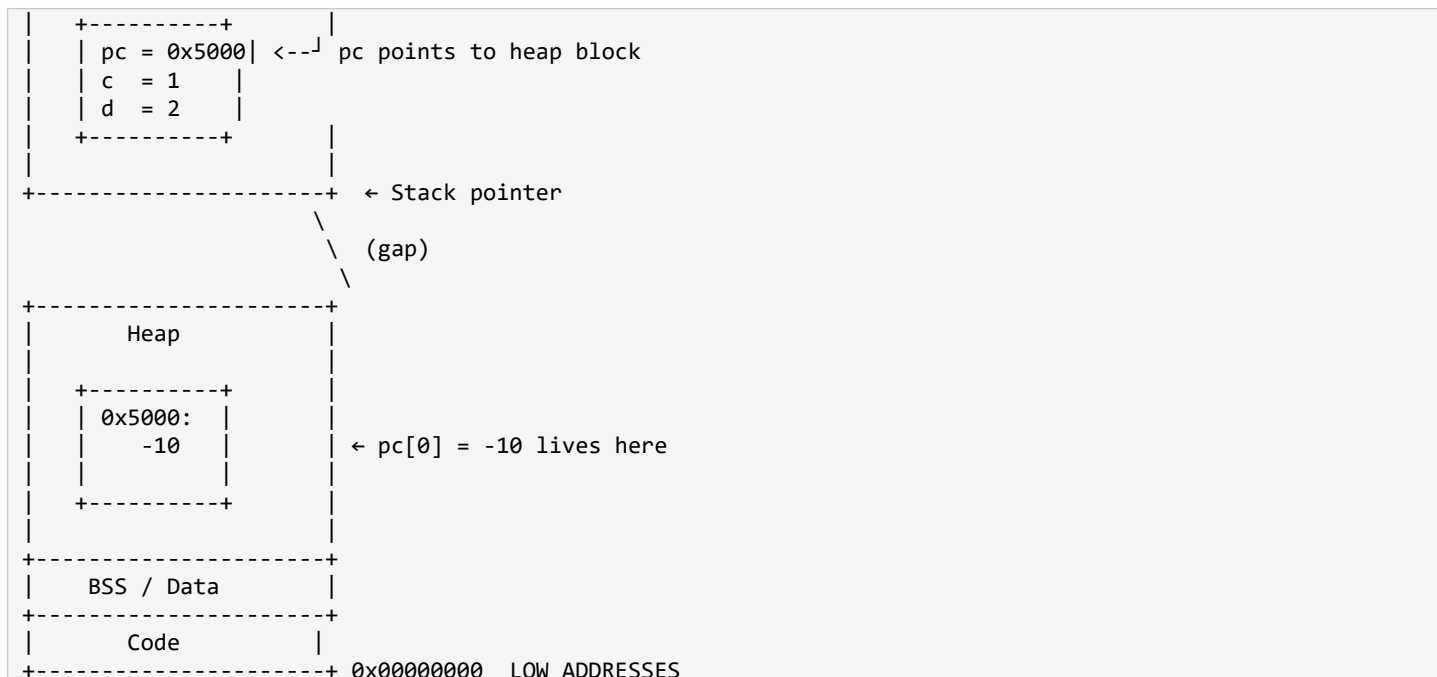
    /* Dynamic (heap) array – size can be computed at runtime */
    int n = 100;
    int* darr = (int*)malloc(n * sizeof(int));
    /* must free when done */
    free(darr);
    return 0;
}
```

Exam fact: Statically-allocated arrays are placed on the **stack** of the function in which they are declared. They do NOT need to be freed. Dynamically-allocated arrays (via `malloc`) are placed on the **heap** and MUST be freed.

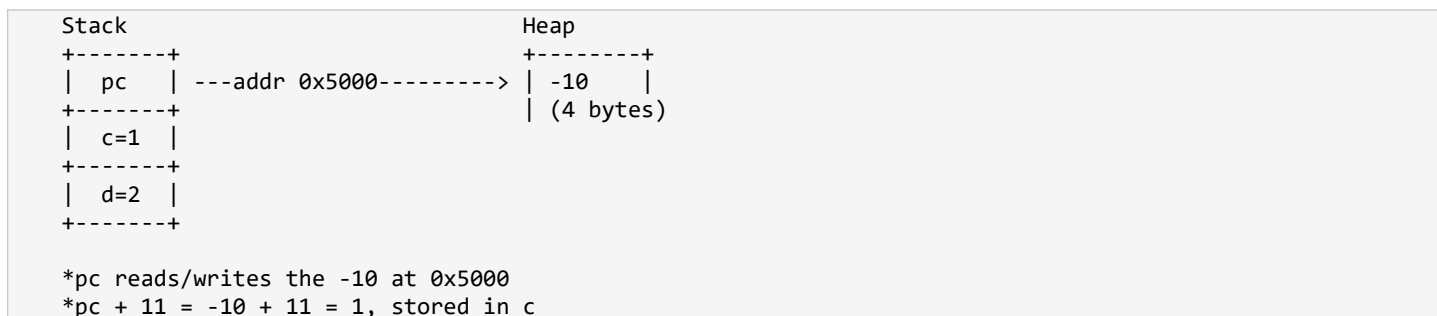
9. ASCII Diagram

Memory layout during the trace-through example:

+-----+ 0xFFFFFFFF HIGH ADDRESSES
Stack



Pointer relationships in the same example:



10. When to Use / When to Avoid

Use pointers when:

- You need to modify a variable inside a called function (pass by address).
- You need a dynamically-sized data structure (linked list, tree, hash table — these are heap-allocated).
- You need to refer to "the same object" from multiple places without copying.
- You're implementing a function that returns multiple values (out-parameters via pointers).

Avoid pointers (prefer plain values) when:

- The data is small and you don't need to share it (just use a local variable).
- The data has a fixed, known size and you can use an array.
- The lifetime is purely lexical (use a local variable — the stack handles cleanup for you).

Always remember:

- After `free(p)`, the pointer `p` itself is unchanged. Set it to `NULL` yourself.
- Never dereference an uninitialized pointer.
- Never dereference a dangling pointer.
- Never `free` the same pointer twice.
- Never return a pointer to a local (stack) variable from a function — it will be a dangling pointer by the time the caller uses it.

11. Exam Hot Spots

This chapter has **120 MCQ corpus questions** assigned to it. They cluster into the following patterns. For each, the pattern, the correct reasoning, and example question IDs are given.

Pattern A: malloc / free behavior

Q24, Q48, Q84: "What is the output of this malloc + pointer-arithmetic C code?" (canonical trace)

- **Correct reasoning:** Walk through the code line by line. The most common wrong answer is misreading one of the arithmetic operations. Always re-read the operator carefully: `c = *pc + 11` (not `c = *pc + 1` followed by another step).

Pattern B: Static vs dynamic array characteristics

Q31, Q47, Q88, Q102: "Select the characteristics of statically allocated arrays"

- **Correct:** "Size is fixed", "Allocated on the stack where they are declared"
- **Wrong:** "Must be freed with `free()`" (only heap arrays need free), "Size can be set at runtime" (static arrays need compile-time size), "Can be resized with `realloc()`" (only heap arrays)

Pattern C: Pointer-after-free

Q32, Q51, Q98, Q155: "What is true about the pointer variable `p` after calling `free(p)`?"

- **Correct:** "It still holds the address it had at allocation"
- **Wrong:** "It is set to NULL by free" (false — `free` doesn't change the pointer), "It is deallocated" (false — the pointer variable on the stack is unchanged)

Pattern D: Uninitialized pointer bug

Q27, Q53: "Identify the problem: `int* p; *p = 10;`"

- **Correct:** "Memory is not allocated before use" (i.e., missing `malloc`)
- **Wrong:** "Pointer is constant" (false), "Variable `p` is in read-only code segment" (false — it's on the stack, just uninitialized)

Pattern E: Pointer variable properties

Q37, Q56, Q111, Q112: "Which statements are true about pointer variables in C?"

- **Correct:** "Has special declaration syntax (with `*`)", "Stores a memory address"
- **Wrong:** "Can be used in any arithmetic" (false — limited to pointer arithmetic), "Can only be used to access heap memory" (false — can point anywhere), "Can be used in any calculation" (false)

Pattern F: Array indexing and bounds

Q3, Q74, Q76, Q96, Q114: "What is the last valid index of `char mesaj[10]`?"

- **Correct:** 9 (zero-based: indices 0 through `n-1`)
- **Wrong:** 10 (that's the size, not the last valid index), `mesaj + 1` (that's the address of the byte past the array — one-past-the-end is a valid pointer value but not a dereferenceable location)

Q3: "What does contiguous storage usually improve?"

- **Correct:** "Cache locality"
- **Wrong:** "Pointer chasing" (opposite — pointer chasing is what scattered memory does), "Hash collisions" (unrelated), "Tree balancing" (unrelated)

Pattern G: List nodes are in the heap

Q13, Q72: "Select the true statements for linked lists and static arrays"

- **Correct:** "List nodes are allocated at random addresses in the HEAP memory", "Lookup operation in an array is a $O(1)$ complexity time operation"
- **Wrong:** "Static arrays have the memory allocated at runtime" (false — at compile time / function-entry), "Accessing element i in a list is a $O(1)$ operation" (false — it's $O(n)$), "Memory size for static arrays is limited only by the HEAP size" (false — static arrays are on the stack, which is much smaller)

Pattern H: Pass-by-pointer semantics

Q91, Q106, Q118, Q127, Q210: "Complete the code to free and nullify a pointer" / "What does this pointer-modifying function do?"

- **Correct:** `free(v); v = NULL;`
- **Wrong:** `delete(v); v = NULL;` (delete is C++, not C), `free(v); v = malloc(nrEl);` (re-allocates, doesn't nullify), `clear(v); v = NULL;` / `clean(v); v = NULL;` (those aren't C standard functions)

Pattern I: Complete question inventory

All 120 corpus questions for this chapter, grouped by pattern:

ID	Topic	Pattern Group
Q3	Contiguous storage improves cache locality	F
Q12	Circular linked list has no NULL pointer	G
Q13	List nodes allocated at random heap addresses	G
Q20	Doubly linked list advantages over singly	E
Q24	malloc + pointer-arithmetic output (c=1,d=2)	A
Q26	malloc() is the C dynamic allocation function	A
Q27	Uninitialized pointer: missing malloc before dereference	D
Q31	Static array: fixed size, allocated on stack	B
Q32	Pointer after free retains its heap address	C
Q33	Array-pointer output: v[0]=10, pv[0]=10	A
Q37	Pointer variable: special syntax, stores address	E
Q44	free(v); v = NULL; to release dynamic memory	H
Q45	malloc() for dynamic array allocation	A
Q47	Static array: fixed size, stack-allocated	B
Q48	Array-pointer output: v[0]=10, pv[0]=10	A
Q51	Pointer after free retains its heap address	C
Q53	Uninitialized pointer: missing malloc before	D

	dereference	
Q56	Pointer variable: special syntax, stores address	E
Q68	Heap is visualized as a binary tree	misc
Q70	Circular singly linked list: last node points to first	G
Q72	List nodes in heap, array lookup is O(1)	G
Q74	Pointer arithmetic: finding address of int data	F
Q76	Array bounds: accessing beyond allocated size	F
Q83	Hash table chaining search implementation correctness	misc
Q84	malloc + pointer-arithmetic output (c=1,d=2)	A
Q86	Hash table chaining: allocate array of Nod* pointers	misc
Q87	free(v); v = NULL; to release dynamic memory	H
Q88	Linear probing search: correct when studName is key	misc
Q91	malloc() is the C dynamic allocation function	A
Q94	Uninitialized pointer: missing malloc before dereference	D
Q95	Linear probing search: correct when studName is key	misc
Q96	Static array: fixed size, allocated on stack	B
Q98	Pointer after free retains its heap address	C
Q99	Hash table chaining search implementation correctness	misc
Q100	Pointer after free retains its heap address	C
Q102	Pointer variable: special syntax, stores address	E
Q103	Hash table chaining: allocate array of Nod* pointers	misc
Q106	Function returns head info, deletes first node	H
Q108	Linked list append: malloc + set next + NULL	misc
Q111	Linked list append: malloc + set next + NULL	misc
Q112	Function returns head info, deletes first node	H
Q114	Pointer variable: special syntax, stores address	E
Q118	Array-pointer output: v[0]=10, pv[0]=10	A

Q119	Uninitialized pointer: missing malloc before dereference	D
Q120	Pointer after free retains its heap address	C
Q121	Static array: fixed size, allocated on stack	B
Q123	free(v); v = NULL; to release dynamic memory	H
Q127	malloc + pointer-arithmetic output (c=1,d=2)	A
Q210	Hash table chaining search implementation correctness	misc
Q212	free(v); v = NULL; to release dynamic memory	H
Q213	Static array: fixed size, allocated on stack	B
Q215	malloc + pointer-arithmetic output (c=1,d=2)	A
Q216	Linear probing search: correct when studName is key	misc
Q217	Pointer after free retains its heap address	C
Q219	Array-pointer output: v[0]=10, pv[0]=10	A
Q220	Uninitialized pointer: missing malloc before dereference	D
Q223	Pointer variable: special syntax, stores address	E
Q224	Linked list append: malloc + set next + NULL	misc
Q233	AVL right rotation: left child becomes root	misc
Q237	BST limitation: finding min in O(1) not supported	misc
Q239	Linked list delete: handle head and middle cases	misc
Q241	Dynamic array allocated in heap, freed with free()	misc
Q243	Heap memory: manual management with malloc/free	misc
Q247	Pointer can point to any data type	misc
Q249	NULL pointer indicates no valid memory address	misc
Q250	Linked list traversal: while(head != NULL) loop	misc
Q461	malloc + pointer-arithmetic output (c=1,d=2)	A
Q463	free(v); v = NULL; to release dynamic memory	H
Q464	Static array: fixed size, allocated on stack	B

Q465	Hash table chaining search implementation correctness	misc
Q466	Uninitialized pointer: missing malloc before dereference	D
Q467	Linear probing search: correct when studName is key	misc
Q468	Pointer after free retains its heap address	C
Q470	Array-pointer output: v[0]=10, pv[0]=10	A
Q474	Pointer variable: special syntax, stores address	E
Q475	Linked list append: malloc + set next + NULL	misc
Q484	AVL right rotation: left child becomes root	misc
Q488	BST limitation: finding min in O(1) not supported	misc
Q492	Dynamic array allocated in heap, freed with free()	misc
Q493	Linked list delete: handle head and middle cases	misc
Q494	Heap memory: manual management with malloc/free	misc
Q498	Pointer can point to any data type	misc
Q500	NULL pointer indicates no valid memory address	misc
Q502	Linked list traversal: while(head != NULL) loop	misc
Q963	free(v); v = NULL; to release dynamic memory	H
Q965	Static array: fixed size, allocated on stack	B
Q966	Hash table chaining search implementation correctness	misc
Q968	Uninitialized pointer: missing malloc before dereference	D
Q969	malloc + pointer-arithmetic output (c=1,d=2)	A
Q970	Pointer after free retains its heap address	C
Q972	Array-pointer output: v[0]=10, pv[0]=10	A
Q976	Pointer variable: special syntax, stores address	E
Q977	Linked list append: malloc + set next + NULL	misc
Q978	Linear probing search: correct when studName is key	misc

Q981	Hash table chaining: allocate array of Nod* pointers	misc
Q983	malloc + pointer-arithmetic output (c=1,d=2)	A
Q985	Function returns head info, deletes first node	H
Q987	Static array: fixed size, allocated on stack	B
Q988	Hash table chaining search implementation correctness	misc
Q989	malloc() is the C dynamic allocation function	A
Q991	Uninitialized pointer: missing malloc before dereference	D
Q992	Linear probing search: correct when studName is key	misc
Q993	free(v); v = NULL; to release dynamic memory	H
Q995	Array-pointer output: v[0]=10, pv[0]=10	A
Q999	Pointer variable: special syntax, stores address	E
Q1000	Linked list append: malloc + set next + NULL	misc
Q1001	Pointer after free retains its heap address	C
Q1006	Hash table chaining: allocate array of Nod* pointers	misc
Q1007	Function returns head info, deletes first node	H
Q3938	Uninitialized pointer: missing malloc before dereference	D
Q3939	Pointer variable: special syntax, stores address	E
Q3940	Array-pointer output: v[0]=10, pv[0]=10	A
Q3941	malloc + pointer-arithmetic output (c=1,d=2)	A
Q3942	free(v); v = NULL; to release dynamic memory	H
Q3943	Pointer after free retains its heap address	C
Q3946	Static array: fixed size, allocated on stack	B
Q3949	Hash table chaining: allocate array of Nod* pointers	misc
Q3950	Linear probing search: correct when studName is key	misc
Q3955	Linked list append: malloc + set next + NULL	misc
Q3956	Function returns head info, deletes first node	H

12. Common Traps

The corpus has a consistent set of wrong-answer patterns. Memorize why each is wrong so you don't pick them.

✗ Wrong	Why it's wrong
"Static arrays are allocated at runtime"	Static arrays are allocated when the function is entered (effectively at compile time). Runtime allocation = heap.
"Memory size for static arrays is limited only by the HEAP size"	Static arrays are on the stack , which is much smaller (typically a few MB).
"List nodes are allocated in contiguous memory"	Each node is independently <code>malloc</code> 'd and lives at a random heap address. The list is defined by the pointers between nodes, not by physical contiguity.
"Accessing element <i>i</i> in a list is $O(1)$ "	Lists are sequential — you must walk from the head, which is $O(n)$.
" <code>free(p)</code> sets <i>p</i> to NULL"	<code>free</code> only releases the heap block. The pointer variable on the stack is unchanged. You must write <code>p = NULL;</code> yourself.
" <code>free(p)</code> deallocates the variable <i>p</i> "	The variable <code>p</code> is on the stack and is not "deallocated" by anything. Only the heap block it points to is returned.
" <code>delete</code> is the C function to free memory"	<code>delete</code> is C++. In C, the function is <code>free</code> .
"Pointer is constant after initialization"	Pointers are variables — they can be reassigned. (<code>const int*</code> is a pointer to a <code>const int</code> — different thing.)
"Pointer can only be used for heap access"	A pointer can hold any address — stack, heap, BSS, code segment, even a literal numeric address (though that last one is a bug).
"Dangling pointer is fine to read"	Reading a dangling pointer is undefined behavior. Always <code>p = NULL</code> after <code>free(p)</code> .

13. Quick-Recall Card

DEFINITION

Pointer = a variable that stores a memory address.
Dereferencing = going to that address and reading/writing the value there.

MEMORY REGIONS

Stack: local variables, auto-freed when function returns. $O(1)$ allocs.
Heap: `malloc`'d memory, freed by `free()`. Bigger but manual.
BSS: zero-initialized static/global vars.
Data: initialized static/global vars.
Code: the compiled instructions (read-only).

THE FOUR C FUNCTIONS YOU MUST KNOW

`malloc(n)` allocate *n* bytes, return `void*` to them
`calloc(count, sz)` allocate `count*sz` bytes, ZERO-initialize
`realloc(p, n)` resize block, may move it
`free(p)` return block to heap. *p* is NOT changed.

THE THREE POINTER BUGS

Uninitialized `int* p; *p = 10;` // *p* has garbage
Dangling `free(p); *p = 10;` // *p*'s address is now invalid
Memory leak `p = malloc(n);` // forgot to free

EXAM GOLD

- "List nodes are allocated at random addresses in the HEAP"
- "char `arr[10]` last valid index = 9"
- "free(*p*) does NOT set *p* to NULL"

- "Accessing element `i` in a list is $O(n)$, not $O(1)$ "
- "malloc is the C function; delete is C++"

WHEN TO USE

Pass-by-pointer when you need to modify the caller's variable or refer to large/dynamic data without copying.

\newpage

Chapter 2 — Bits, Memory Classes & User-Defined Types

Concept chapter. Underneath the Ch 1 pointer model: *where* each variable lives (storage classes), *how* its bits are inspected / manipulated (bitwise ops and bit fields), and *how* one memory region can be interpreted as different types (unions). The chapter has **0 MCQ corpus questions assigned to it directly** — bits-and-memory questions appear inside pointer / struct / `sizeof` questions in the Ch 1 and Ch 3 buckets. The patterns here are what make those questions solvable.

1. Learning Objectives

After reading this chapter, you will be able to:

- Explain the four storage class specifiers (`auto`, `static`, `extern`, `register`) and the storage location, initial value, scope, and lifetime of each.
- Predict the output of a C program that uses bitwise operators on signed and unsigned values, including the sign-changing effect of left-shifting into the sign bit.
- Define, declare, and access a struct with bit fields, and predict its `sizeof` and the order of fields in memory.
- Explain how a union overlaps all its members in the same memory region.
- Distinguish `typedef` from `#define` and recognize the trap that makes `INT_PTR a, b;` declare `a` as a pointer but `b` as a plain int.
- Read a binary file format (the BMP header) as a layout of fixed-offset fields.
- Use bitwise masks (`&`, `|`, `^`, `~`, `<<`, `>>`) to inspect, set, clear, and toggle bits.

2. Mental Model & Analogy

Think of a single byte of memory as a wall of eight light switches. **Bitwise operators** let you flip, combine, or rearrange those switches without thinking of the byte as a single number. **Bit fields** *name* some of those switches — "the first switch means `is_active`, the next three mean `priority`" — so your code can read `f.is_active` instead of doing manual masking. A **union** is one safe that holds *one* thing at a time but that thing can be re-labeled: a 4-byte safe can hold four dollar bills, or one 4-letter word, or one small integer — whatever you put in last is what's inside, and reading it under a different "type hat" is just a re-interpretation of the same 32 bits.

Storage class specifiers answer a different question: *where is the box?* Some boxes are on your desk (`auto`, `stack`) — gone when you leave the room. Some are in the building's archive (`static`, `data segment`) — permanent but you walk further. Some live in a filing cabinet in another building (`extern`) — you must announce you'll look first. Some are kept in your hand (`register`) — fastest access, but you can't have too many.

3. Memory Layout

Every C variable lives in one of four storage regions, determined by its storage-class specifier (or its absence — `auto` is the default for locals). The specifier decides **where** the compiler puts it, its default initial value, where it can be seen, and how long it lives.

Specifier	Storage	Initial value	Scope	Lifespan
-----------	---------	---------------	-------	----------

auto (default)	Stack	Garbage	Block	Until block exits
static (local)	Data	Zero	Block	Entire program
static (global)	Data	Zero	File (or external)	Entire program
extern (declaration)	Data	Per definition	Any file	Entire program
register (hint)	CPU reg	Garbage	Block	Until block exits

Key facts the exam will test:

- `auto` is the default — `int x;` inside a function is `auto` whether you say so or not. `auto int x;` is allowed but pointless.
- `static` inside a function does **not** mean "constant" — it means "stored in the data segment instead of the stack." Writable, but persists between calls.
- `static` on a **global** changes **linkage**: external (visible to other files) → internal (visible only in this file). Same keyword, different meaning.
- `extern` is a **declaration**, not a definition. `extern int x;` doesn't allocate memory. The actual `int x = ...;` (without `extern`) must appear in exactly one place.
- `register` is a *hint*; the compiler may ignore it. You cannot take the address of a register variable.

For the full memory map (Stack / Heap / BSS / Data / Code) see Ch 1, §3. The takeaway: the storage class you write determines **which** of those regions the variable lands in.

4. Definition & Properties

4.1 Bitwise operators

Six operators that work on the individual bits of integer operands. None have short-circuit or boolean-coercion semantics.

Op	Name	Rule	Identity
<code>&</code>	Bitwise AND	$1 \ \& \ 1 = 1$, else 0	$x \ \& \ 0 = 0$, $x \ \& \ \sim 0 = x$
<code>\ </code>	Bitwise OR	$0 \ \ \ 0 = 0$, else 1	$x \ \ \ 0 = x$, $x \ \ \ \sim 0 = \sim 0$
<code>^</code>	Bitwise XOR	same bits = 0, different = 1	$x \ ^ \ 0 = x$, $x \ ^ \ x = 0$
<code>~</code>	One's complement	flips every bit	$\sim x == -x - 1$ (two's complement)
<code><<</code>	Left shift	$x \ << \ n = x * 2^n$ (unsigned); sign-out bits discarded	fills with 0 on the right
<code>>></code>	Right shift	$x \ >> \ n = x / 2^n$ (logical for unsigned; arithmetic for signed, but implementation-defined)	fills with 0 (logical) or sign bit (arithmetic) on the left

4.2 Struct with bit fields

A struct member declared with `: width` packs an integer-typed member into `width` bits. The compiler groups bit fields into "storage units" (usually one `int`) in declaration order.

```
/* Example bit-field struct */
struct Example {
    unsigned int is_valid : 1;    /* named bit field */
    int         : 3;             /* anonymous padding */
};
```

Properties: `type` is `int` / `signed int` / `unsigned int` (sometimes `char` / `_Bool`); `width` is a non-negative integer constant no more than the bits in the type; an unnamed member (`int : 3;`) reserves padding; fields are accessed with normal `struct.member` or `->` syntax; `&f.priority` is a compile error; the packing order within a storage unit, and the behavior at a unit boundary, is **implementation-defined** — don't depend on it.

4.3 Union

A type whose members all **share the same memory region**. Size = size of the **largest** member. Reading from a member other than the last one written is "type punning" and is allowed in C.

```
/* Example union: all members share the same memory region */
union Data { int i; float f; char c; };
```

Properties: all members start at offset 0; writing one overwrites the bits of every other; reading a member that wasn't the most recently written is safe only if the bit pattern is a valid representation of the read type (e.g., `float` and `unsigned int` of the same size — **not** a `float*` and a `struct Foo*`). Canonical use: the `BYTEREGS` / `WORDREGS` trick for high / low bytes of a 16-bit register.

4.4 Enumeration

A list of named integer constants. The compiler assigns each name a successive `int` starting from 0 (or an explicit initializer); two enumerators may share a value (aliasing). An enum-typed variable may hold any `int` in C — not a strict type, just a naming convenience. Useful for `switch` labels, state machines, and bit-flag definitions.

4.5 typedef vs #define

Both create a new name for an existing type, but they work very differently — and the difference is one of the most-tested traps in C.

- `typedef int* int_ptr;` introduces `int_ptr` as a real type alias. `int_ptr a, b;` makes ***both*** `a` and `b` pointers to `int` (the alias names a complete type).
- `#define INT_PTR int*` is **text substitution**. `INT_PTR a, b;` is rewritten to `int* a, b;` — `a` is a pointer, `b` is a plain `int`.
- `typedef` is **scoped** (normal C scope); `#define` is **not** (survives from `#define` to end of file, or until `#undef`).
- `#define` can define *value* aliases and parameterized macros. `typedef` can only name types.

5. Operations — Step-by-Step Mechanics

5.1 Bitwise operator trace on a signed char

The canonical trace from the course PDF. Watch the sign bit flip when we shift left past bit 7.

```
int main() {
    signed char x = 7;
    x = x << 1; /* 14 */
    x = x << 3; /* 112 */
    x = x << 2; /* -64 (sign bit now 1) */
    x = x >> 1; /* -32 (arithmetic) */
    x = x >> 2; /* -8 (arithmetic) */

    return 0;
}
```

Step	Operation	Bits (8)	Decimal (signed)
start	<code>x = 7</code>	0000 0111	7

1	<code>x << 1</code>	0000 1110	14
2	<code>x << 3</code>	0111 0000	112
3	<code>x << 2</code>	1100 0000	-64
4	<code>x >> 1</code>	1110 0000	-32
5	<code>x >> 2</code>	1111 1000	-8

Exam trap #1: left-shifting a positive value enough times can flip it negative. After step 3, `x` is `-64`, not `+192` (the unsigned interpretation of the same bits). **Exam trap #2:** signed `>>` is **implementation-defined** in C. The trace assumes two's-complement arithmetic shift; on a logical-shift machine, step 4 would give `+64`. The exam usually assumes arithmetic.

5.2 Declaring a bit-field struct

```
struct Packet {
    unsigned int version : 4; /* 4 bits */
    unsigned int ttl      : 8; /* 8 bits */
    unsigned int protocol : 4; /* 4 bits */
    unsigned int checksum : 16; /* 16 bits */
};
```

The compiler groups these into `unsigned int` storage units. $4 + 8 + 4 + 16 = 32$ bits = exactly one `unsigned int` on a 32-bit-int system, so `sizeof(struct Packet)` is 4. Writing a field reads the storage unit, masks the appropriate bits, ORs in the new value, and writes it back — invisible to you, but the CPU never exposes sub-word access, so every bit-field access costs a read-mask-shift-write. Reordering the fields can change `sizeof` if it forces a boundary crossing, but the *values* stored are independent of field order.

5.3 Union memory overlap

```
#include <stdio.h>
int main() {
    union Word { short i; char c[2]; }; /* both 2 bytes, both at offset 0 */
    union Word w;
    w.i = 0x1234; /* writes 0x34 0x12 (LE) or 0x12 0x34 (BE) */
    printf("%c\n", w.c[0]); /* '4' on LE, '1' on BE */

    return 0;
}
```

The compiler allocates 2 bytes (the largest member); both `w.i` and `w.c` start at offset 0. Writing `w.i = 0x1234` writes two bytes — the low-order byte (`0x34`) lands at the lower address on LE, the higher on BE. Reading `w.c[0]` returns the byte at offset 0 — `0x34` on LE, `0x12` on BE. **The same code prints different characters on different machines** — the canonical endianness test and a clean demonstration of type punning through a union.

The classic **BYTEREGS / WORDREGS** variant (a union of two structs) is how the DOS `INT 21h` interface let C code read the low 8 bits of a 16-bit register via `r.h.al`, the high 8 bits via `r.h.ah`, and the full 16-bit value as `r.x.ax`.

6. Trace-Through Example

Reading the first 14 bytes of a BMP file ("Bitmap Header") as a struct — combining bit-field / union / binary-format ideas. (Course PDF uses a simplified layout; this is the actual 14-byte file header: signature 2B, reserved area = two 2B fields.)

Off	Size	Field	Example
0	2	File signature ("BM")	0x4D42
2	4	File size	0x46 (70)

6	2	Reserved 1	0x0000
8	2	Reserved 2	0x0000
10	4	Offset to pixel data	0x36 (54)

```
#include <stdio.h>
struct BMPHeader {
    unsigned short signature;    /* 2B */
    unsigned int   file_size;    /* 4B */
    unsigned short reserved1;    /* 2B */
    unsigned short reserved2;    /* 2B */
    unsigned int   data_offset;  /* 4B */
};
int main(void) {
    /* Simulate the first 14 bytes of a BMP file. */
    unsigned char raw[14] = {
        0x42, 0x4D, 0x46, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00,
        0x36, 0x00, 0x00, 0x00
    };
    struct BMPHeader* h = (struct BMPHeader*)raw;
    printf("signature=0x%04X size=%u offset=%u\n",
        h->signature, h->file_size, h->data_offset);
    return 0;
}
```

Output (LE): signature=0x4D42 size=70 offset=54.

Trace: the cast is legal because `sizeof(struct BMPHeader)` is exactly 14 on x86/x64/ARM. On LE, `h->signature` reads bytes 0..1 as 0x4D42, `h->file_size` reads bytes 2..5 as 70, `h->data_offset` reads bytes 10..13 as 54. **Exam point:** the struct definition is the spec of the binary format — same pattern as WAV, TIFF, PNG chunks, IP/TCP/UDP headers. If the compiler inserts padding bytes that aren't in the file, the layout breaks — which is why some parsers use `#pragma pack(1)` or `__attribute__((packed))`.

7. Operations + Big-O Table

For all operations in this chapter the unit of work is a machine word (4 or 8 bytes) — they all complete in constant time.

Operation	Complexity	Notes
<code>&, \, , ^, ~</code> on int	$O(1)$	One word read + one word write
<code><<, >></code> on int	$O(1)$	One word read + one word write
Read / write a bit-field member	$O(1)$	Read-modify-write of one storage unit
<code>sizeof(struct S) / sizeof(union U)</code>	$O(1)$	Compile-time constant
Access union member by name	$O(1)$	Same address, different interpretation

All $O(1)$. The chapter has no asymptotic surprises. The "cost" of bit fields and unions is hidden in code size and aliasing rules, not in runtime complexity.

8. C Code Reference

8.1 The four storage classes side by side

```
#include <stdio.h>
int g_initialized = 10; /* global, static, external linkage */
static int g_file_scope = 20; /* global, static, internal linkage */
```

```

extern int g_from_other;      /* declaration only - defined elsewhere */
void demo(void) {
    auto    int a = 1;        /* pointless 'auto' */
    static  int counter = 0;  /* local scope, storage = data segment */
    register int i;           /* &i would not compile */
    counter++; a++;
}
int main(void) {
    int i; for (i = 0; i < 3; i++) demo();
    return 0;
}

```

`g_initialized` and `g_file_scope` both live in the data segment; only linkage differs. `auto int a = 1;` is identical to `int a = 1;` — the `auto` is noise.

8.2 Bitwise operator showcase

```

#include <stdio.h>
int main(void) {
    unsigned char a = 0xCA, b = 0xB1;
    printf("a & b = %02X\n", a & b);          /* 0x80 */
    printf("a | b = %02X\n", a | b);          /* 0xFB */
    printf("a ^ b = %02X\n", a ^ b);          /* 0x7B */
    printf("~a = %02X\n", (unsigned char)~a); /* 0x35 */
    printf("a << 2 = %02X\n", (unsigned char)(a << 2)); /* 0x28 */
    printf("a >> 3 = %02X\n", a >> 3);        /* 0x19 */
    return 0;
}

```

Output: 0x80 0xFB 0x7B 0x35 0x28 0x19. **Exam trap:** the cast `(unsigned char)(a << 2)` is essential — without it, `a << 2` is computed as an `int` (0x2A8 on 32-bit) and `%02X` prints four hex digits.

8.3 Bit-field struct with : width syntax

```

#include <stdio.h>
struct Flags {
    unsigned int is_active : 1; unsigned int priority : 3;
    unsigned int mode      : 2; unsigned int reserved : 2;
};
int main(void) {
    struct Flags f = {1, 5, 2, 0};
    printf("sizeof=%zu a=%u p=%u m=%u\n",
        sizeof(struct Flags), f.is_active, f.priority, f.mode);
    return 0;
}

```

Output: sizeof=4 a=1 p=5 m=2. **Exam point:** even with only 8 bits of payload, `sizeof` is 4. The compiler rounds up to the storage unit (`sizeof(int) = 4`, or 8 on 64-bit).

8.4 Union showing overlapping memory

```

#include <stdio.h>
union Number { int i; float f; unsigned char bytes[4]; };
int main(void) {
    union Number n;
    n.i = 0x12345678;
    printf("0x%08X = %02X %02X %02X %02X\n", n.i,
        n.bytes[0], n.bytes[1], n.bytes[2], n.bytes[3]);
    n.f = 1.0f;
    printf("1.0f = %02X %02X %02X %02X\n",
        n.bytes[0], n.bytes[1], n.bytes[2], n.bytes[3]);
    return 0;
}

```

Output (LE): 0x12345678 = 78 56 34 12 and 1.0f = 00 00 80 3F. The first line shows `n.bytes` and `n.i` viewing the same 4 bytes; the second shows the *same* 4 bytes, reinterpreted as `float`, equal 1.0 — 00 00 80 3F is the IEEE 754 encoding of 1.0 in LE.

8.5 typedef vs #define

```
#include <stdio.h>
#define INT_PTR int* /* macro substitution */
typedef int* int_ptr; /* real type alias */
int main(void) {
    INT_PTR a, b; /* a=ptr, b=int */
    int_ptr c, d; /* c=ptr, d=ptr */
    printf("a=%zu b=%zu c=%zu d=%zu\n",
           sizeof(a), sizeof(b), sizeof(c), sizeof(d));
    return 0;
}
```

Output (64-bit): 8 4 8 8. The `*` from `INT_PTR` binds only to the first name; `typedef` has no such trap — the alias is a single name for a complete type.

9. ASCII Diagram

9.1 Bit-field layout in memory

`struct Flags` from §8.3 packs 8 bits of data into one unsigned `int` storage unit (24 bits padding).

+-----+ <- struct Flags starts here	
.....	is_active <-- 1 bit, value 1
.....	priority <-- 3 bits, value 5 (101b)
.....	mode <-- 2 bits, value 2 (10b)
.....	reserved <-- 2 bits, value 0
+-----+	
bits 31..8 unused	bits 7..0 used
(field order WITHIN a unit is implementation-defined)	

9.2 Union memory overlap

`union Number` from §8.4 — three members, all at offset 0 of the same 4 bytes.

+-----+ <- union Number starts here			
byte 0	byte 1	byte 2	byte 3
+-----+			
+-- n.bytes[0..3] (4 separate bytes)			
+-- n.i (one 4-byte int)			
+-- n.f (same 4 bytes, IEEE 754 float)			

All three names point to the same storage. Writing one writes all three; reading one reads the current bit pattern under that type interpretation.

9.3 BMP header byte layout

+-----+													
42	4D	46	00	00	00	00	00	00	00	36	00	00	00
+-----+													
off 0	2	4	6	8	10								
"BM"		70		rsv		rsv		54					

The `struct BMPHeader*` cast from §6 turns the 14 raw bytes into 5 named fields.

10. When to Use / When to Avoid

``static` inside a function` → value that must persist between calls (call counter, memoization cache). ``static` on a global` → "private" name visible only in this .c file.

``extern`` → accessing a global defined in another .c file; the `extern` declaration goes in a header, the actual definition (without `extern`) in exactly one .c file. `extern int x;` reserves no storage — it is a *declaration*, not an *allocation*.

``register`` → tight loop counter hint; redundant with modern optimizers. **Never** for a variable you need to take the address of.

Bit fields → map onto a hardware register or binary file format with sub-byte fields (network headers, partition tables, flag registers), or pack many small flags. **Avoid** when you need the field's address (illegal) or portable layout.

Unions → view the same memory under two type interpretations (BYTEREGS / WORDREGS, `float` ↔ `int` for IEEE 754) or build a tagged union. **Avoid** when the variants differ wildly in size or you need both at once (use a struct).

``enum`` → small set of named integer constants (colors, days, states, opcodes). ``typedef`` → real type alias that respects C scoping. ``#define`` → constant value or parameterized macro.

11. Exam Hot Spots

This chapter has **0 MCQ corpus questions assigned to it directly** — bits-and-memory questions appear inside pointer / struct / `sizeof` questions in the Ch 1 and Ch 3 buckets. The patterns below are the concept patterns the exam uses to test this material.

Pattern A: Storage class of a declared variable

Q: "Where is `static int counter;` inside a function stored, and what is its initial value?"

- **Correct:** Data segment, initial value zero.
- **Wrong:** "Stack because it's inside a function" — `static` overrides; or "Garbage" — only `auto/register` are uninitialized.

Pattern B: Bit-shift sign change

Q: "What is the value of `signed char x = 96; x << 2;`?"

- **Correct:** `96 = 0110 0000`, shifted left 2 = `1000 0000 = -128` as signed char (sign bit flipped).
- **Wrong:** "`96 * 4 = 384`" — true for unsigned, false for signed 8-bit.

Pattern C: typedef vs #define pointer trap

Q: "What does `#define INT_PTR int* INT_PTR a, b; declare?`"

- **Correct:** `int* a, b;` — `a` is a pointer, `b` is a plain int. The `*` binds only to `a`.
- **Wrong:** "Both `a` and `b` are pointers" (the trap). `typedef int* int_ptr; int_ptr c, d;` correctly declares both as pointers.

Pattern D: Union size and overlap

Q: "What is `sizeof(union U)` if `U` has members `int i, double d, char c`?"

- **Correct:** `max(sizeof(int), sizeof(double), sizeof(char)) = sizeof(double)` (typically 8).
- **Wrong:** "Sum of the member sizes" (that's a struct); "Always `sizeof(int)`" (it's the largest member).

Pattern E: Bit-field sizeof (Q102 from Ch 3)

Q: "What is the size of a struct with 4 bit fields totaling 8 bits?"

- **Correct:** Padded to the storage unit — `sizeof(int) = 4` (or 8 on 64-bit).
- **Wrong:** "1 byte" — the *data* fits in 1 byte, but the struct rounds up.

12. Common Traps

The exam has a consistent set of wrong-answer patterns. Memorize why each is wrong so you don't pick them.

✗ Wrong	Why it's wrong
"static means the variable cannot change"	<code>static</code> = stored in the data segment and persists; the value is still mutable.
" <code>register int i;</code> lets you do <code>&i</code> "	You cannot take the address of a register variable — it may not even be in memory.
" <code>extern int x;</code> allocates and initializes <code>x</code> to 0"	<code>extern</code> is a declaration , not a definition . No storage is allocated.
"A static local is re-initialized to 0 every call"	Static locals are initialized once at program startup; subsequent calls see the value left over.
"Bitwise <code>&</code> is the same as logical <code>&&</code> " (and <code>\ </code> vs <code>\ </code>)	<code>&</code> is bit-by-bit AND with no short-circuit; <code>&&</code> is boolean AND with short-circuit.
" <code>^</code> is exponentiation"	<code>^</code> is XOR. Exponentiation is <code>pow()</code> from <code><math.h></code> .
" <code>a << n</code> is the same as <code>a * 2^n</code> for signed integers"	For unsigned yes; for signed no — left-shift can flip the sign bit and is UB on overflow.
"Signed <code>>></code> always zero-fills"	Signed <code>>></code> is implementation-defined — typically arithmetic, but the standard allows logical shift.
"A union stores all its members simultaneously"	A union stores one member at a time. Writing one overwrites the bit pattern of all the others.
" <code>sizeof(union U)</code> is the sum of the member sizes"	It's the maximum member size, possibly rounded up for alignment.
"You can take the address of a bit-field member"	<code>&f.priority</code> is a compile error. The field lives inside a storage unit.
"Bit fields are laid out in the order I wrote them, on every compiler"	Order within a storage unit, and behavior at unit boundaries, are implementation-defined .
"Precedence: <code>&</code> and <code>\ </code> bind tighter than <code>==</code> "	<code>==</code> has higher precedence than <code>&</code> and <code>\ </code> . <code>x & y == z</code> parses as <code>x & (y == z)</code> . Always parenthesize.

13. Quick-Recall Card

DEFINITION

Bitwise operators (`&` `|` `^` `~` `<<` `>>`) work on the bits of an integer.
Bit field = struct member declared with `" : width "` to pack N bits.
Union = all members share the same memory region of size = max member.
Storage class = keyword that decides WHERE a variable lives and HOW LONG.

STORAGE CLASSES (initial value, scope, lifespan)

<code>auto</code>	stack,	garbage,	block-local,	until block exits
<code>static</code>	data,	zero,	block-local,	entire program (PERSISTS)
<code>static</code>	data,	zero,	file-scope,	entire program (global = static storage)
<code>extern</code>	data,	zero,	whole prog,	entire program (DECLARATION, not definition)
<code>register</code>	CPU reg,	garbage,	block-local,	until block exits (no <code>&</code> allowed)

BITWISE TRICK

`&` both 1 -> 1 `|` either 1 -> 1 `^` different -> 1 `~` flip every bit
`<<` multiply by 2^n (may flip sign) `>>` divide by 2^n
`a << 2 == a * 4` ONLY for unsigned. Signed overflow = undefined.

UNION RULE

`sizeof(union U)` = max member size (not the sum).

Writing one member overwrites all others. Use for type punning (float <-> int).

BIT FIELD RULE

: width packs an int member into 'width' bits.
sizeof(struct) rounds up to sizeof(storage unit) -- often 4 or 8 bytes.
You CANNOT take the address of a bit field. Order within a unit is implementation-defined.

TYPEDEF vs #define TRAP

```
#define INT_PTR int*    INT_PTR a, b;  -> a is int*, b is int    (TRAP)
typedef int* int_ptr;  int_ptr c, d;  -> c is int*, d is int*  (SAFE)
```

EXAM GOLD

- static local persists between calls (not on the stack)
- extern is a DECLARATION, not a definition
- left-shift a signed value can flip the sign bit (UB on overflow)
- union size = max member, NOT sum
- bit field address is illegal; order within a unit is implementation-defined
- & has LOWER precedence than == (parenthesize)

WHEN TO USE

static local	-> value that must survive between calls
static global	-> "private" name visible only in this .c file
extern	-> access a global defined in another .c file
union	-> view one storage region under different types
bit field	-> map onto a hardware register or a file format
enum	-> small set of named integer constants
typedef	-> real type alias (correct for "type a, b;")
#define	-> constants and bit-flag macros

\npage

Chapter 3 — Arrays

Foundation chapter. The array is the simplest and most-tested data structure in the corpus. Every other data structure in this course (linked lists, hash tables, heaps, stacks, queues) is built *on top of* the array. This chapter covers static and dynamic arrays in C: contiguous memory layout, indexing math, `sizeof` traps, vector expansion, multidimensional (row-major) arrays, and the canonical "last valid index" / "output of this access" question patterns.

45 MCQ corpus questions are assigned to this chapter — many of them are direct array questions, but several test array-using data structures (hash tables, heaps) that depend on this material.

1. Learning Objectives

After reading this chapter, you will be able to:

- Explain what "contiguous memory" means for a C array, and draw a memory map showing how `arr[i]` is located at `arr + i * sizeof(T)`.
- Distinguish a statically allocated array (stack) from a dynamically allocated array (heap), including the `free` responsibility.
- Predict the output of C code that reads or writes array elements by index, including 2D arrays in row-major order.
- Compute `sizeof(arr)` for a real array and `sizeof(p)` for a pointer-to-array, and explain why the two differ.
- Implement a vector-style dynamic array with capacity doubling, and justify the **amortized $O(1)$** cost of `push_back`.
- Identify the last valid index of an array of size `n` (`n - 1`), and recognize off-by-one and buffer-overflow bugs in C code.
- Read the multidimensional-array declaration grammar (only the leftmost dimension can be inferred; the rest must be specified).

2. Mental Model & Analogy

Think of a C array as a **row of numbered mailboxes** sitting side-by-side on a single shelf. Every mailbox is the same size (one `int`, one `char`, one `struct`, ...), the mailboxes are **touching** — there are no gaps between them — and they are numbered starting from 0. To find mailbox `i`, the post office doesn't have to walk along the shelf; it computes the address as `base + i * box_size` and goes straight there. That's why array access is $O(1)$ and why the phrase "random access" is used for arrays.

Two key mental rules:

1. **The row is fixed in size once built.** You can't add a 6th mailbox to a row of 5 — the shelf is too short. A static array has its size baked in at compile time. A dynamic array (vector) is a row that can be **replaced** with a longer row when it fills up, but at any instant the row has a fixed length.
2. **Every mailbox is the same type.** An array of `int` holds only `ints`, an array of `char` holds only `chars`, an array of `struct Node` holds only `struct Nodes`. This is what lets the post office compute mailbox `i` from a single base address: it knows the size of one mailbox in advance.

When you write `arr[i]`, the compiler does **not** check that `i` is in range. It just computes the address and reads or writes. Walk past the last mailbox and you read the neighbor's mail, walk far enough and you segfault. **In C, bounds are the programmer's job.**

3. Memory Layout

An array is a **contiguous block** of identical-typed elements. The block lives in **either** the stack (if statically declared inside a function) **or** the heap (if `malloc`'d). The bytes are touching — element `i+1` immediately follows element `i` in memory.

+-----+ HIGH ADDRESSES	
Stack	static arrays: <code>int v[5];</code>
[v[4]..v[0]]	
+-----+	
Heap	dynamic arrays: <code>int* v = malloc(...)</code>
[arr[0]..arr[n-1]]	
+-----+	
BSS / Data / Code	
+-----+ LOW ADDRESSES	

Key facts the exam will test:

- **Contiguous** = bytes adjacent, no gaps. Makes `arr[i]` computable in $O(1)$ and makes arrays cache-friendly.
- **Indexing math:** `arr[i]` is at address `arr + i * sizeof(*arr)`. The compiler multiplies `i` by the element size; you do not.
- **The array name `arr` is NOT a separate variable holding a pointer.** In most contexts, `arr` "decays" to a pointer to its first element (`&arr[0]`). The array itself is the storage, not a pointer variable on the stack.
- **Statically declared arrays** (`int v[5];` inside a function) live in the **stack** and are auto-freed. You do **not** call `free` on them.
- **Dynamically declared arrays** (`int* v = malloc(5 * sizeof(int));`) live in the **heap** and **must** be free'd.
- **Lifetime mismatch is the #1 exam trap:** `int v[5];` with no `free` is fine; `malloc` with no `free` is a **memory leak**.

What lives where — for the exam:

Declaration	Region	Lifetime	Who frees it?
<code>int v[5];</code> inside a function	Stack	Until function returns	Automatic
<code>int v[5];</code> at file scope	BSS/Data	Entire program	Never
<code>int* v = malloc(5 * sizeof(int));</code>	Heap	Until you call <code>free(v)</code>	You
<code>static int v[5];</code> inside a	BSS/Data	Entire program	Never

function			
----------	--	--	--

4. Definition & Properties

Array — a contiguous sequence of n elements, all of the same type T , occupying $n * \text{sizeof}(T)$ bytes total, accessed by integer index in the range $0..n-1$.

Properties:

- **Contiguous, zero-indexed, type-homogeneous, random-access $O(1)$, fixed size.** Address of `arr[i]` is `arr + i * sizeof(T)` — one multiply, one add, one memory access.
- **No bounds checking:** C does NOT validate the index. `arr[100]` on a 5-element array compiles and runs — the behavior is **undefined**; the program may crash, silently corrupt memory, or appear to work.
- **Array name decays to pointer:** in most contexts (function arguments, + arithmetic), `arr` becomes a T^* to the first element. Exceptions: `sizeof` operand, unary `&` operand, string-literal initializer.
- **`arr[i]` and `i[arr]` are the same thing.** `arr[i]` is defined as `*(arr + i)`; by commutativity, `i[arr]` is `*(i + arr)`.
- **Storage:** statically declared arrays live in stack or data segment; dynamically allocated arrays live in the heap (via `malloc/calloc/realloc`).

5. Operations — Step-by-Step Mechanics

5.1 Declaring a static array

```
int v[5];
```

The compiler reserves $5 * \text{sizeof}(\text{int}) = 20$ bytes on the **stack** in the current function's frame. The bytes are **uninitialized** — they contain whatever was left in that stack slot. `v[0]` is *not* `0` by default. Initialization forms: `int v[5] = {1,2,3,4,5};` (initialize all five), `int v[5] = {0};` (zero all five), `int v[] = {1,2,3};` (let the compiler count — length is 3).

5.2 Accessing an element

```
int main() {
    int v[5] = {10, 20, 30, 40, 50};
    int x = v[2];          /* x = 30 */

    return 0;
}
```

Compiler computes `&v[0] + 2 * sizeof(int)`, reads 4 bytes at that address, interprets as `int`. Same mechanics for `*(v + 2)`, `*(2 + v)`, `2[v]` — they all compute the same address.

5.3 The `sizeof` of an array vs the `sizeof` of a pointer

```
#include <stdio.h>
int main() {
    int v[5];
    int* p = v;
    printf("%zu\n", sizeof(v));    /* 20 (5 ints * 4 bytes) */
    printf("%zu\n", sizeof(p));    /* 4 or 8 - sizeof(int*), NOT the array */

    return 0;
}
```

`sizeof(v)` does **not** trigger array-to-pointer decay; it returns the whole array size. `sizeof(p)` returns the pointer size, regardless of what `p` points to. **This is the most common `sizeof` trap in the exam.**

(See §8 for full code on allocating dynamic arrays, vector expansion, passing to functions, and 2D arrays.)

6. Trace-Through Example

The canonical exam trace — a static array on the stack with a pointer aliasing its first element.

```
#include <stdio.h>
int main() {
    int v[5] = {1, 2, 3, 4, 5};
    int* p = v;
    int s = v[2] + p[3];
    v[0] = s;
    p[1] = v[4] - 2;
    printf("v[0]=%d v[1]=%d s=%d\n", v[0], v[1], s);
    return 0;
}
```

Line-by-line trace:

Line	Effect	State after (v, p, s)
<code>int v[5] = {1,2,3,4,5};</code>	Reserve 20 bytes on stack, fill with 1,2,3,4,5	<code>v = {1,2,3,4,5}, p = ?, s = ?</code>
<code>int* p = v;</code>	<code>v</code> decays to <code>&v[0]</code> ; copy into <code>p</code>	<code>v = {1,2,3,4,5}, p = &v[0], s = ?</code>
<code>int s = v[2] + p[3];</code>	<code>v[2]=3, p[3]=v[3]=4, sum=7</code>	<code>v = {1,2,3,4,5}, p = &v[0], s = 7</code>
<code>v[0] = s;</code>	Write 7 into <code>v[0]</code>	<code>v = {7,2,3,4,5}, p = &v[0], s = 7</code>
<code>p[1] = v[4] - 2;</code>	<code>v[4]=5, 5-2=3, write to p[1]=v[1]</code>	<code>v = {7,3,3,4,5}, p = &v[0], s = 7</code>
<code>printf(...);</code>	Print <code>v[0], v[1], s</code>	(unchanged)

Output: `v[0]=7 v[1]=3 s=7`

7. Operations + Big-O Table

Operation	Static array (stack)	Dynamic array (heap)	Vector (doubling)
Declare / allocate	$O(1)$ compile-time	$O(1)$ amortized	$O(1)$ amortized
Access by index <code>arr[i]</code>	$O(1)$	$O(1)$	$O(1)$
Linear search	$O(n)$	$O(n)$	$O(n)$
Insert at end	Impossible	$O(1)$	$O(1)$ amortized, $O(n)$ worst case
Insert at beginning	Impossible	$O(n)$	$O(n)$
Insert at index <code>i</code>	Impossible	$O(n)$	$O(n)$
Delete at end	Impossible	$O(1)$	$O(1)$
Delete at beginning	Impossible	$O(n)$	$O(n)$
Delete at index <code>i</code>	Impossible	$O(n)$	$O(n)$
Resize (grow)	Impossible	$O(n)$	$O(n)$ per resize
<code>sizeof(arr)</code>	$O(1)$ compile-time	$O(1)$	$O(1)$ compile-time

<code>free(arr)</code>	n/a (auto)	O(1) amortized	O(1) amortized
------------------------	------------	----------------	----------------

Key asymptotic facts:

- **Access is O(1).** Always — the entire reason arrays exist.
- **Search is O(n).** No structure to exploit. Binary search ($O(\log n)$) requires a **sorted** array.
- **Insert/delete in the middle is O(n)** because you shift remaining elements. This is the canonical "array is bad at insert/delete" fact, and the reason linked lists are introduced in Ch 4.
- **Vector `push\_back` is amortized O(1).** Doubling capacity ensures the total work of n pushes is $O(n)$, even though any individual push can be $O(n)$ in the worst case.

8. C Code Reference

8.1 Static array (stack)

```
#include <stdio.h>
int main() {
    int v[5] = {1, 2, 3, 4, 5};
    printf("v[3] = %d\n", v[3]);           /* prints 4 */
    printf("sizeof(v) = %zu\n", sizeof(v)); /* prints 20 (5 ints * 4 bytes) */

    return 0;
}
```

`sizeof(v)` is the **whole array** ($5 * \text{sizeof}(\text{int})$), not the pointer size. This works **only** because the compiler sees the actual array declaration in this scope.

8.2 Dynamic array (heap)

```
#include <stdlib.h>
#include <stdio.h>
int main() {
    int n = 5;
    int* v = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) v[i] = (i + 1) * 10;
    printf("v[3] = %d\n", v[3]);           /* prints 40 */
    free(v);
    v = NULL;                             /* defensive clear */

    return 0;
}
```

The `v[i]` syntax is identical to the static case. The `free(v); v = NULL;` pair is the canonical "I'm done with this heap block" pattern.

8.3 Vector expansion (capacity doubling)

```
#include <stdlib.h>
int* v;
int size = 0, capacity = 4;

void push_back(int x) {
    if (size == capacity) {
        capacity *= 2;
        int* bigger = (int*)malloc(capacity * sizeof(int));
        for (int i = 0; i < size; i++) bigger[i] = v[i];
        free(v); v = bigger;
    }
    v[size++] = x;
}

int main(void) {
    v = (int*)malloc(4 * sizeof(int));
```

```

push_back(10);
free(v);
return 0;
}

```

A resize doubles `capacity` and copies the existing elements. **Amortized $O(1)$ per push** (single push can be $O(n)$ at resize; average over a long sequence is $O(1)$). The exam expects the "amortized" qualifier.

8.4 2D array (row-major)

```

int m[2][3] = {{1, 2, 3},
               {4, 5, 6}};
/* m[0][0]=1, m[1][2]=6, sizeof(m)=24. Walked row-major: 1 2 3 4 5 6. */

```

Elements are contiguous in memory in row-major order. `&m[i][j] == (char*)m + (i * 3 + j) * sizeof(int)`. **The inner dimension (3) is the row stride and must be known at compile time; the outer (2) can be inferred from the initializer.**

9. ASCII Diagram

9.1 2D array `m[2][3]` in row-major memory layout

	<code>m[0][0]</code>	<code>m[0][1]</code>	<code>m[0][2]</code>	<code>m[1][0]</code>	<code>m[1][1]</code>	<code>m[1][2]</code>
	1	2	3	4	5	6
bytes:	01 00	02 00	03 00	04 00	05 00	06 00 (LE)
offset	0	4	8	12	16	20 (bytes from <code>m[0][0]</code>)

Memory view (1D, contiguous, one row = 3 ints = 12 bytes = row stride):

<code>m[0][0]</code> <code>m[0][1]</code> <code>m[0][2]</code> <code>m[1][0]</code> <code>m[1][1]</code> <code>m[1][2]</code>

The 2D array IS a 1D array; the second index is just a different stride. `m[1][0]` is at offset $1 * 3 * \text{sizeof(int)} = 12$ from `m[0][0]`; `m[1][2]` is at offset $1 * 3 * 4 + 2 * 4 = 20$. The compiler does this arithmetic on every access.

10. When to Use / When to Avoid

Use static arrays when the size is known at compile time, small, and matches a single function's lifetime — e.g., lookup tables, buffers, backing storage for a bounded stack or queue.

Use dynamic arrays (``malloc`` / ``free``) when the size is computed at runtime, the array must outlive the function that created it, or you're implementing a data structure that needs an array as backing storage (hash table buckets, heap elements, vectors).

Use a vector (capacity-doubling) when you need `push_back` amortized $O(1)$ and don't know the final size in advance (lines in a text editor, items in a shopping cart).

Avoid arrays when you need: frequent insert/delete in the middle (use linked lists, Ch 4), $O(\log n)$ search on a dynamic dataset (use a sorted array + binary search, or a BST), sparse data with large keys (use a hash table), or unbounded size (use linked list / tree / hash table).

11. Exam Hot Spots

This chapter has **45 MCQ corpus questions assigned to it**. Many are direct array questions; a substantial block tests array-using data structures (hash tables, heaps) that depend on array mechanics.

Pattern A: Last valid index of an array

Q25, Q89, Q126, Q208, Q459, Q962, Q984, Q3945: "If an array is declared as `char mesaj[10]`, what is the valid index for the last element of the array?"

- **Correct:** `'9'` (zero-based: indices 0 through $n-1$).
- **Wrong:** 10 (the *size*, not the last valid index), `mesaj + 1` (one-past-the-end — valid pointer value but not dereferenceable), `mesaj - 1` (non-dereferenceable), `last` (not a C keyword).

Pattern B: Output of an array access by index

Q41, Q107, Q1005, Q3944: "What value is printed by `int v[] = {1, 2, 3, 4, 5}; printf("%d", v[3]);?`"

- **Correct:** `'4'` — the 4th element (index 3).
- **Wrong:** 3 (that's `v[2]`), 5 (that's `v[4]`, the last), 9 (distractor).

Pattern C: Time complexity of access

Q1: "What is the time complexity of accessing an array element by index?"

- **Correct:** `'O(1)'` — the address is `arr + i * sizeof(T)`, one multiply and one add.
- **Wrong:** $O(\log n)$ (BST/heap index), $O(n)$ (linear search), $O(n \log n)$ (mergesort).

Pattern D: Insert near the beginning

Q2: "Which operation is expensive in a fixed-size array when inserting near the beginning?"

- **Correct:** "Shifting following elements to make room" — insert at index `i` requires shifting $n - i$ elements right.
- **Wrong:** "Reading the first element" / "Reading the array length" / "Comparing two indexes" (all $O(1)$ in an array).

Pattern E: Array vs linked list — which for which operation

Q4: "For efficient deletion of arbitrary nodes, the most suitable structure is:" — Answer: **doubly linked list**.

- **Correct:** a doubly linked list can delete a node in $O(1)$ once you have a pointer to it. An array must shift all following elements left, $O(n)$.
- **Wrong:** "Static array" (deleting from middle is $O(n)$), "Singly linked list" (deleting needs the *previous* node, $O(n)$ to find), "Circular singly linked list" (still $O(n)$ to find predecessor), "Binary heap" (irrelevant).

Q1968: "What is a linked list node typically composed of?" — Answer: **stored data plus one or more links to other nodes**.

Reinforces: lists are made of independently-`malloc`'d nodes at random heap addresses; arrays are one contiguous block.

Pattern F: 2D array declaration syntax

Q60: "Which of the following are valid ways to declare and/or initialize a statically allocated 2D array `m` in C?"

- **Correct:** `int mat[2][3] = {1, 2, 3, 4, 5, 6};` — flat initializer, compiler fills row-major: row 0 = {1,2,3}, row 1 = {4,5,6}.
- **Wrong:** `int mat[2][3] = { (1, 2, 3), {4, 5, 6};` (syntax error — parens not braces), `int mat[[3]] = {1, 2, 3}, {4, 5, 6};` ({} in wrong place), `int mat[2][] = {...};` (inner dimension required — it's the row stride), `int mat[][] = {...};` (both dims required to count initializer).

Pattern G: Dynamic array ("vector") operations

Q67: "What role does the parameter `int* nr` in the `saveflightsvector` function?" — Answer: **"Remembers the number of flights saved in the vector"** (an out-parameter, written through the pointer because the function cannot return two values).

- **Correct:** a C function returns one value. To return both the new array and the new count, the count is passed as a pointer; the function dereferences it and writes the new count into `*nr`.
- **Wrong:** "Remembers the address of the tree" (irrelevant), "Checks if year is valid" (a `bool` return suffices), "Stores the code of the last flight" (different out-parameter type).

Q15: "True statements about heaps" includes **"Efficiently implemented with arrays"**, **"Insertion has $O(\log n)$ complexity"**, and **"Can be either max-heap or min-heap"**. The first is the array link: a binary heap is a complete binary tree stored as a contiguous array, with $\text{parent}(i) = i/2$, $\text{left}(i) = 2i$, $\text{right}(i) = 2i+1$ computed by index.

Pattern H: Hash table chaining (array-of-lists pattern)

Q28, Q39, Q55, Q59, Q81, Q82, Q101, Q109, Q226, Q227, Q477, Q478, Q979, Q980, Q1002, Q1004, Q3947, Q3948: "What does chaining mean in a hash table?" / "Which data structure is used to implement chaining?"

- **Correct:** "Handling collisions through a list of elements" and "Singly linked list".
- **Wrong:** "Using an auxiliary array" (open addressing), "Computing a hash function" (the lookup, not resolution), "Duplicating data sets" / "Translating hash to ASCII" (nonsense).

Q229, Q231, Q482, Q483: "Each array slot contains a linked list of all keys that hash to that slot" / "Chaining uses an array of linked lists to handle collisions." A hash table *is* an array of buckets; each bucket is a (usually singly) linked list of entries that hashed to it.

Q3951: Search function — correct: **"the hash function is used followed by sequential search."** Compute the bucket via the hash, then walk the list at that bucket.

Why this is in the arrays chapter: a hash table is built on top of an array. You cannot understand the structure without array indexing.

Pattern I: Heap array representation

Q4003, Q4007, Q4011: "In a max-heap, the maximum element is at the root" / "HeapSort sorts a vector" / "Heap can be used as a priority queue."

- **Correct:** heap is a complete binary tree linearized into an array; max is at index 0; heapsort sorts an array; a heap implements a priority queue.
- **Wrong:** max at last child (no — at root, index 0), heapsort converts tree to vector (no — sorts a vector in place), heap is a stack (no — stack is LIFO; heap gives max/min in $O(1)$, insert/extract in $O(\log n)$).

Why this is in the arrays chapter: a heap is the canonical "store a tree in an array" — $\text{parent}(i) = i/2$, $\text{left}(i) = 2i$, $\text{right}(i) = 2i+1$. All of it depends on array indexing.

12. Common Traps

The exam has a consistent set of wrong-answer patterns. Memorize why each is wrong so you don't pick them.

✗ Wrong	Why it's wrong
"The last valid index of <code>T v[n]</code> is <code>n</code> "	Indices run <code>0..n-1</code> . <code>v[n]</code> is one-past-the-end — a valid pointer value, illegal to dereference.
" <code>arr[i]</code> and <code>i[arr]</code> are different"	<code>arr[i]</code> is defined as <code>*(arr + i)</code> , so <code>i[arr]</code> is <code>*(i + arr)</code> — the same expression, just unusual syntax.
" <code>sizeof(arr)</code> gives the array size even when <code>arr</code> is a pointer"	<code>sizeof</code> does NOT trigger array-to-pointer decay. If <code>arr</code> is <i>declared</i> as a pointer, <code>sizeof(arr)</code> is <code>sizeof(int*)</code> regardless of what it points to.
" <code>free(p)</code> sets <code>p</code> to NULL"	<code>free</code> releases the heap block. The pointer variable on the stack is

	unchanged. You must write <code>p = NULL;</code> yourself.
"delete is the C function to free memory"	delete is C++. In C, the function is free.
"Statically allocated arrays are allocated at runtime"	Static arrays are allocated when the function is entered (effectively at compile time). Runtime allocation = heap.
"Memory size for static arrays is limited only by the HEAP size"	Static arrays are on the stack , which is much smaller (typically a few MB). A 10 MB static array will likely overflow the stack.
"You can grow a static array with realloc"	realloc only works on malloc'd blocks. Static arrays have a fixed size baked in at compile time.
"You must cast the return of malloc in C"	The cast is optional in C (the conversion <code>void* → T*</code> is implicit). The cast is required in C++ .
"Array elements are scattered in memory"	No — that's a linked list. An array is a <i>contiguous</i> block; the defining property is adjacency.
"List nodes are allocated in contiguous memory"	No — each node is independently malloc'd at a <i>random</i> heap address. Lists are defined by the pointers between nodes, not by physical contiguity.
"Accessing element i in a list is O(1)"	Lists are sequential — you must walk from the head, which is O(n). Arrays are O(1).
"The first element of <code>int v[5]</code> is at index 1"	C arrays are 0-indexed. <code>v[0]</code> is the first element, <code>v[4]</code> is the last.
"Multidimensional arrays can be accessed in any order in memory"	C uses row-major order: row 0 is contiguous, then row 1, etc. The inner dimension is the row stride.
"A 2D array needs both dimensions specified even with an initializer"	Only the rightmost dimension is required — it's the row stride. <code>int m[][3] = {...};</code> is valid.
"Out-of-bounds array access is caught by the compiler"	C does NO bounds checking. <code>arr[100]</code> on a 5-element array compiles and runs — the behavior is undefined .
"Vector <code>push_back</code> is O(1) in the worst case"	The worst case of a single <code>push_back</code> is O(n) — when it triggers a resize. The <i>amortized</i> cost is O(1) when capacity doubles.

13. Quick-Recall Card

DEFINITION

Array = n elements of the same type T, stored contiguously, indexed 0..n-1.
`&arr[i] = arr + i * sizeof(T)`. Access is O(1).
 Last valid index = n - 1.

STATIC vs DYNAMIC

Static T v[n]; stack, auto-freed, size known at compile time
 Dynamic T* v = malloc(n * sizeof(T)); heap, YOU must free(v)
 Vector T* v = malloc(cap*sizeof(T)); int size, cap; resize by *2 when full

INDEXING MATH

`arr[i]` == `*(arr + i)` // both mean: element i, type T
`i[arr]` == `*(i + arr)` // SAME THING (commutativity)
`&arr[i]` == `(char*)arr + i*sizeof(T)`
`arr` == `&arr[0]` // array name decays to pointer to first elem

sizeof GOTCHA

T v[10]; sizeof(v) = 10 * sizeof(T) // array is real
 T* p = v; sizeof(p) = sizeof(T*) // pointer, not array
 In a function: f(T* p) -- sizeof(p) is sizeof(T*), NOT the array size.
 -> always pass length as a separate parameter

```

2D ARRAYS (row-major)
T m[R][C];  &m[i][j] = m + i*C*sizeof(T) + j*sizeof(T)
m[][C] = {...}    is legal (compiler counts rows)
m[][] = {...}    is ILLEGAL (need inner dim = row stride)
Storage: row 0 (all C elems), then row 1, then row 2, ...

VECTOR EXPANSION
size == capacity -> capacity *= 2; new = malloc(2*cap*sizeof(T));
copy old -> new; free(old); v = new;
Single push_back worst case: O(n). Amortized: O(1).

```

EXAM GOLD

- last valid index of T v[n] is n-1 (NOT n)
- v[3] of {1,2,3,4,5} is 4 (NOT 3, NOT 5)
- access by index is O(1) (NOT O(n))
- insert at beginning is O(n) (the shift dominates)
- static array is on the STACK, auto-freed, fixed size at compile time
- malloc'd array is on the HEAP, you must free()
- sizeof(arr) != sizeof(ptr) when array has decayed to pointer
- 2D array: only the LEFTMOST dim can be omitted in a declarator
- hash table with chaining = array of singly linked lists
- heap = complete binary tree stored as a contiguous array

BIG-O SUMMARY

Access	O(1)	Search (linear)	O(n)
Insert at end	O(1)*	Insert at front	O(n)
Delete at end	O(1)	Delete at front	O(n)
Resize (vector)	O(n) per resize, O(1) amortized with doubling		
*static array: insert/delete impossible (fixed size)			

\newpage

Chapter 4 — Linked Lists

Core chapter. Linked lists are the first dynamic data structure in this course. Every subsequent chapter (heaps, trees, hash tables, graphs) builds on the linked-list node pattern. This chapter has 40 corpus questions, making it the second-largest after pointers and arrays.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define the node structure for singly linked, doubly linked, and circular linked lists in C.
- Perform insertion, deletion, and search operations on each list variant and predict their time complexity.
- Explain why doubly linked lists are the best structure for efficient deletion of arbitrary nodes.
- Distinguish between O(1) operations (insert/delete at a known node) and O(n) operations (search by value, access by index).
- Trace through a linked-list C code snippet and predict its output, including head-passing semantics.
- Identify the memory overhead of a doubly linked list versus a singly linked list.
- Recognize and fix the three most common linked-list bugs: lost head pointer, dangling next pointer, and memory leak.

2. Mental Model & Analogy

A linked list is a treasure hunt. Each clue (node) tells you where to find the next clue (the `next` pointer). You start at the entrance (the `head` pointer) and follow clues one by one until you reach a dead end (`NULL`). You cannot skip ahead — you must read each clue to find the next one. That is why linked lists are **sequential access** structures.

A **doubly linked** list is a two-way street: each clue has both a "next" and a "previous" note pinned to it, so you can walk forward or backward without needing to retrace your steps. The cost is that each clue is physically bigger (it carries two pointers instead of one).

A **circular** list is a relay race with no finish line: the last runner hands the baton back to the first. The "dead end" is replaced by a pointer back to the head.

The fundamental trade-off versus arrays: arrays are like a row of numbered mailboxes — you can walk straight to mailbox #47. Linked lists are scattered across a huge warehouse — you must follow the chain from the entrance. Arrays give you $O(1)$ random access; linked lists give you $O(1)$ insertion and deletion **when you already hold a pointer to the target node**.

3. Memory Layout

Every node lives on the **heap** — individually `malloc'd`, at a random address chosen by the OS. The only thing connecting them is the chain of `next` pointers inside each node. This means:

- **No random access.** To reach node #500, you must walk through nodes 1–499 first.
- **No contiguous layout.** Node addresses can jump all over the heap: `0x5000`, `0x1A80`, `0x9F40`, ...
- **Each node costs extra memory** for its pointer(s): `sizeof(Node*)` bytes per pointer, on top of the data.

What the exam says about list nodes:

- "List nodes are allocated at random addresses in the HEAP memory" (Q13, Q72)
- "Nodes can be added or removed without knowing their count in advance" (Q34)
- "A singly linked list is a linear structure" (Q34, Q92)

4. Definition & Properties

Linked list — a sequence of nodes where each node contains (a) a data field and (b) a pointer to the next node. The list is accessed through a single pointer called `head`. The last node's `next` is `NULL`.

Singly linked list:

- Each node has one pointer: `next`.
- You can traverse forward only.
- Head is the only entry point.
- Tail's `next` is `NULL`.

Doubly linked list:

- Each node has two pointers: `next` and `prev`.
- You can traverse forward and backward.
- You can delete a node given only a pointer to that node (no need to find the predecessor).
- Costs one extra pointer per node (memory overhead).

Circular linked list:

- The last node's `next` points back to `head` (or to the first node).
- No `NULL` terminator.
- You can traverse indefinitely — must detect cycle to stop.
- Circular doubly linked list provides "easy traversal in both forward and backward directions indefinitely" (Q17).

Properties the exam tests (Q34, Q92, Q117, Q218, Q472, Q971, Q994, Q3952):

- A singly linked list is a **linear** structure.
- It is **dynamic** — nodes can be added or removed without knowing the count in advance.
- Elements are accessed **sequentially** (not randomly).

- Nodes are **homogeneous** — all nodes store the same type of data.
- It is NOT heterogeneous.
- It does NOT store "multiple values of the same type" per node (each node stores one data item plus one pointer).

5. Operations — Step-by-Step Mechanics

5.1 The node structure

```
/* Singly Linked List node */
struct Node {
    int data;
    struct Node* next;
};

/* Doubly Linked List node */
struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
```

5.2 Insert at head (singly linked, $O(1)$)

```
struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
struct Node {
    int data;
    struct Node* next;
};
#include <stdlib.h>
struct Node* insertHead(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head; /* new node points to old head */
    return newNode;      /* new node becomes the new head */
}
int main() {
    return 0;
}
```

Step 1. Allocate a new node on the heap.

Step 2. Set `newNode->next = head` — this is the critical line. The new node must point to the current first element, or you lose the rest of the list.

Step 3. Return `newNode` — the caller must update its head pointer to the return value.

Exam trap (Q49, Q54): If `head` is passed **by value** (not `struct Node** head`), the caller's `head` variable is NOT updated. The function works correctly only if the caller captures the return value: `head = insertHead(head, 5);`. Passing `head` by value is not a bug per se — it just means the function must return the new head.

5.3 Insert at tail (singly linked, $O(n)$ without tail pointer)

```
struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
```

```

struct Node {
    int data;
    struct Node* next;
};
#include <stdlib.h>
struct Node* insertTail(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) return newNode; /* empty list */
    struct Node* curr = head;
    while (curr->next != NULL) /* walk to last node */
        curr = curr->next;
    curr->next = newNode; /* attach at end */
    return head;
}
int main() {
    return 0;
}

```

Step 1. If the list is empty, the new node IS the list.

Step 2. Walk from `head` to the last node (where `next == NULL`). This is $O(n)$.

Step 3. Set the last node's `next` to the new node.

Exam pattern (Q77, Q38): "Choose the correct code block to add a node at the end of a singly linked list." The correct answer always: (a) creates a new node with `next = NULL`, (b) handles the empty-list case, (c) walks to the tail, (d) links the tail to the new node.

5.4 Insert after a known node (singly linked, $O(1)$)

```

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
struct Node {
    int data;
    struct Node* next;
};
#include <stdlib.h>
void insertAfter(struct DNode* prevNode, int value) {
    if (prevNode == NULL) return;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = prevNode->next; /* point to the node that was after prev */
    prevNode->next = newNode; /* prev now points to new node */
}
int main() {
    return 0;
}

```

This is $O(1)$ because you already have the predecessor pointer. This is the key insight behind Q10, Q240, Q491, Q1970.

5.5 Delete a node (singly linked, given predecessor, $O(1)$)

```

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
struct Node {

```

```

    int data;
    struct Node* next;
};
#include <stdlib.h>
void deleteAfter(struct Node* prevNode) {
    if (prevNode == NULL || prevNode->next == NULL) return;
    struct Node* toDelete = prevNode->next;
    prevNode->next = toDelete->next; /* skip over the node to delete */
    free(toDelete);
}
int main() {

    return 0;
}

```

Step 1. The predecessor's `next` is redirected to skip the target node.

Step 2. The skipped node is freed.

Exam fact (Q11, Q240, Q491): Deleting a node from a singly linked list, given the previous node, has complexity **O(1)**. This is because you don't need to search — you already hold the pointer to the predecessor.

5.6 Delete head (singly linked, O(1))

```

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
struct Node {
    int data;
    struct Node* next;
};
#include <stdlib.h>
struct Node* deleteHead(struct Node* head) {
    if (head == NULL) return NULL;
    struct Node* newHead = head->next;
    free(head);
    return newHead;
}
int main() {

    return 0;
}

```

5.7 Search (singly linked, O(n))

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
struct Node* search(struct Node* head, int target) {
    struct Node* curr = head;
    while (curr != NULL) {
        if (curr->data == target) return curr;
        curr = curr->next;
    }
    return NULL; /* not found */
}

```

```

}
int main() {

    return 0;
}

```

Exam fact (Q244, Q495, Q1969): Searching an unsorted singly linked list by value has worst-case complexity $O(n)$ — you must visit every node.

5.8 Doubly linked list insert at head ($O(1)$)

```

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
struct Node {
    int data;
    struct Node* next;
};
#include <stdlib.h>
struct DNode* insertHeadD(struct DNode* head, int value) {
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL)
        head->prev = newNode; /* old head's prev must point back */
    return newNode;
}
int main() {

    return 0;
}

```

5.9 Delete a node from a doubly linked list (given only the node, $O(1)$)

```

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
struct Node {
    int data;
    struct Node* next;
};
#include <stdlib.h>
void deleteDNode(struct DNode* node) {
    if (node == NULL) return;
    if (node->prev != NULL)
        node->prev->next = node->next; /* predecessor skips forward */
    if (node->next != NULL)
        node->next->prev = node->prev; /* successor skips backward */
    free(node);
}
int main() {

    return 0;
}

```

This is the key operation the exam tests most (Q85, Q125, Q211, Q462, Q961, Q982, Q3953): "For efficient deletion of arbitrary nodes, the best structure is: doubly linked list." With a singly linked list, you need the predecessor ($O(n)$ to find). With a doubly linked list, you already have `prev` in the node itself — $O(1)$.

6. Trace-Through Example

Build a list, insert, delete, then traverse.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* insertHead(struct Node* head, int value) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = value;
    n->next = head;
    return n;
}

void printList(struct Node* head) {
    struct Node* curr = head;
    while (curr != NULL) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;
    head = insertHead(head, 30);    /* List: 30 -> NULL */
    head = insertHead(head, 20);   /* List: 20 -> 30 -> NULL */
    head = insertHead(head, 10);   /* List: 10 -> 20 -> 30 -> NULL */
    printList(head);
    /* Output: 10 -> 20 -> 30 -> NULL */

    /* Delete head */
    struct Node* temp = head;
    head = head->next;    /* head now points to 20 */
    free(temp);
    printList(head);
    /* Output: 20 -> 30 -> NULL */

    return 0;
}
```

Line-by-line trace:

Line	Effect	State after
head = NULL	Empty list	head = NULL
insertHead(head, 30)	Allocate node at heap, next = NULL, data = 30	head -> [30, NULL]
insertHead(head, 20)	Allocate node, next = head (30), data = 20	head -> [20] -> [30] -> NULL
insertHead(head, 10)	Allocate node, next = head (20), data = 10	head -> [10] -> [20] -> [30] -> NULL
printList(head)	Walk: 10, 20, 30, NULL	prints 10 -> 20 -> 30 -> NULL
temp = head	Save pointer to node 10	temp -> [10]

<code>head = head->next</code>	Advance head to node 20	<code>head -> [20] -> [30] -> NULL</code>
<code>free(temp)</code>	Deallocate node 10	node 10 gone
<code>printList(head)</code>	Walk: 20, 30, NULL	prints 20 -> 30 -> NULL

Key insight: `free(temp)` does NOT change `head` — `head` was already updated to `head->next` before the `free`. If you reversed the order (`free` first, then read `head->next`), you'd access freed memory.

7. Operations + Big-O Table

Operation	Singly	Doubly	Notes
Insert at head	$O(1)$	$O(1)$	Just allocate and point
Insert at tail	$O(n)$	$O(1)^*$	* $O(1)$ if tail pointer maintained
Insert after known node	$O(1)$	$O(1)$	Q10, Q240, Q491
Delete head	$O(1)$	$O(1)$	Redirect head, free old
Delete given predecessor	$O(1)$	$O(1)$	Q11: "given the previous node"
Delete given only the node	$O(n)$	$O(1)$	Q85, Q125, Q211: doubly linked is best
Search by value	$O(n)$	$O(n)$	Must walk the list
Access by index	$O(n)$	$O(n)$	No random access
Get list length	$O(n)$	$O(n)$	No stored length field (unless maintained)
Traverse entire list	$O(n)$	$O(n)$	Forward: both; backward: doubly only

Exam critical fact (Q244, Q495, Q1969): Searching an unsorted singly linked list by value has worst-case complexity **$O(n)$** — not $O(\log n)$, not $O(1)$.

8. C Code Reference

8.1 Singly linked list — full implementation

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int value) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = value;
    n->next = NULL;
    return n;
}

struct Node* insertHead(struct Node* head, int value) {
    struct Node* n = createNode(value);
    n->next = head;
    return n;
}
```

```

void insertAfter(struct Node* prev, int value) {
    if (!prev) return;
    struct Node* n = createNode(value);
    n->next = prev->next;
    prev->next = n;
}

struct Node* deleteHead(struct Node* head) {
    if (!head) return NULL;
    struct Node* next = head->next;
    free(head);
    return next;
}

void deleteAfter(struct Node* prev) {
    if (!prev || !prev->next) return;
    struct Node* target = prev->next;
    prev->next = target->next;
    free(target);
}

void printList(struct Node* head) {
    for (struct Node* c = head; c; c = c->next)
        printf("%d -> ", c->data);
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;
    head = insertHead(head, 30);
    head = insertHead(head, 20);
    head = insertHead(head, 10);
    printList(head);           /* 10 -> 20 -> 30 -> NULL */
    insertAfter(head->next, 25); /* insert 25 after 20 */
    printList(head);           /* 10 -> 20 -> 25 -> 30 -> NULL */
    head = deleteHead(head);
    printList(head);           /* 20 -> 25 -> 30 -> NULL */
    return 0;
}

```

8.2 Doubly linked list — insert and delete

```

#include <stdio.h>
#include <stdlib.h>

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};

struct DNode* insertHeadD(struct DNode* head, int value) {
    struct DNode* n = (struct DNode*)malloc(sizeof(struct DNode));
    n->data = value;
    n->prev = NULL;
    n->next = head;
    if (head) head->prev = n;
    return n;
}

void deleteDNode(struct DNode* node) {
    if (!node) return;
    if (node->prev) node->prev->next = node->next;
    if (node->next) node->next->prev = node->prev;
}

```

```

    free(node);
}

void printListD(struct DNode* head) {
    for (struct DNode* c = head; c; c = c->next)
        printf("%d <-> ", c->data);
    printf("NULL\n");
}

int main() {
    struct DNode* head = NULL;
    head = insertHeadD(head, 30);
    head = insertHeadD(head, 20);
    head = insertHeadD(head, 10);
    printListD(head);           /* 10 <-> 20 <-> 30 <-> NULL */
    deleteDNode(head->next);    /* delete 20, given only its pointer */
    printListD(head);           /* 10 <-> 30 <-> NULL */
    return 0;
}

```

8.3 Circular singly linked list

```

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};

struct Node {
    int data;
    struct Node* next;
};

#include <stdlib.h>
struct Node* insertHeadCircular(struct Node* tail, int value) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = value;
    if (!tail) {
        n->next = n;    /* points to itself - one-node circle */
        return n;
    }
    n->next = tail->next; /* new node points to old head */
    tail->next = n;       /* tail now points to new head */
    return tail;         /* tail is unchanged */
}

int main() {
    return 0;
}

```

Exam fact (Q70, Q12): In a circular singly linked list, the last node's `next` points to the **first node** (not NULL).

8.4 The head-by-value pattern (Q49, Q54)

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};

struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};

#include <stdlib.h>
/* Note: head is passed BY VALUE.
   The caller must capture the return value. */

```

```

struct Node* insertHeadByValue(struct Node* head, int value) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = value;
    n->next = head;
    return n;
}

int main() {
    struct Node* head = NULL;
    head = insertHeadByValue(head, 10);  /* correct: capture return */
    /* WRONG: insertHeadByValue(head, 20); - return value discarded,
       head pointer NOT updated, 20 is lost */
    return 0;
}

```

9. ASCII Diagram

Singly linked list — three nodes scattered in heap:

```

head
|
v
+---+---+ +---+---+ +---+---+
| 10| o---->| 20| o---->| 30|NULL|
+---+---+ +---+---+ +---+---+
0x5000      0x1A80      0x9F40

```

Doubly linked list — with prev pointers:

```

head
|
v
+---+---+---+ +---+---+---+ +---+---+---+
| 10| o | NULL|<---| 20| o | o |---->| 30| NULL| o |
+---+---+---+ +---+---+---+ +---+---+---+
| ^          | ^          | ^
+---+        +---+        +---+
(prev)       (prev)       (prev)

```

Circular singly linked list:

```

      tail
      |
      v
+---+---+ +---+---+ +---+---+
| 10| o---->| 20| o---->| 30| o--+
+---+---+ +---+---+ +---+---+ |
^                                     |
+-----+
(last node's next = first node)

```

Delete from doubly linked list (given only the node):

BEFORE:

```

... <-> [A] <-> [B] <-> [C] <-> ...
        ^       ^
      prev    next

```

AFTER deleteDNode(B):

```

... <-> [A] <-> [C] <-> ...
        ^       ^
      A.next=C   C.prev=A
      (B is freed)

```

10. When to Use / When to Avoid

Use linked lists when:

- You need frequent insertion/deletion at known positions (especially at the head — $O(1)$).
- The data set size is unpredictable and may grow/shrink dramatically.
- You don't need random access by index.
- You're implementing another data structure (stack, queue, hash bucket, adjacency list for graphs).

Avoid linked lists when:

- You need random access by index (use arrays).
- Memory overhead per element matters (each node costs an extra pointer — or two for doubly linked).
- You need cache-friendly sequential access (arrays are better for cache locality).
- You frequently need to search by value ($O(n)$ — consider a sorted array or hash table).

Doubly linked vs singly linked — when to choose doubly:

- When you need $O(1)$ deletion of an arbitrary node given only a pointer to it (Q85, Q125, Q211, Q462, Q961, Q982, Q3953).
- When you need backward traversal.
- The trade-off: one extra pointer per node (typically 4 or 8 bytes of overhead per node).

11. Exam Hot Spots

This chapter has **40 MCQ corpus questions**. They cluster into 7 patterns.

Pattern A: Singly linked list properties

Q34, Q92, Q117, Q218, Q472, Q971, Q994, Q3952 (8 questions)

All are variations of: "Which statements are true about singly linked lists?"

- **Correct:** Linear structure, dynamic (nodes added/removed without knowing count), sequential access, homogeneous nodes.
- **Wrong:** "Nodes store heterogeneous data" (false — all nodes store the same type), "Each node stores multiple values of the same type" (false — each node stores one data value and one pointer).
- **Note:** Q92/Q117 are in Romanian (grile1.txt) but test the exact same facts as Q34 in English.

Pattern B: Doubly linked list for efficient deletion

Q85, Q125, Q211, Q462, Q961, Q982, Q3953 (7 questions)

All are variations of: "For efficient deletion of arbitrary nodes, the best structure is:"

- **Correct:** Doubly linked list.
- **Wrong:** Singly linked list (needs predecessor — $O(n)$ to find), circular singly linked list (still needs predecessor), binary heap (wrong operation entirely), static array (no dynamic deletion).

Pattern C: $O(1)$ operations with known node reference

Q10, Q240, Q248, Q491, Q499, Q1970 (6 questions)

"Operations on linked lists with $O(1)$ complexity (assuming direct node reference):"

- **Correct:** Insertion at known position, deletion at known position.
- **Wrong:** Sorting, search, traversal (all $O(n)$).

Pattern D: Search complexity

Q244, Q495, Q1969 (3 questions)

"Worst-case search complexity in an unsorted singly linked list:"

- **Correct:** $O(n)$.
- **Wrong:** $O(1)$, $O(\log n)$, $O(n \log n)$.

Pattern E: Doubly linked list advantages

Q8, Q66, Q21 (3 questions)

"Which data structure allows traversing in both forward and backward directions easily?"

- **Correct:** Doubly linked list.
- **Wrong:** Stacks, queues, circular linked list (forward only), simple linked list (forward only).

Q21: "Main disadvantage of doubly linked lists:"

- **Correct:** Increased memory overhead.
- **Wrong:** Difficult traversal, difficult random access (both are disadvantages of singly linked lists, not doubly), unpredictable node addresses.

Pattern F: Code comprehension (image-based)

Q38, Q49, Q54, Q77 (4 questions)

These questions show a code snippet image. Common patterns:

- "Choose the correct code to add a node at the end" — requires: create node with `next = NULL`, handle empty list, walk to tail, link.
- "What does this function do?" — the correct answer identifies both (a) what the function returns (useful info from head) and (b) that it deletes the container/first node.

Pattern G: Other

Q7, Q17, Q18, Q22, Q50, Q65, Q3970, Q3972 (8 questions)

- Q7: Advantages of linked lists over heaps: "Able to preserve the order of the elements", "Easier insertion/deletion operations at known positions."
- Q17: Circular doubly linked list provides "both forward and backward directions indefinitely."
- Q18: Properties of binary heaps: "Complete binary tree structure", "Heap property for all nodes."
- Q22: Primary use of a min-heap: "Accessing smallest element quickly."
- Q50: Linked list advantages over heap: simplified dynamic memory management, easier insertions/deletions at known positions.
- Q65: Singly linked list best for: "sequential access and frequent modification."
- Q3970: Non-recursive tree traversal uses a **stack** (not a linked list — but this tests the relationship).
- Q3972: Worst-case search in a binary tree is the same as a **singly linked list** (both $O(n)$).

12. Common Traps

✗ Wrong	Why it's wrong
"Linked list nodes are contiguous in memory"	Each node is individually <code>malloc'd</code> at a random heap address. The pointers connect them, not physical adjacency.
"Searching a linked list is $O(1)$ "	Must walk the list from head — $O(n)$. Only arrays give $O(1)$ random access.
"Deleting from a singly linked list is always $O(n)$ "	Given the predecessor pointer, deletion is $O(1)$ (Q11, Q240, Q491). It's $O(n)$ only if you must find the predecessor first.
"Doubly linked lists have the same memory overhead as singly"	Each node has an extra pointer (4-8 bytes). For small data, this can

	double the per-node cost.
"A circular linked list's last node points to NULL"	It points to the first node (head). No NULL terminator exists.
"Passing head by value to a function updates the caller's head"	It does NOT. The function receives a copy of the pointer. The caller must capture the return value (Q49, Q54).
"A linked list can grow to fill all available heap memory"	True in principle, but each node is a separate <code>malloc</code> — fragmentation can prevent large allocations even when total free heap is sufficient.
"Freeing a node automatically sets the pointer to NULL"	<code>free</code> does NOT change any pointer. You must set it to NULL yourself to avoid a dangling pointer.
"A singly linked list stores heterogeneous data"	All nodes store the same type (homogeneous). The "heterogeneous" option is always wrong (Q34, Q92, Q117, Q218, Q472).
"Each node stores multiple values of the same type"	Each node stores ONE data value and ONE (or two) pointer(s). The "multiple values" option is always wrong.

13. Quick-Recall Card

DEFINITION

Linked list = sequence of nodes, each containing data + pointer(s).
Singly: next only. Doubly: next + prev. Circular: last -> first.

NODE STRUCT (singly)

```
struct Node { int data; struct Node* next; };
```

INSERT AT HEAD — $O(1)$

```
newNode->next = head;
head = newNode;
```

INSERT AFTER KNOWN NODE — $O(1)$

```
newNode->next = prev->next;
prev->next = newNode;
```

DELETE GIVEN PREDECESSOR — $O(1)$

```
prev->next = target->next;
free(target);
```

DELETE GIVEN ONLY THE NODE — $O(1)$ doubly, $O(n)$ singly

```
node->prev->next = node->next;
node->next->prev = node->prev;
free(node);
```

SEARCH — $O(n)$

Walk from head, compare each node's data.

COMPLEXITY CHEAT SHEET

```
Insert at head:      O(1)
Insert at tail:      O(n) singly, O(1) with tail ptr
Insert after known node: O(1)
Delete given predecessor: O(1)
Delete arbitrary node:  O(n) singly, O(1) doubly
Search by value:      O(n)
```

EXAM GOLD

- Doubly linked list is best for $O(1)$ deletion of arbitrary nodes
- Search is always $O(n)$ for unsorted lists
- Passing head by value does NOT update the caller's head
- List nodes are homogeneous, linear, dynamic
- Circular list last node points to FIRST (not NULL)
- Doubly linked disadvantage: extra memory per node

\newpage

Chapter 5 — Stacks & Queues

Concept chapter. Two closely related linear structures. The chapter has only **2 MCQ corpus questions** (Q19, Q64), both concept-driven. Master LIFO vs FIFO, the four operations, the two implementations of each, and the "stack" / "heap" naming collisions.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define a **stack** as a LIFO linear structure and list its core operations — push, pop, peek — each $O(1)$.
- Define a **queue** as a FIFO linear structure and list its core operations — enqueue, dequeue, front, rear — each $O(1)$.
- Explain the **array-based** stack (single `top` index) and **linked-list-based** stack (`head = top`).
- Explain the **circular-buffer** array queue (`head/tail` with mod-arithmetic) and the head/tail pointer pattern of a linked-list queue.
- Match each structure to its use cases: stack for function calls, undo, expression evaluation, DFS; queue for BFS, task scheduling, buffering.
- Recognize the two exam question patterns: "right data structure for this use case" (heap / stack / queue) and "where in memory does this live" (stack segment).

2. Mental Model & Analogy

Stack = a stack of plates. You can only touch the **top** plate. Add a plate, you put it on top (push). Take a plate, you take the top one (pop). The plate placed most recently is the one that comes off first — **Last-In, First-Out**.

Queue = a line at the movies. Newcomers join the **back**; the next person served is the one who has been waiting longest — at the **front**. First one in is first one out — **First-In, First-Out**.

The two structures differ in **one** rule: which end is the "active" one. A stack is active at one end only (the top). A queue is active at two ends — you add at the tail and remove at the head. Everything else (linear, homogeneous, $O(1)$ insert/remove) follows from that one rule.

3. Memory Layout

Both a stack and a queue are **logical** structures — what changes between implementations is the physical storage they sit on.

Array-based (contiguous storage): all elements in a single block (good for cache, bad for size changes). Stack needs one `top` index. Queue needs **two** indices (`head`, `tail`) and the standard form is a **circular buffer** so vacated slots at the front can be reused (§9.3).

Linked-list-based (scattered storage): each node is a **separate heap allocation** (Ch 1 §3, Ch 4). The list is defined by **pointers**, not physical contiguity. List stack: `head = top`; push/pop are head insertion/removal, both $O(1)$. List queue: keep **two** pointers; enqueue appends after `tail` and advances it, dequeue advances `head`.

	Array-based	Linked-list-based
Storage	Contiguous block	Scattered heap nodes
Size	Fixed at allocation	Grows on demand
Stack ops	<code>top</code> index ++/--	Insert / remove at head
Queue ops	<code>head/tail</code> wrap mod-N	<code>head/tail</code> pointer advance
Cache	Friendly	Unfriendly (pointer-chasing)

4. Definition & Properties

Stack — a linear structure in which both insertion (push) and deletion (pop) happen at a **single end** called the **top**. Rule: **LIFO** — the most recently pushed element is the next to be popped.

Queue — a linear structure in which insertion happens at the **tail** (also *back* or *rear*) and deletion happens at the **head** (also *front*). Rule: **FIFO** — the element that has been in the queue longest is the next to be dequeued.

Properties shared by both: linear, homogeneous, restricted access (only the top of a stack, or the head/tail of a queue, are reachable), $O(1)$ insert/remove.

	Stack	Queue
Order	LIFO	FIFO
Active ends	One (top)	Two (head, tail)
Insert	Push at top	Enqueue at tail
Remove	Pop from top	Dequeue from head
Inspect	peek	front, rear
Empty	top == -1 (array) / head == NULL (list)	head == tail (array) / head == NULL (list)
Bad op	Pop empty → underflow	Dequeue empty → underflow
Full	Push full → overflow	Enqueue full → overflow

Naming trap #1: the *Stack* data structure (LIFO) and the *Stack* memory region (the call-stack segment — Ch 1 §3) are different things that share a name. Q64 is the canonical example. **Naming trap #2:** the *Heap* data structure (Ch 7) and the *Heap* memory region (Ch 1 §3) also share a name. Q19 tests that collision.

5. Operations — Step-by-Step Mechanics

5.1 Stack: array-based

State: `int data[MAX]` array, `int top = -1` (empty). `top` names the element at that position.

- `isEmpty(s)` — `return top == -1`.
- `push(s, x)` — `if top == MAX - 1 → overflow; else s->data[++s->top] = x`.
- `pop(s)` — `if top == -1 → underflow; else return s->data[s->top--]`.
- `peek(s)` — `return s->data[s->top]` (no state change).

5.2 Stack: linked-list-based

State: `Node* top` initially `NULL`. `Node = { int info; Node* next; }`.

- `push(&top, x)` — `n = malloc(...); n->info = x; n->next = *top; *top = n`.
- `pop(&top)` — `if *top == NULL → underflow; else x = (*top)->info; tmp = *top; *top = (*top)->next; free(tmp); return x`.
- `peek(top)` — `return top->info` (no state change).

The `Node**` is required so `pop` can change the caller's `top` pointer (Ch 1 §5.6).

5.3 Queue: linked-list-based

State: `Node* head` (dequeue from here) and `Node* tail` (enqueue here), both initially `NULL`.

- `enqueue(q, x) — n = malloc(...); n->info = x; n->next = NULL. If q->tail == NULL (empty): q->head = q->tail = n. Else: q->tail->next = n; q->tail = n.`
- `dequeue(q) — if q->head == NULL → underflow; else x = q->head->info; tmp = q->head; q->head = q->head->next; if q->head == NULL q->tail = NULL; free(tmp); return x.`
- `front(q) / rear(q) — return q->head->info / q->tail->info.`

5.4 Queue: array-based (circular buffer)

State: `int data[MAX], int head (next to dequeue), int tail (next free slot), both start at 0. One slot is sacrificed to disambiguate full from empty.`

- **Empty:** `head == tail. Full: (tail + 1) % MAX == head.`
- `enqueue(q, x) — if full → overflow; else q->data[q->tail] = x; q->tail = (q->tail + 1) % MAX.`
- `dequeue(q) — if empty → underflow; else x = q->data[q->head]; q->head = (q->head + 1) % MAX; return x.`

The mod-arithmetic (`% MAX`) is what makes the buffer "circular" — `tail` at `MAX - 1` wraps to `0` on the next enqueue, reusing a slot vacated by an earlier dequeue.

6. Trace-Through Example

6.1 Array stack: push 1, 2, 3 then pop

State: `data[MAX=5], top = -1.`

Step	Op	Returns	State
1	<code>push(s, 1)</code>	—	<code>top = 0; [1, _, _, _, _]</code>
2	<code>push(s, 2)</code>	—	<code>top = 1; [1, 2, _, _, _]</code>
3	<code>push(s, 3)</code>	—	<code>top = 2; [1, 2, 3, _, _]</code>
4	<code>peek(s)</code>	3	<code>top = 2 (unchanged)</code>
5	<code>pop(s)</code>	3	<code>top = 1; [1, 2, _, _, _]</code>
6	<code>pop(s)</code>	2	<code>top = 0; [1, _, _, _, _]</code>
7	<code>pop(s)</code>	1	<code>top = -1 (empty)</code>

The last element pushed (3) is the first one popped. That is LIFO.

6.2 Linked-list queue: enqueue 1, 2, 3 then dequeue

State: `head = NULL, tail = NULL.`

Step	Op	Returns	State
1	<code>enqueue(q, 1)</code>	—	<code>head -> 1 -> NULL <- tail</code>
2	<code>enqueue(q, 2)</code>	—	<code>head -> 1 -> 2 -> NULL <- tail</code>
3	<code>enqueue(q, 3)</code>	—	<code>head -> 1 -> 2 -> 3 -> NULL <- tail</code>

4	dequeue (q)	1	head -> 2 -> 3 -> NULL <- tail
5	dequeue (q)	2	head -> 3 -> NULL <- tail
6	dequeue (q)	3	'tail = NULL' reset; empty

The first element enqueued (1) is the first one dequeued. That is FIFO. **Step 6:** when the last element is dequeued, `tail` must be reset to `NULL` — forget this and a later `enqueue` dereferences a freed/dangling `tail`. The most-tested list-queue trap.

7. Operations + Big-O Table

Operation	Stack	Queue	Complexity
Insert	push at top	enqueue at tail	O(1)
Remove	pop from top	dequeue from head	O(1)
Inspect top/head	peek	front	O(1)
Inspect rear	(n/a)	rear	O(1)
isEmpty	one comparison	one comparison	O(1)
Access middle	not allowed	not allowed	— (needs O(k) pops)
Search by value	not supported	not supported	—

No asymptotic surprises. Every standard operation is O(1) in both implementations. The exam sometimes asks you to recognize that a proposed operation is *not* O(1) because it tries to access the middle of the structure (e.g., "find the k-th element" requires O(k) pops).

8. C Code Reference

8.1 Array-based stack:

```
#include <stdio.h>
#define MAX 100
struct Stack { int data[MAX]; int top; };
void push(struct Stack* s, int x) {
    if (s->top == MAX - 1) return;
    s->data[++s->top] = x;
}
int pop(struct Stack* s) {
    if (s->top == -1) return -1;
    return s->data[s->top--];
}
int peek(struct Stack* s) { return s->top == -1 ? -1 : s->data[s->top]; }
int isEmpty(struct Stack* s) { return s->top == -1; }
```

8.2 Linked-list-based stack:

```
#include <malloc.h>
struct Node { int info; struct Node* next; };
void push(struct Node** top, int x) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->info = x; n->next = *top; *top = n;
}
int pop(struct Node** top) {
    if (*top == NULL) return -1;
    struct Node* tmp = *top; int x = tmp->info;
    *top = tmp->next; free(tmp);
    return x;
}
```

```
}
```

8.3 Array-based circular queue (EMPTY: head==tail; FULL: (tail+1)%MAX==head — one slot wasted):

```
#include <stdio.h>
#define MAX 100
struct Queue { int data[MAX]; int head, tail; };
void enqueue(struct Queue* q, int x) {
    if ((q->tail + 1) % MAX == q->head) return;
    q->data[q->tail] = x; q->tail = (q->tail + 1) % MAX;
}
int dequeue(struct Queue* q) {
    if (q->head == q->tail) return -1;
    int x = q->data[q->head]; q->head = (q->head + 1) % MAX;
    return x;
}
```

8.4 Linked-list-based queue (head + tail):

```
#include <malloc.h>
struct Node { int info; struct Node* next; };
struct LQueue { struct Node* head; struct Node* tail; };
void enqueue(struct LQueue* q, int x) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->info = x; n->next = NULL;
    if (q->tail == NULL) q->head = q->tail = n;
    else { q->tail->next = n; q->tail = n; }
}
int dequeue(struct LQueue* q) {
    if (q->head == NULL) return -1;
    struct Node* tmp = q->head; int x = tmp->info;
    q->head = q->head->next;
    if (q->head == NULL) q->tail = NULL;
    free(tmp);
    return x;
}
```

The `q->tail = NULL` reset in §8.4 is the most-skipped step in textbook list-queue code, and the most-tested detail.

8.5 Parenthesis matching with a stack — the canonical "use a stack" exam example. LIFO == nesting: the last open bracket must be the first one closed.

```
int is_balanced(const char* s) {
    char stk[256]; int top = -1;
    for (int i = 0; s[i]; i++) {
        char c = s[i];
        if (c=='(' || c=='[' || c=='{') stk[++top] = c;
        else if (c==')' || c==']' || c=='}') {
            if (top < 0) return 0;
            char open = stk[top--];
            if ((c==')' && open!='(') || (c==']' && open!='[') || (c=='}' && open!='{')) return 0;
        }
    }
    return top == -1;
}
/* is_balanced("([{}])") = 1 ; is_balanced("([)])" = 0 */
```

9. ASCII Diagram

9.1 Array stack with `top` index:

+	+	+	+	+	+	+	+	+	+	+	+	+
	1		2		3							
+	+	+	+	+	+	+	+	+	+	+	+	+

top = 2 (names the last valid element)
push(s,4)->top=3 pop(s)->top=2

0 1 2 3 4 5 6

9.2 Linked-list stack (head = top):

```
top -> [3|*] -> [5|*] -> [7|*] -> [9|NULL]
push(s, 1) inserts a new node before the current top:
top -> [1|*] -> [3|*] -> [5|*] -> [7|*] -> [9|NULL]
```

9.3 Circular queue (MAX = 8) — head = next to dequeue, tail = next free slot, $\text{tail} = (\text{tail} + 1) \% \text{MAX}$ wraps. Index 0 is the "sacrificed slot" that distinguishes full from empty.

Empty (h == t):	enq 2,4,1,3:	deq 2x (returns 2,4):
+---+---+---+---+ +---+---+---+---+ ^ h=t	+---+---+---+---+ 2 4 1 3 +---+---+---+---+ ^ ^ head tail	+---+---+---+---+ 1 3 +---+---+---+---+ ^ ^ head tail

9.4 Linked-list queue (head + tail):

head v	tail v
+---+---+ 1 * --->	+---+---+ 2 * --->
+---+---+ 3 * --->	+---+---+ 4 NULL
enqueue(q,5) -> [1]->[2]->[3]->[4]->[5 NULL] dequeue frees head, advances head.	
When the last element leaves, also set tail = NULL.	

10. When to Use / When to Avoid

Use a stack when: matching **nesting** (parentheses, brackets, function calls — LIFO == innermost first); **reversing** a sequence; **undo**; **DFS** (push neighbours, pop the most recent); **expression evaluation** (infix→postfix, stack-machine); **call-stack** management (push on call, pop on return).

Use a queue when: needing **fairness / FIFO ordering**; **BFS** (enqueue neighbours, dequeue the oldest unvisited first); **task scheduling**, print spooling, producer/consumer buffers; **buffering** between a fast producer and a slow consumer.

Avoid both when: you need **middle access** in $O(1)$ (use array or hash table); you need **search by value** (use hash table, BST, sorted array); you need the **min or max** in $O(1)$ (use a heap, Ch 7); you need **arbitrary middle insert/remove** at $O(1)$ (use a doubly linked list, Ch 4).

11. Exam Hot Spots

This chapter has only **2 MCQ corpus questions** (Q19, Q64). Both are **concept-driven** — they test the *idea* of LIFO/FIFO/stack-as-memory-region, not the implementation mechanics. This is the chapter where you get the most points for the least code-reading.

Q19 — "Suitable for heap usage" (multi-select)

What it tests: the **heap data structure** (Ch 7), not the *heap memory region* (Ch 1) and not the *stack/queue* of this chapter. The exam reuses the word "heap" in three unrelated senses. The chapter title is misleading — the question is really about the **priority queue / heap** concept.

- **Correct:** "Heap sort", "Priority queue", "Maintaining dynamic min/max", "Scheduling applications based on priority" — all need a structure that finds the min or max quickly.
- **Wrong:** "Direct indexing of arbitrary elements" — that is a *hash table* (Ch 8) or a plain array (Ch 3). You can only inspect the root.

Q64 — "Which memory segment is used for allocating local variables?"

What it tests: the **Stack memory region**, not the Stack data structure. The five choices (BSS, Stack, Data, Heap, Code) are the five memory regions of a C process (Ch 1 §3). Local variables are `auto` by default (Ch 2 §4) and live on the **stack** segment.

- **Correct: Stack.**
- **Wrong:** BSS / Data (static and global variables); Heap (`malloc` data); Code (compiled instructions).
- **Naming trap:** "stack segment" always means the memory region, never the ADT — but they share a name because the call stack is a stack data structure.

Other chapters that test stack / queue indirectly

Most applications appear elsewhere: **Ch 6/8 (Trees/Graphs)** — DFS uses a stack, BFS uses a queue. **Ch 1/4 (Pointers/Linked Lists)** — the list implementation is the canonical exercise in pointer manipulation and `malloc/free` discipline. **Ch 7 (Heaps)** — Q19 is logically a heap question but lives in this bucket.

The exam's strategy: test the *concept* in this chapter (Q19, Q64) and test the *implementation* in the pointer/list chapters.

12. Common Traps

✗ Wrong	Why it's wrong
"Pop on an empty stack returns 0"	Empty stack is underflow — an error condition, not a return value.
"Push on a full array stack wraps around to slot 0"	Array stacks don't wrap — a full array stack is overflow . Circular wrap is for <i>queues</i> , not stacks.
"Queue is FIFO so the head is the most recently enqueued"	FIFO: head is the oldest (next to dequeue); tail is the newest . "Front" / "back" also mean head / tail.
"List queue: <code>enqueue</code> walks the list to find the tail"	That would be $O(n)$. The whole point of the <code>tail</code> pointer is $O(1)$ enqueue.
"List queue: after <code>dequeue</code> of the last element, <code>tail</code> is still set to the freed node"	If the dequeue empties the queue, you MUST also set <code>tail = NULL</code> . The canonical list-queue bug.
"Circular buffer is full when <code>head == tail</code> "	No — <code>head == tail</code> is empty . Full is $(tail + 1) \% MAX == head$ (one slot sacrificed).
"Stack overflow means the data on the stack is corrupted"	"Stack overflow" in a data-structures context means the stack ADT ran out of room. The "stack overflow" runtime error (recursive explosion) is a different phenomenon.
"LIFO and FIFO are interchangeable, just different naming"	They are opposite disciplines. A stack used as a queue gives you a stack, not a queue.
"The peek operation changes the state of the stack"	Peek is a non-destructive read. It does NOT decrement <code>top</code> and does NOT free anything.
"Recursion is unrelated to stacks"	Every recursive call is a push of a new frame onto the call stack; every return is a pop .
"You can use a stack to implement a queue in $O(1)$ per operation"	Each <code>enqueue</code> on a stack-backed queue costs $O(n)$. True $O(1)$ needs a real queue (or two stacks with amortized $O(1)$).
"You can search for a value in $O(1)$ in a stack or queue"	Neither structure supports search; the API gives you push/pop or enqueue/dequeue only.

13. Quick-Recall Card

DEFINITION

Stack = LIFO linear structure. push and pop at one end (the top).
Queue = FIFO linear structure. enqueue at tail, dequeue at head.

STACK OPS (all $O(1)$) push, pop, peek, isEmpty

Errors: pop on empty = UNDERFLOW, push on full = OVERFLOW

QUEUE OPS (all $O(1)$) enqueue (put), dequeue (get), front, rear, isEmpty

Errors: dequeue on empty = UNDERFLOW, enqueue on full = OVERFLOW

IMPLEMENTATIONS

Stack array : int data[MAX]; int top = -1; push = ++top; pop = top--;
Stack list : head of list = top of stack; push/pop at head
Queue list : head + tail pointers; enqueue at tail, dequeue at head
Queue array : circular buffer;
EMPTY when head == tail;
FULL when (tail+1)%MAX == head; one slot sacrificed

USE CASES

Stack function calls, undo, DFS, parenthesis matching, expression evaluation
Queue BFS, task scheduling, print spooling, producer/consumer buffers

EXAM GOLD

- LIFO = stack, FIFO = queue. Don't mix them up.
- All stack/queue ops are $O(1)$ in the standard implementations.
- Circular array queue: one slot wasted so EMPTY (head==tail) and FULL ((tail+1)%MAX==head) are distinguishable.
- List queue: when the last element is dequeued, set tail = NULL too.
- "Stack" the data structure and "stack" the memory segment share a name. Q64 tests the segment, not the ADT.
- "Heap" the data structure (Ch 7) and "heap" the memory region (Ch 1) are also different. Q19 tests the data structure.

WHEN TO USE

Stack -> LIFO order, nesting, reversal, DFS, undo, expression evaluation
Queue -> FIFO order, fairness, BFS, buffering, task scheduling

\newpage

Chapter 6 — Hash Tables

Performance chapter. Hash tables combine an array with a hash function to achieve average $O(1)$ insert, search, and delete. The exam tests: hash-function properties, load factor, collision-resolution mechanics (chaining vs open addressing), rehashing, and Big-O analysis. 15 MCQ corpus questions are assigned to this chapter.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define a hash table as an array of buckets indexed by a hash function, and explain why it achieves $O(1)$ average-case access.
- Implement a hash function for integer and string keys and explain why determinism and uniform distribution matter.
- Trace through insert, search, and delete for both chaining (linked-list buckets) and open addressing (linear probing).
- Compute the load factor and predict when rehashing is necessary.
- Distinguish between chaining and open addressing, including the collision and clustering pitfalls of each.
- Explain why worst-case hash table performance is $O(n)$ and what causes it.
- Identify the exam's standard wrong-answer patterns for hash-table questions.

2. Mental Model & Analogy

Think of a hash table as a row of mailboxes, each labeled with a number from 0 to $m-1$. You want to store letters (values) and retrieve them later by a name (key). A **hash function** is the mail clerk: given a name, it calculates which mailbox number to use. The clerk is fast -- $O(1)$ -- because it only does arithmetic, not a search.

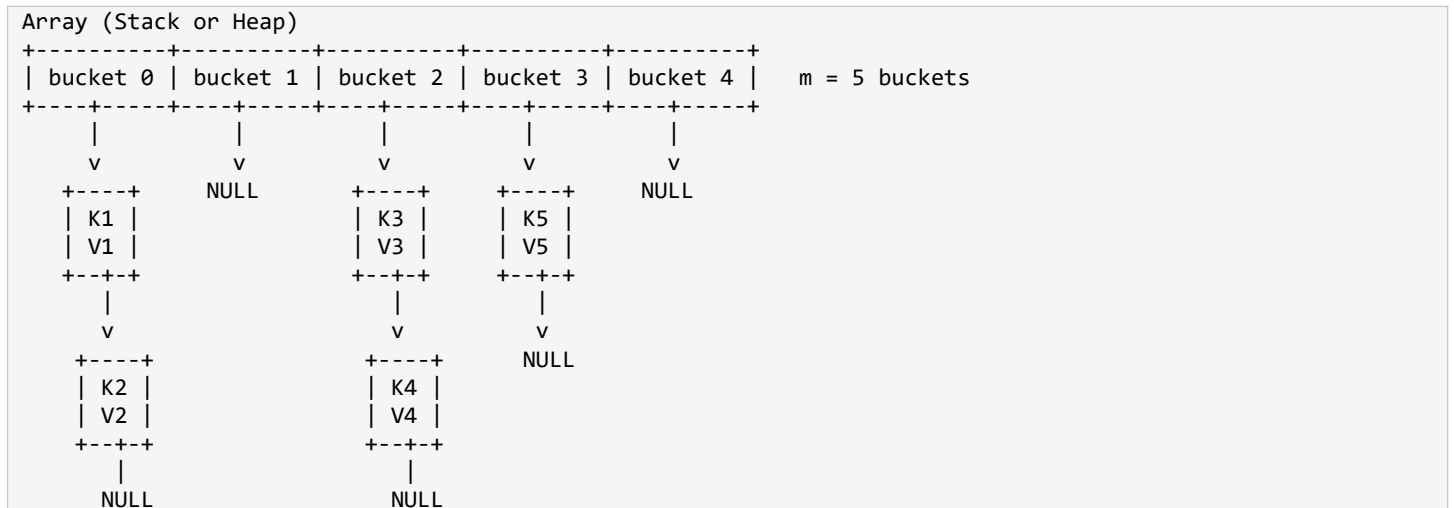
Sometimes two people get assigned the same mailbox. That is a **collision**. Two solutions exist:

1. **Chaining:** Each mailbox has a small pegboard attached. When two letters land on the same box, both hang on the same pegboard as a linked list. To find your letter, the clerk goes to the right box, then walks the pegboard.
2. **Open addressing:** There are no pegboards. When a box is full, the clerk tries the next box, then the next, until an empty one is found. The clerk keeps a note of which boxes were checked so the letter can be retrieved later.

The **load factor** is the ratio of letters to mailboxes. When it gets too high (most boxes are full), the clerk spends more time searching for an empty box. The solution: rent a bigger row of mailboxes and re-sort all the letters -- that is **rehashing**.

3. Memory Layout

A hash table with chaining is an array of bucket pointers. Each bucket is a linked list of nodes stored on the heap.



- The **array** (the bucket heads) lives on the stack if declared as `Node* table[m]`, or on the heap if `Node** table = malloc(m * sizeof(Node*))`.
- Each **node** in a chain is heap-allocated via `malloc` and contains the key, the value, and a `next` pointer.
- For **open addressing**, the array stores the records directly (no linked lists). A special sentinel value (e.g., `EMPTY_KEY = -1`) marks vacant slots.

4. Definition & Properties

Hash table -- a data structure that maps keys to values using a **hash function** $h(\text{key}) = \text{index}$ to compute an array position. An array of size m (the **bucket array**) stores the data either directly (open addressing) or via linked lists (chaining).

Hash function -- a deterministic function $h: \text{Key} \rightarrow \{0, 1, \dots, m-1\}$. A good hash function satisfies:

Property	Meaning
Deterministic	Same key always produces the same index
Uniform distribution	Keys spread evenly across all m buckets
Efficient to compute	$O(1)$ time for practical key types

Collision -- occurs when two distinct keys $k_1 \neq k_2$ satisfy $h(k_1) = h(k_2)$. Collisions are inevitable when the key space exceeds the table size.

Collision resolution methods:

Method	Idea	Tradeoff
Chaining	Each bucket holds a linked list of all keys that hash to it	Extra pointer overhead per node; always works

Linear probing	If bucket $h(k)$ is full, try $h(k)+1$, $h(k)+2$, ... (mod m)	Simple, but suffers from primary clustering
Quadratic probing	Try $h(k)+1^2$, $h(k)+2^2$, $h(k)+3^2$, ... (mod m)	Reduces clustering; may not visit all buckets
Double hashing	Use a second hash function: probe at $h_1(k) + i \cdot h_2(k)$ (mod m)	Best distribution; more computation

Load factor -- $\alpha = n / m$ where n is the number of keys and m is the number of slots. When α exceeds a threshold (typically 0.7 for open addressing), **rehashing** creates a larger array (usually $2m+1$ or next prime) and re-inserts all existing keys.

Amortized $O(1)$ -- individual operations are $O(1)$ on average. Rehashing is $O(n)$ but happens rarely enough that the amortized cost per operation remains constant.

5. Operations — Step-by-Step Mechanics

5.1 Insert with chaining

```
INSERT(table, m, key, value):
1. Compute index = h(key) mod m
2. Walk the linked list at table[index]
3. If key already found -> update value, return
4. Otherwise -> prepend new node (key, value) to the list at table[index]
5. Increment count n
6. If n/m > threshold -> rehash (double m, re-insert all keys)
```

Step-by-step for inserting key=25 into a table of size $m=7$ with $h(k)=k \bmod 7$:

1. $h(25) = 25 \bmod 7 = 4$
2. Check bucket 4. If empty, create node (25, val) and set bucket 4 to point to it.
3. If bucket 4 already has nodes, walk to the end and append, or prepend (both valid).

5.2 Search with chaining

```
SEARCH(table, m, key):
1. Compute index = h(key) mod m
2. Walk the linked list at table[index]
3. If a node with matching key is found -> return its value
4. If end of list reached (NULL) -> key not present, return NOT_FOUND
```

Time: $O(1)$ average if the load factor is low and the hash function distributes well. $O(n)$ worst case when all keys hash to the same bucket (one long list).

5.3 Delete with chaining

```
DELETE(table, m, key):
1. Compute index = h(key) mod m
2. Walk the linked list at table[index], keeping a pointer to the previous node
3. If found:
   a. If it is the head of the list -> table[index] = node->next
   b. Otherwise -> prev->next = node->next
   c. free(node), decrement count n
4. If not found -> no-op
```

5.4 Insert with linear probing

```
INSERT_LINEAR(table, m, key, value):
1. Compute index = h(key) mod m
2. If table[index] is EMPTY -> place (key, value) there, return
3. Probe sequentially: index = (index + 1) mod m
4. Repeat step 2-3 until an empty slot is found or we return to start
5. If returned to start -> table is full, error
```

Primary clustering problem: When many keys cluster around the same initial index, probe sequences overlap and grow long. A run of occupied slots causes every key hashing into that run to traverse the entire run.

5.5 Search with linear probing

```
SEARCH_LINEAR(table, m, key):
1. Compute index = h(key) mod m
2. Probe: check table[index]
   a. If EMPTY -> key not present, return NOT_FOUND
   b. If key matches -> return value
   c. Otherwise -> index = (index + 1) mod m, go to step 2
3. If we wrap around to the starting index -> not found
```

Exam trap: You must stop at an EMPTY slot, not just when you find the key. An EMPTY slot proves the key was never inserted. A DELETED sentinel does NOT terminate the search in standard linear probing.

5.6 Rehashing

```
REHASH(table, old_m):
1. Allocate new_table with new_m slots (typically 2 * old_m + 1 or next prime)
2. For each bucket i in old_table:
   walk the chain (or scan all occupied slots for open addressing)
   INSERT each (key, value) into new_table
3. Free old_table
4. Set table = new_table, m = new_m
```

Rehashing is $O(n)$ because every key is re-inserted. It is triggered when α exceeds the threshold. The amortized cost of rehashing across all subsequent inserts is $O(1)$ per operation.

6. Trace-Through Example

Setup: Hash function $h(k) = k \bmod 7$, table size $m = 7$, chaining.

Insert the following 6 keys: 10, 22, 31, 4, 15, 28.

Step	Key	$h(\text{key}) = \text{key} \bmod 7$	Bucket	Action
1	10	$10 \bmod 7 = 3$	3	Bucket 3 empty -> insert (10)
2	22	$22 \bmod 7 = 1$	1	Bucket 1 empty -> insert (22)
3	31	$31 \bmod 7 = 3$	3	Collision with 10 -> append (31) to list at bucket 3
4	4	$4 \bmod 7 = 4$	4	Bucket 4 empty -> insert (4)
5	15	$15 \bmod 7 = 1$	1	Collision with 22 -> append (15) to list at bucket 1
6	28	$28 \bmod 7 = 0$	0	Bucket 0 empty -> insert (28)

Final table state:

```
Index 0: [28] -> NULL
Index 1: [22] -> [15] -> NULL
Index 2: NULL
Index 3: [10] -> [31] -> NULL
Index 4: [4] -> NULL
Index 5: NULL
```

Search for key 31:

1. $h(31) = 31 \bmod 7 = 3$
2. Go to bucket 3. First node is 10 -- no match. Follow next pointer.
3. Second node is 31 -- match found. Return value.

Load factor: $\alpha = n/m = 6/7 = 0.857$

This is above 0.7 -- in practice, a rehash would be triggered soon.

Complexity of the search for 31: $O(1)$ hash computation + 2 list comparisons = $O(1) + O(\text{chain length})$. With good hashing, average chain length is α .

7. Operations + Big-O Table

Chaining

Operation	Average Case	Worst Case	Notes
Insert	$O(1)$	$O(n)$	Prepend to list at bucket $h(k)$
Search	$O(1)$	$O(n)$	Walk one chain; average chain length = α
Delete	$O(1)$	$O(n)$	Find in chain, unlink node
Rehash	$O(n)$	$O(n)$	Re-insert all n keys into new table

Average cases assume: uniform hash function, α stays bounded.

Linear Probing

Operation	Average Case	Worst Case	Notes
Insert	$O(1)$ amortized	$O(n)$	Probe until empty slot found
Search	$O(1)$ amortized	$O(n)$	Probe until key found or EMPTY slot
Delete	$O(n)$	$O(n)$	Must shift or mark DELETED to preserve probe chains

Quadratic probing avoids primary clustering but may not visit all buckets (only guarantees visiting $m/2$ buckets if m is prime and $\alpha < 0.5$). **Double hashing** visits all buckets and has the best probe distribution but costs two hash computations per probe.

Exam point: Chaining tolerates higher load factors than open addressing. Open addressing degrades badly above $\alpha = 0.7$ and requires rehashing; chaining can work with $\alpha > 1$ (more keys than slots) but chains grow longer.

8. C Code Reference

8.1 Hash table struct (chaining)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 7
```

```

struct Node {
    int key;
    int value;
    struct Node* next;
};

struct HashTable {
    struct Node* buckets[TABLE_SIZE];
    int count;
};

```

8.2 Hash function

```

int hash_function(int key, int m) {
    int h = key % m;
    if (h < 0) h += m;    /* handle negative keys */
    return h;
}

```

For string keys, the course material defines:

```

int hash_string(const char* s, int m) {
    unsigned long h = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        h = h * 31 + (unsigned char)s[i];
    }
    return (int)(h % m);
}

```

8.3 Insert with chaining

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define TABLE_SIZE 7
struct Node {
    int key;
    int value;
    struct Node* next;
};
struct HashTable {
    struct Node* buckets[TABLE_SIZE];
    int count;
};
int hash_function(int key, int m) {
    int h = key % m;
    if (h < 0) h += m;
    return h;
}
#include <stdlib.h>
void insert(struct HashTable* ht, int key, int value) {
    int idx = hash_function(key, TABLE_SIZE);

    /* Check if key already exists -> update */
    struct Node* curr = ht->buckets[idx];
    while (curr != NULL) {
        if (curr->key == key) {
            curr->value = value;
            return;
        }
        curr = curr->next;
    }

    /* Key not found -> prepend new node */
}

```

```

    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->key = key;
    new_node->value = value;
    new_node->next = ht->buckets[idx];
    ht->buckets[idx] = new_node;
    ht->count++;
}
int main() {

    return 0;
}

```

8.4 Search

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define TABLE_SIZE 7
struct Node {
    int key;
    int value;
    struct Node* next;
};
struct HashTable {
    struct Node* buckets[TABLE_SIZE];
    int count;
};
int hash_function(int key, int m) {
    int h = key % m;
    if (h < 0) h += m;
    return h;
}
int search(struct HashTable* ht, int key) {
    int idx = hash_function(key, TABLE_SIZE);
    struct Node* curr = ht->buckets[idx];
    while (curr != NULL) {
        if (curr->key == key)
            return curr->value;
        curr = curr->next;
    }
    return -1; /* not found */
}
int main() {

    return 0;
}

```

8.5 Delete

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define TABLE_SIZE 7
struct Node {
    int key;
    int value;
    struct Node* next;
};
struct HashTable {
    struct Node* buckets[TABLE_SIZE];
    int count;
};
int hash_function(int key, int m) {
    int h = key % m;
    if (h < 0) h += m;

```

```

    return h;
}
#include <stdlib.h>
void delete_key(struct HashTable* ht, int key) {
    int idx = hash_function(key, TABLE_SIZE);
    struct Node* curr = ht->buckets[idx];
    struct Node* prev = NULL;

    while (curr != NULL) {
        if (curr->key == key) {
            if (prev == NULL)
                ht->buckets[idx] = curr->next;
            else
                prev->next = curr->next;
            free(curr);
            ht->count--;
            return;
        }
        prev = curr;
        curr = curr->next;
    }
}
int main() {

    return 0;
}

```

8.6 Rehash trigger

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define TABLE_SIZE 7
struct Node {
    int key;
    int value;
    struct Node* next;
};
struct HashTable {
    struct Node* buckets[TABLE_SIZE];
    int count;
};
int hash_function(int key, int m) {
    int h = key % m;
    if (h < 0) h += m;
    return h;
}
#include <stdlib.h>
#include <stdio.h>
void maybe_rehash(struct HashTable* ht) {
    float load_factor = (float)ht->count / TABLE_SIZE;
    if (load_factor > 0.7) {
        printf("Load factor %.2f exceeds 0.7 -- rehash needed\n", load_factor);
        /* In practice: allocate new table, re-insert all keys, free old table */
    }
}
int main() {

    return 0;
}

```

8.7 Linear probing insert

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>
#define TABLE_SIZE 7
struct Node {
    int key;
    int value;
    struct Node* next;
};
struct HashTable {
    struct Node* buckets[TABLE_SIZE];
    int count;
};
int hash_function(int key, int m) {
    int h = key % m;
    if (h < 0) h += m;
    return h;
}
#include <stdlib.h>
void insert_linear(struct HashTable* ht, int key, int value) {
    int idx = hash_function(key, TABLE_SIZE);
    int start = idx;

    while (ht->buckets[idx] != NULL) {
        if (ht->buckets[idx]->key == key) {
            ht->buckets[idx]->value = value; /* update */
            return;
        }
        idx = (idx + 1) % TABLE_SIZE;
        if (idx == start) return; /* table full */
    }

    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->key = key;
    new_node->value = value;
    new_node->next = NULL;
    ht->buckets[idx] = new_node;
    ht->count++;
}
int main() {
    return 0;
}

```

9. ASCII Diagram

9.1 Chaining: array of linked lists after inserting 10, 22, 31, 4, 15, 28

$h(k) = k \bmod 7, \quad m = 7$

Index 0: +-----+ NULL
 | 28 |----->
 | val |
 +-----+

Index 1: +-----+ +-----+ NULL
 | 22 |-----> | 15 |----->
 | val | | val |
 +-----+ +-----+

Index 2: NULL (empty bucket)

Index 3: +-----+ +-----+ NULL
 | 10 |-----> | 31 |----->
 | val | | val |
 +-----+ +-----+

```

Index 4: +-----+      NULL
         | 4   |----->
         | val |
         +-----+

```

Index 5: NULL (empty bucket)

Index 6: NULL (empty bucket)

load factor $\alpha = 6/7 = 0.857$

9.2 Linear probing: same keys, no chaining

$h(k) = k \bmod 7$, $m = 7$, linear probing

```

Index 0: [28|val]    -- h(28) = 0, slot free
Index 1: [22|val]    -- h(22) = 1, slot free
Index 2: [15|val]    -- h(15) = 1 -> probe (1+1)=2, slot free
Index 3: [10|val]    -- h(10) = 3, slot free
Index 4: [4 |val]    -- h(4)  = 4, slot free
Index 5: [31|val]    -- h(31) = 3 -> busy -> 4 -> busy -> 5, slot free
Index 6: EMPTY

```

Search for 31:

```

h(31)=3 -> index 3: key 10, no match -> index 4: key 4, no match
-> index 5: key 31, match. (2 probes past initial position)

```

9.3 Quadratic probing sequence ($h(k)=3$, $c=1$)

```

Probe 0: 3          -> slot 3
Probe 1: 3+1^2=4    -> slot 4
Probe 2: 3+2^2=7%7 = 0
Probe 3: 3+3^2=12%7 = 5

```

10. When to Use / When to Avoid

Use a hash table when:

- You need $O(1)$ average-case insert, search, and delete on an unordered collection.
- You are implementing a dictionary, symbol table, set, or cache.
- The keys are alphanumeric (strings) or numeric and you need fast lookup.
- Order of elements does not matter.

Avoid a hash table when:

- You need elements in sorted order (use a BST instead).
- You need guaranteed $O(\log n)$ worst case (hash tables have $O(n)$ worst case).
- Memory is extremely constrained (each chain node adds pointer overhead).
- The key distribution is pathological and a good hash function is hard to design.

Key exam point: Hash tables do NOT maintain any ordering of their elements. If the exam asks "which data structure stores elements in sorted order," hash table is always the wrong answer.

11. Exam Hot Spots

This chapter has **15 MCQ corpus questions** assigned to it. They cluster into the following patterns.

Pattern A: Hash function role (Q228, Q479)

Q: "Care este rolul functiei hash intr-o tabela de dispersie?" / "What is the role of the hash function in a hash table?"

- **Correct:** "Sa mapeze o valoare de intrare la o pozitie in tabela" / "To map an input value to a position in the table."

- **Wrong:** "Sa comprime datele" (compress data -- no), "Sa cripteze datele" (encrypt -- no), "Sa converteasca in valori numerice" (convert to numeric -- it does more than that: it maps to a table index), "Sa elimine duplicatele" (remove duplicates -- no).

Pattern B: Search correctness in chaining (Q43)

Q: "What is correct about searching in a hash table with chaining?"

- **Correct:** "Correct because it uses the hash function followed by sequential search" -- hash to find the bucket, then walk the chain.
- **Wrong:** "Incorrect because hash allows direct access to a student list that caused a conflict" (chaining is correct), "Incorrect because hash generates values outside the allocated range" (the mod step prevents this), "Correct because it uses only sequential access" (no -- it uses hashing first).

Pattern C: Linear probing search correctness (Q52)

Q: "What is true about the following C function using linear probing?"

- **Correct:** "The function correctly implements the search operation for the studName parameter if it is a key in the hash table."
- **Wrong answers** claim the function is incorrect or that it identifies all elements.

Pattern D: Insert code arrangement (Q75)

Q: "Arrange the C code" (code with missing lines for hash table insertion)

- **Correct lines:** `i=0;` (loop initialization) and `ht[i]=s` (place student at computed index).
- The exam tests whether you can reconstruct the insert logic from partial code.

Pattern E: Correct search behavior (Q42)

Q: "Which of the following statements is true?" (code snippet with hash table search)

- **Correct:** "The function correctly implements the search operation for the parameter studName if it is a key in the hash table."
- **Wrong:** "The function identifies all elements of type Student stored in the hash table" (search finds one, not all).

Pattern F: Worst-case complexity of chaining (Q234, Q485)

Q: "What is the worst-case time complexity of searching in a hash table with chaining?"

- **Correct:** $O(n)$ -- when all n keys hash to the same bucket, you must walk a list of length n .
- **Wrong:** $O(1)$ -- that is the AVERAGE case, not worst. $O(\log n)$ -- no, there is no sorting involved. $O(n \log n)$ -- not applicable.

Pattern G: Good hash function properties (Q235, Q486)

Q: "Which property must a hash function satisfy to be considered good?"

- **Correct:** "It must be deterministic and distribute keys uniformly across the table."
- **Wrong:** "It must produce a unique value for every input" (impossible when $m < \text{key space}$), "It must always map distinct keys to the same value" (nonsensical), "It must be a one-way function" (irrelevant for hash tables), "It must be reversible" (not required).

Pattern H: Load factor definition (Q236, Q487)

Q: "What does the load factor of a hash table measure?"

- **Correct:** "The ratio of keys to the number of slots in the table" -- $\alpha = n/m$.
- **Wrong:** "The ratio of empty slots to occupied slots", "The total memory used by the table in bytes", "The maximum number of collisions possible."

Pattern I: Hash table applicability (Q245, Q497)

Q: "Precizati care dintre urmatoarele probleme pot fi rezolvate prin utilizarea unei tabele de dispersie"

- **Correct:** "Cautarea unui element intr-un set de date dinamic fara nicio garantie asupra limitei superioare a timpului de cautare" -- searching in a dynamic set WITHOUT worst-case guarantee. This is the defining property of hash tables.
- **Wrong:** "Cautarea rapida ... cu garantie asupra limitei superioare" (fast search WITH worst-case guarantee -- that is BST, not hash table), "Sortarea datelor" (sorting -- hash tables do not sort), "Stocarea ... cu cautarea in $O(\log n)$ doar daca setul de date este static" ($O(\log n)$ on static data -- that is a sorted array or BST).

Pattern J: BST vs hash table comparison (Q7951)

Q: "What is the main risk of an unbalanced binary search tree?"

- **Correct:** "Operations can degrade to $O(n)$." This question appears in the hash table question set because the exam contrasts BST worst-case with hash table worst-case. Both degrade to $O(n)$, but for different reasons: BST degrades from unbalanced tree shape; hash table degrades from poor hash function or all keys colliding.
- **Wrong:** "It becomes a hash table" -- that is a joke answer; unbalanced BST does not turn into a hash table.

12. Common Traps

The corpus has a consistent set of wrong-answer patterns. Memorize why each is wrong so you don't pick them.

Wrong Answer	Why It's Wrong
"Average case is $O(1)$, so worst case is also $O(1)$ "	Average and worst case are different. Worst case is $O(n)$ when all keys collide in one bucket.
"Hash table always gives $O(1)$ lookup"	Only on average with a good hash function. Worst case is $O(n)$. The exam explicitly tests this distinction (Q234, Q485).
"Load factor = empty slots / occupied slots"	Load factor = n/m = keys / slots, not the ratio of empty to occupied.
"Chaining eliminates collisions"	Chaining MANAGES collisions; it does not prevent them. Collisions still occur and chains can grow long.
"Linear probing has no clustering problems"	Linear probing suffers from PRIMARY CLUSTERING -- long runs of occupied slots and slow down all subsequent operations.
"Hash tables preserve insertion order"	Hash tables have no ordering guarantee. If you need order, use a linked list or ordered map.
"Rehashing is $O(1)$ "	Rehashing is $O(n)$ because every key must be re-inserted. It is amortized $O(1)$ per insert operation.
"Double hashing uses the same hash function twice"	Double hashing uses TWO DIFFERENT hash functions. The second determines the probe step size.
"Open addressing uses less memory than chaining"	Open addressing stores records directly in the array, but the array must be larger (lower load factor). Chaining stores pointers plus nodes. The comparison depends on key/value sizes.
"A hash function must be one-way / irreversible"	One-way hash functions are for cryptography (passwords, signatures). Hash table functions just need to map keys to indices uniformly.
"Deleting from linear probing is as simple as marking a slot empty"	Marking a slot empty breaks the probe chain for other keys that

	hashed to the same initial position. You must use a DELETED sentinel and handle it specially during search.
"Hash table search always terminates"	If the table is full and using linear probing with no DELETED markers, a search for a non-existent key loops forever. Always ensure at least one EMPTY slot exists.

13. Quick-Recall Card

DEFINITION

Hash table = array of m buckets + hash function $h(\text{key}) \rightarrow \text{index in } [0, m-1]$.
Maps keys to values with $O(1)$ average insert/search/delete.

HASH FUNCTION REQUIREMENTS

Deterministic: same key $\rightarrow$ same index, always.
Uniform spread: keys distribute evenly across all m buckets.
Efficient: $O(1)$ to compute.

COLLISION RESOLUTION

Chaining: each bucket is a linked list. Collisions append to the list.
Linear probing: if $h(k)$ occupied, try $(h(k)+1)\%m$, $(h(k)+2)\%m$, ...
Prone to PRIMARY CLUSTERING.
Quadratic probing: try $h(k)+1^2$, $h(k)+2^2$, ... Reduces clustering.
Double hashing: probe step = $h_2(k)$. Best distribution, two hashes per probe.

LOAD FACTOR

$\alpha = n / m$ (n = keys, m = slots).
 $\alpha > 0.7$ (open addressing) $\rightarrow$ rehash.
Chaining tolerates $\alpha > 1$ (more keys than slots).

REHASHING

Allocate new array (typically $2m+1$ or next prime).
Re-insert all n keys. Cost: $O(n)$ one-time.
Amortized cost per insert: $O(1)$.

BIG-O SUMMARY

Chaining: Average $O(1)$, Worst $O(n)$.
Linear probing: Average $O(1)$, Worst $O(n)$.
Quadratic probing: Average $O(1)$, Worst $O(n)$.
Double hashing: Average $O(1)$, Worst $O(n)$.
Rehash: $O(n)$ one-time.

EXAM GOLD

- Worst case for ANY hash table scheme is $O(n)$ (all keys in one bucket/cluster).
- Average case is $O(1)$ only with a good hash function and bounded load factor.
- Load factor = n/m , NOT empty/occupied ratio.
- Hash tables do NOT sort or preserve insertion order.
- Chaining manages collisions; it does not prevent them.
- Linear probing causes primary clustering; quadratic probing reduces it.
- Rehashing is $O(n)$ but amortized $O(1)$ per operation.
- Hash function must be deterministic + uniform. Not one-way. Not reversible.

\newpage

Chapter 7 — Binary Heaps

Core data structure chapter. The binary heap is a complete binary tree stored in an array -- no pointers needed. It is the foundation of priority queues and heap sort. The chapter has **28 MCQ corpus questions** assigned to it, covering index arithmetic, operation complexity, heap properties, and comparison with other structures.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define a binary heap (min-heap and max-heap) and distinguish it from a BST and a sorted array.
- Map parent and child positions using the index formulas: $\text{parent}(i) = i/2$, $\text{left}(i) = 2i$, $\text{right}(i) = 2i+1$ (1-indexed) or $\text{parent}(i) = (i-1)/2$, $\text{left}(i) = 2i+1$, $\text{right}(i) = 2i+2$ (0-indexed).
- Trace through insert (bubble-up) and extract-min/max (bubble-down) operations step by step on both the tree and the array.
- Explain why build-heap runs in $O(n)$ and not $O(n \log n)$.
- Analyze the complexity of all heap operations: insert $O(\log n)$, extract $O(\log n)$, peek $O(1)$, build $O(n)$.
- Implement a heap in C using an array and compile-safe code.
- Identify when a heap is the right choice (priority queue, top-k) and when it is not (sorted order, search).

2. Mental Model & Analogy

Think of a heap as a **tournament bracket** where only one player stands at the top. In a min-heap, the smallest value is always the champion. When a new player arrives (insert), they challenge their way up the bracket (bubble-up) until they find their rightful level. When the champion leaves (extract-min), the last player in the bracket moves to the top and challenges downward (bubble-down) to fill the gap.

Three rules to internalize:

1. **Shape rule:** The tree is *complete* -- every level is full except possibly the last, which is filled left to right. This is why it fits perfectly in an array with no wasted space.
2. **Order rule (heap property):** In a min-heap, every parent is less than or equal to its children. In a max-heap, every parent is greater than or equal to its children. This does *not* mean the tree is sorted -- only that the root is always the extreme value.
3. **No cross constraint:** Unlike a BST, there is no relationship between left and right siblings. The left child is not necessarily smaller than the right child. This partial ordering is what makes the heap efficient for insert and extract but useless for search.

3. Memory Layout

A complete binary tree fits into a contiguous array. No pointers are needed because parent-child relationships are encoded by index arithmetic.

1-indexed layout (the classical textbook form):

Tree:	Array (1-indexed):
<pre> 10 / \ 20 15 / \ / \ 30 25 40 26 </pre>	<pre> Index: 1 2 3 4 5 6 7 Value: 10 20 15 30 25 40 26 parent(i) = i / 2 (integer division) left(i) = 2 * i right(i) = 2 * i + 1 </pre>

0-indexed layout (the C array form):

Tree:	Array (0-indexed):
<pre> 10 / \ 20 15 / \ / \ 30 25 40 26 </pre>	<pre> Index: 0 1 2 3 4 5 6 Value: 10 20 15 30 25 40 26 parent(i) = (i - 1) / 2 (integer division) left(i) = 2 * i + 1 right(i) = 2 * i + 2 </pre>

Mapping rule: For a node at index i , its children are at $2i$ and $2i+1$ (1-indexed) or $2i+1$ and $2i+2$ (0-indexed). Its parent is at $i/2$ (1-indexed) or $(i-1)/2$ (0-indexed). The root is always at index 0 (0-indexed) or index 1 (1-indexed).

Key fact the exam will test: The last level of a complete binary tree is filled **left to right**. This means the next insertion always goes at the end of the array (the first available position on the last level).

4. Definition & Properties

Binary heap -- a complete binary tree that satisfies the heap property. It can be implemented as a min-heap or a max-heap.

Min-heap property: For every node i (other than the root), $A[\text{parent}(i)] \leq A[i]$. The root holds the minimum element.

Max-heap property: For every node i (other than the root), $A[\text{parent}(i)] \geq A[i]$. The root holds the maximum element.

Properties:

- A heap is a **complete binary tree** -- all levels are fully filled except the last, which is filled left to right. This guarantees $\text{height} = \text{floor}(\log_2 n)$.
- A heap is **partially ordered**, not fully sorted. Siblings have no ordering relationship.
- A heap can be stored in a **plain array** -- no pointers, no node structs. The tree structure is implicit in the array indices.
- The root is always at index 0 (0-indexed) or index 1 (1-indexed).
- The **minimum** (min-heap) or **maximum** (max-heap) element is always at the root, accessible in $O(1)$.
- A heap is NOT a BST: you cannot efficiently search for an arbitrary element.
- A heap is NOT sorted: in-order traversal does not produce sorted output.
- A min-heap can be converted to a max-heap by reversing the comparison (swap $\leq$ with $\geq$ everywhere).

5. Operations -- Step-by-Step Mechanics

5.1 Insert (bubble-up / sift-up / re-heapification upward)

Idea: Place the new element at the next available position (end of the array). Then walk it up by swapping with its parent until the heap property is restored.

Pseudocode (0-indexed, max-heap):

```
1. Append the new element at the end of the array (index = size).
2. Set  $i = \text{size}$ ; increment size.
3. While  $i > 0$  AND  $A[i] > A[\text{parent}(i)]$ :
    swap  $A[i]$  with  $A[\text{parent}(i)]$ 
     $i = \text{parent}(i)$ 
4. Done. Heap property restored.
```

Why $O(\log n)$: The element walks up at most one level per swap, and the tree has height $\text{floor}(\log_2 n)$.

5.2 Extract-min / Extract-max (bubble-down / sift-down / re-heapification downward)

Idea: Remove the root (the extreme value). Move the last element to the root position. Then walk it down by swapping with the appropriate child until the heap property is restored.

Pseudocode (0-indexed, max-heap):

```
1. Save  $A[0]$  (the max) as the result.
2. Move  $A[\text{size}-1]$  to  $A[0]$ . Decrement size.
3. Set  $i = 0$ .
4. While  $i$  has a child and  $A[i] < A[\text{larger\_child}]$ :
    swap  $A[i]$  with  $A[\text{larger\_child}]$ 
     $i = \text{larger\_child}$ 
5. Return the saved result.
```

Why $O(\log n)$: The element walks down at most one level per swap, and the tree has height $\text{floor}(\log_2 n)$.

5.3 Peek (access root)

```
int peek(int* heap) {
    return heap[0];
}
```

```
}
```

$O(1)$ -- just read the first element.

5.4 Heapify (fix a single violation)

Given an index i whose left and right subtrees are valid heaps but $A[i]$ may violate the property, bubble-down from i to restore the heap.

5.5 Build-heap $O(n)$

Naive approach: Insert n elements one by one. Each insert is $O(\log n)$, total $O(n \log n)$.

Efficient approach (Floyd's algorithm): Start from the last non-leaf node and call heapify (bubble-down) on each, working backward to the root.

Why $O(n)$: A node at height h does at most h swaps. The sum over all nodes is:

$$\text{Sum} = n/4 * 1 + n/8 * 2 + n/16 * 3 + \dots < n * (1/2 + 2/4 + 3/8 + \dots) = n * 2 = O(n)$$

The exact sum converges to n . This is a well-known result -- build-heap is linear, not $n \log n$.

6. Trace-Through Example

Insert 5 keys into a min-heap: 10, 20, 5, 15, 8

Step 1: Insert 10. Array is empty; 10 becomes the root.

Tree: Array: [10]
10

Step 2: Insert 20. Append at end. $20 > \text{parent}(0) = 10$? No. Done.

Tree: Array: [10, 20]
10
/
20

Step 3: Insert 5. Append at end. $5 < \text{parent}(1) = 20$? Yes. Swap.

Tree: Array: [10, 5, 20] --> swap 5 and 20
10 --> then $5 < \text{parent}(0) = 10$? Yes. Swap.
/
20 5 --> Array: [5, 10, 20]

Step 4: Insert 15. Append at end. $15 < \text{parent}(1) = 10$? No. Done.

Tree: Array: [5, 10, 20, 15]
5
/
10 20
/
15

Step 5: Insert 8. Append at end. $8 < \text{parent}(2) = 20$? Yes. Swap.

Tree: Array: [5, 10, 8, 15, 20] --> swap 8 and 20
5 --> then $8 < \text{parent}(0) = 5$? No. Done.
/
10 8
/
15 20

Final min-heap: Array = [5, 10, 8, 15, 20]

Extract-min twice:

Step 6: Extract-min. Root = 5. Move last element (20) to root. Bubble-down.

Before: [5, 10, 8, 15, 20]
After moving 20 to root: [20, 10, 8, 15]

Compare 20 with children 10 and 8. Min child = 8. $20 > 8$, swap.
[8, 10, 20, 15]

Compare 20 with child 15. $20 > 15$, swap.
[8, 10, 15, 20]

Result: min = 5, Array = [8, 10, 15, 20]

Step 7: Extract-min. Root = 8. Move last element (20) to root. Bubble-down.

Before: [8, 10, 15, 20]
After moving 20 to root: [20, 10, 15]

Compare 20 with children 10 and 15. Min child = 10. $20 > 10$, swap.
[10, 20, 15]

Result: min = 8, Array = [10, 20, 15]

7. Operations + Big-O Table

Operation	Complexity	Notes
Insert	$O(\log n)$	Append + bubble-up, at most height of tree
Extract-min / Extract-max	$O(\log n)$	Replace root + bubble-down, at most height
Peek (get min/max)	$O(1)$	Always at index 0
Build-heap (from array)	$O(n)$	Floyd's algorithm: heapify from last non-leaf to root
Heap sort	$O(n \log n)$	Build heap $O(n)$ + n extracts $O(n \log n)$ total
Decrease-key	$O(\log n)$	Bubble-up from modified position
Increase-key	$O(\log n)$	Bubble-down from modified position
Search (arbitrary)	$O(n)$	No BST property; must scan
Delete arbitrary element	$O(n)$	Find $O(n)$ + remove $O(\log n)$

Heap sort: Repeatedly extract the root and place it at the end of the array. This builds a sorted array in $O(n \log n)$ time, $O(1)$ extra space. Not stable (equal elements may change relative order).

8. C Code Reference

8.1 Heap structure

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int* data;
    int size;
    int capacity;
} MaxHeap;

MaxHeap* heap_create(int cap) {
    MaxHeap* h = (MaxHeap*)malloc(sizeof(MaxHeap));
    h->data = (int*)malloc(cap * sizeof(int));
    h->size = 0;
    h->capacity = cap;
}
```

```

    return h;
}

void heap_free(MaxHeap* h) {
    free(h->data);
    free(h);
}

```

8.2 Insert with bubble-up

```

typedef struct {
    int* data;
    int size;
    int capacity;
} MaxHeap;

void swap(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}

void heap_insert(MaxHeap* h, int val) {
    if (h->size == h->capacity) return; /* full */
    int i = h->size;
    h->data[i] = val;
    h->size++;

    /* Bubble up */
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (h->data[i] > h->data[parent]) {
            swap(&h->data[i], &h->data[parent]);
            i = parent;
        } else {
            break;
        }
    }
}

int main() {

    return 0;
}

```

8.3 Extract with bubble-down

```

void swap(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}

typedef struct {
    int* data;
    int size;
    int capacity;
} MaxHeap;

int heap_extract_max(MaxHeap* h) {
    if (h->size == 0) return -1; /* underflow */

    int root = h->data[0];
    h->data[0] = h->data[h->size - 1];
    h->size--;

    /* Bubble down */
    int i = 0;
    while (1) {
        int left = 2 * i + 1;

```

```

    int right = 2 * i + 2;
    int largest = i;

    if (left < h->size && h->data[left] > h->data[largest])
        largest = left;
    if (right < h->size && h->data[right] > h->data[largest])
        largest = right;

    if (largest != i) {
        swap(&h->data[i], &h->data[largest]);
        i = largest;
    } else {
        break;
    }
}
return root;
}
int main() {
    return 0;
}

```

8.4 Heapify (bubble-down from a given index)

```

void heapify(int* arr, int n, int i);

void swap(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}

void heapify(int* arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}
int main() {
    return 0;
}

```

8.5 Build-heap (Floyd's algorithm, $O(n)$)

```

void heapify(int* arr, int n, int i);

void build_max_heap(int* arr, int n) {
    /* Start from last non-leaf node */
    int last_non_leaf = (n / 2) - 1;
    for (int i = last_non_leaf; i >= 0; i--) {
        heapify(arr, n, i);
    }
}
int main() {

```

```

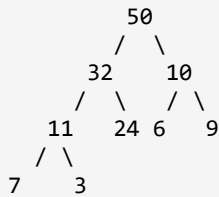
return 0;
}

```

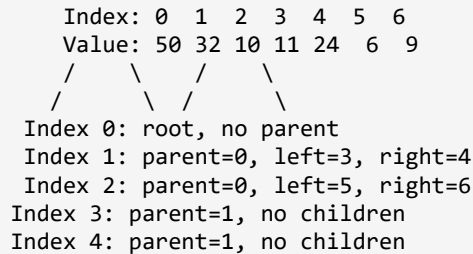
9. ASCII Diagram

Complete binary tree with array indices (0-indexed):

Complete Binary Tree (max-heap):



Array (0-indexed):



Array:

50	32	10	11	24	6	9
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Index formulas in action (0-indexed):

Node at i=1 (value 32):

```

parent = (1-1)/2 = 0 --> value 50 (root)
left   = 2*1+1 = 3 --> value 11
right  = 2*1+2 = 4 --> value 24

```

Node at i=2 (value 10):

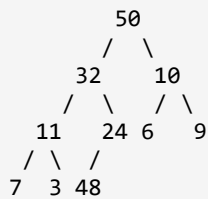
```

parent = (2-1)/2 = 0 --> value 50 (root)
left   = 2*2+1 = 5 --> value 6
right  = 2*2+2 = 6 --> value 9

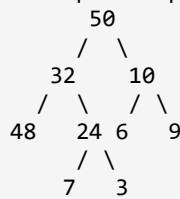
```

Insert 48 into the heap (bubble-up trace):

After insert 48 at position 7:



After swap with parent (i=3):



48 > parent(3) = 11? Yes, swap.

48 > parent(1) = 32? No. Done.

10. When to Use / When to Avoid

Use a heap when:

- You need a **priority queue** -- repeatedly insert items and extract the highest (or lowest) priority element.
- You need the **top-k** elements from a large collection (maintain a k-element heap).
- You want **heap sort** -- $O(n \log n)$ in-place sort, useful when extra space is limited.
- You need efficient access to the **min or max** only, not to arbitrary elements.
- Tasks have priorities: operating systems scheduling, event-driven simulation, Dijkstra's shortest path, medical triage.

Avoid a heap when:

- You need **sorted order** of all elements -- use merge sort or built-in sort.
- You need to **search for an arbitrary element** -- use a BST or hash table.

- You need **stable sort** -- heap sort is not stable.
- You need **sorted traversal** -- in-order traversal of a heap does not produce sorted output (unlike BST).
- You need **fast deletion of arbitrary elements** -- $O(n)$ search in a heap.

11. Exam Hot Spots

This chapter has **28 MCQ corpus questions** assigned to it. They cluster into the following patterns.

Pattern A: Index formulas (most tested)

Q6, Q63, Q4006, Q4010: "If a node is at position i , its left/right child is at position $_?$ "

- **Correct (0-indexed):** left = $2i + 1$, right = $2i + 2$, parent = $(i-1)/2$
- **Correct (1-indexed):** left = $2i$, right = $2i+1$, parent = $i/2$
- **Wrong:** $2i-1$ (not a valid formula), $i-1$ (that is not the left child), $i/2$ (that is the parent, not a child)

Pattern B: Operation complexity

Q5, Q14, Q61: "What is the complexity of inserting / extracting / accessing root?"

- **Correct:** Insert $O(\log n)$, extract $O(\log n)$, peek/access root $O(1)$
- **Wrong:** $O(n)$ for extract, $O(1)$ for insert, $O(n \log n)$ for any single operation

Pattern C: Heap property after operations

Q23, Q69: "After inserting a new element in a heap, the structure..."

- **Correct:** "Maintains heap property" (the bubble-up restores it)
- **Wrong:** "Becomes fully sorted" (only the root is guaranteed), "Becomes completely unstructured" (bubble-up fixes it), "Is unchanged"

Pattern D: Heap classification

Q9, Q16: "A binary heap is considered a $_?$ "

- **Correct:** "Partially ordered structure" (only parent-child relationship, no sibling ordering)
- **Wrong:** "Fully sorted structure" (sibling order is not constrained), "Completely unordered structure"

Pattern E: Heap root identification

Q4004, Q4008: "How do we determine the root of a heap?" / "In a MAX-heap, the element with highest priority is always on..."

- **Correct:** The root holds the min (min-heap) or max (max-heap). "Root" = index 0 (0-indexed) or index 1 (1-indexed).
- **Wrong:** "Leftmost leaf", "Rightmost node", "Last element"

Pattern F: Two types of heaps

Q3999, Q4005: "Heaps can be of two types:" / "Which statement is true about heap?"

- **Correct:** "min-heap & max-heap"
- **Wrong:** "complete & incomplete" (completeness is a property, not a type), "sorted & unsorted"

Pattern G: False statements about heaps

Q4000, Q4001, Q4009: "Which statement is false about heap?" / "Which is false about min-heap?"

- **Correct (false statement to identify):** "Heap can be implemented with 2D arrays" (no -- it uses a 1D array), "Last level filled right to left" (no -- left to right), "Root of min-heap is the max value" (no -- it is the min)

- **Wrong (true statements):** "Propriety means elements form a complete binary tree", "Max-heap: value of any node $\leq$ value of parent"

Pattern H: Heap sort

Q4002: "What type of sort can be implemented with a binary heap?"

- **Correct:** Heap sort
- **Wrong:** Bubble sort, quicksort, binary search (not a sort)

Pattern I: 0-indexed vs 1-indexed formulas

Q4010: "For traversing a 1D array storing a heap, which formula is INVALID?"

- **Correct:** "Left child = $2 * \text{position} - 1$ " is INVALID for 0-indexed (it gives the wrong index)
- **Valid:** left = $2\text{position} + 1$, right = $2(\text{position} + 1) = 2\text{position} + 2$, right = $2\text{position} + 2$

Pattern J: Linked lists vs heaps (Romanian questions)

Q93, Q122, Q214, Q469, Q967, Q990, Q3954: "Advantages of linked lists over heaps"

- **Correct:** "Simplified dynamic management in certain contexts", "Simpler insert/delete at known positions"
- **Wrong:** "Highest efficiency in sorting" (not a list advantage), "Faster random access" (lists have $O(n)$ access), "Direct access to min/max element" (that is a heap advantage, not a list advantage)

Pattern K: BST vs heap structure

Q3963, Q3998: "Which statement is false about BST?" / "AVL tree structure contains:" (included because heap is often compared to BST)

- **Q3963:** "None of the above" (all listed BST statements are true)
- **Q3998:** AVL node structure: "info, left child pointer, right child pointer" (correct). Wrong: "only left and right pointers" (missing content), "only left and content" (missing right).
- **Wrong (true statements):** "Right child is always greater than parent" -- this is true for BST, but the question asks what is false

12. Common Traps

The corpus has a consistent set of wrong-answer patterns. Memorize why each is wrong so you don't pick them.

Wrong	Why it is wrong
"Heap is a sorted structure"	Heap is <i>partially ordered</i> only. Siblings have no ordering constraint. In-order traversal does NOT produce sorted output.
"Heap property means left child < right child"	There is no ordering between siblings. Only the parent-child relationship matters.
"Insert into a heap is $O(1)$ "	Insert is $O(\log n)$ because bubble-up may traverse the full height. Peek is $O(1)$.
"Build-heap is $O(n \log n)$ "	Floyd's build-heap is $O(n)$ by heapifying from the bottom up. The sum converges to n .
"Extract-min/max is $O(1)$ "	Extract is $O(\log n)$ because bubble-down must restore the property. Peek is $O(1)$.
"Heap is the same as the memory heap"	The memory heap (malloc/free region) is completely unrelated to the binary heap data structure.

"Left child index = $2i-1$ "	Wrong formula. For 0-indexed: left = $2i+1$. For 1-indexed: left = $2i$.
"Root of a min-heap is the max element"	Root of a min-heap is the <i>minimum</i> . Root of a max-heap is the <i>maximum</i> .
"Last level filled right to left"	Last level is filled <i>left to right</i> . The next insertion goes at the first available slot on the left.
"Heap sort is stable"	Heap sort is <i>unstable</i> -- equal elements may change relative order during swaps.
"You can search any element in $O(\log n)$ "	Search for an arbitrary element is $O(n)$. No BST ordering property exists in a heap.
"0-indexed parent = $i/2$ "	For 0-indexed: parent = $(i-1)/2$. For 1-indexed: parent = $i/2$. Mixing these up is the most common error.

13. Quick-Recall Card

DEFINITION

Binary heap = complete binary tree + heap property (min or max).
 Stored in a plain array -- no pointers needed.
 Partially ordered: parent $\leq$ children (min-heap) or parent $\geq$ children (max-heap).

INDEX FORMULAS

1-indexed: parent(i) = $i/2$ left(i) = $2i$ right(i) = $2i+1$
 0-indexed: parent(i) = $(i-1)/2$ left(i) = $2i+1$ right(i) = $2i+2$

OPERATIONS

Insert: append at end, bubble up $O(\log n)$
 Extract root: move last to root, bubble down $O(\log n)$
 Peek root: read array[0] $O(1)$
 Build-heap: heapify from last non-leaf to 0 $O(n)$
 Heap sort: build + n extracts $O(n \log n)$

BUBBLE-UP (insert)

Place new element at the end.
 While it violates heap property with its parent, swap with parent.
 Stop when parent is OK or root is reached.

BUBBLE-DOWN (extract)

Move last element to root.
 While it violates heap property with its larger child (max-heap) or smaller child (min-heap), swap with that child.
 Stop when both children are OK or it is a leaf.

BUILD-HEAP WHY $O(n)$

Nodes at height h do at most h swaps.
 Sum = $n/4*1 + n/8*2 + n/16*3 + \dots < n * (1/2 + 2/4 + 3/8 + \dots) = O(n)$.

HEAP SORT

1. Build max-heap from the array.
 2. Swap root with last element, shrink heap, bubble-down.
 3. Repeat until heap is empty.
 Result: sorted array in $O(n \log n)$, $O(1)$ extra space, NOT stable.

EXAM GOLD

- Heap is PARTIALLY ordered, not fully sorted.
- Left child is NOT necessarily smaller than right child.
- Build-heap is $O(n)$, NOT $O(n \log n)$.
- 0-indexed parent = $(i-1)/2$, NOT $i/2$.
- Last level filled LEFT to RIGHT.
- Root of min-heap = minimum, root of max-heap = maximum.
- Heap sort is NOT stable.
- Search for arbitrary element is $O(n)$.

WHEN TO USE

Priority queue (insert + extract-min/max repeatedly)
Top-k selection
Heap sort (in-place, $O(n \log n)$)
Dijkstra's shortest path (min-heap for frontier)

\newpage

Chapter 8 — Maps & Sets

Concept chapter. Two fundamental abstract data types: the **map** (key-value pairs with $O(1)$ lookup) and the **set** (unique-element collection). C has no built-in map or set -- these ADTs are implemented via the structures already covered: hash tables (Ch 6) and binary search trees (Ch 9). The chapter has **0 MCQ corpus questions assigned to it directly** -- map/set concepts appear inside hash-table and BST questions. Master the ADT definitions, the two implementation families, and the set-operation algorithms.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define a **map** (dictionary, associative array) as an ADT mapping unique keys to values, and list its core operations: put, get, containsKey, remove.
- Define a **set** as an ADT storing unique elements with no inherent order, and list its core operations: add, remove, contains.
- Distinguish between a map/set as an **abstract data type** (what it does) and its **implementation** (how it does it -- hash table or BST).
- Explain why hash-based implementations give **$O(1)$ average** for put/get/contains while tree-based implementations give **$O(\log n)$** but maintain sorted order.
- Compute the three set operations -- **union**, **intersection**, and **difference** -- on two sets and state their time complexity.
- Recognize that C has no built-in map or set, and identify how to implement each using a hash table (Ch 6) or a BST (Ch 9).

2. Mental Model & Analogy

Map = a dictionary. You look up a word (the **key**) and find its definition (the **value**). Each word appears exactly once. You can add new words, look up definitions, or remove entries -- but you cannot have two entries for the same word. The order in which words appear in the dictionary is not guaranteed (hash-based) or is sorted alphabetically (tree-based).

Set = a bag of unique items. Imagine a collection of distinct marbles. You can check if a marble is in the bag, add a new marble, or remove one. Putting the same marble in twice has no effect -- the bag only tracks **membership**, not quantity. A set is really a map where you only care about the keys and the values are discarded.

The key insight: **a set can be implemented as a map with dummy values.** This connection is why maps and sets are covered together -- they share the same underlying structures (hash tables and BSTs).

3. Memory Layout

A map or set can be stored in two fundamental ways, corresponding to the two implementation families.

Hash-based storage

An array of **buckets**. Each bucket either holds an entry (key-value pair or just a key) or is empty. A hash function maps each key to a bucket index. Collisions are resolved by chaining (each bucket points to a linked list of entries that hashed to the same index).

Hash Table (Map)	Hash Table (Set)
+-----+	+-----+
0 -> NULL	0 -> NULL
+-----+	+-----+
1 -> ["apple",5] -> NULL	1 -> "apple" -> NULL
+-----+	+-----+
2 -> NULL	2 -> NULL
+-----+	+-----+
3 -> ["cat",9] -> NULL	3 -> "cat" -> "dog" -> NULL
+-----+	+-----+
4 -> ["dog",2] -> NULL	4 -> NULL
+-----+	+-----+
...	...

Trade-offs: fast lookup ($O(1)$ average), no ordering guarantee, extra memory for the bucket array and collision chains.

Tree-based storage

A **binary search tree** (BST), where each node holds a key (and optionally a value). Left children are smaller, right children are larger. Balanced BSTs (AVL, red-black) guarantee $O(\log n)$ height.

BST (Map)	BST (Set)
["cat",9] / \	"cat" / \
["apple",5] ["dog",2]	"apple" "dog"

Trade-offs: $O(\log n)$ lookup, elements accessible in sorted order, no bucket array overhead but each node has left/right pointers.

	Hash-based	Tree-based
Lookup	$O(1)$ average, $O(n)$ worst	$O(\log n)$ guaranteed (balanced)
Ordering	None	Sorted by key
Extra memory	Bucket array + chains	Left/right pointers per node
Worst case	All keys collide -> $O(n)$ chain	Unbalanced tree -> $O(n)$

4. Definition & Properties

4.1 Map (Dictionary, Associative Array)

A **map** is an ADT that stores a collection of **key-value pairs** where each key is unique.

Property	Description
Keys	Unique -- each key appears at most once
Values	Can be duplicated -- two keys may map to the same value
Ordering	Not guaranteed (hash-based) or sorted by key (tree-based)
Size	Number of key-value pairs, denoted n

Core operations:

Operation	Description	Returns
<code>put(key, value)</code>	Insert or update key-value pair	Previous value (or none)
<code>get(key)</code>	Retrieve value for key	Value, or indicator for "not found"
<code>containsKey(key)</code>	Check if key exists	Boolean
<code>remove(key)</code>	Delete key-value pair	Previous value (or none)
<code>size()</code>	Number of entries	Integer

Key rule: inserting a duplicate key **replaces** the old value. The map does not store both.

4.2 Set

A **set** is an ADT storing a collection of **unique elements** with no associated values.

Property	Description
Elements	Unique -- each element appears at most once
Ordering	Not guaranteed (hash-based) or sorted (tree-based)
Size	Number of elements, denoted n

Core operations:

Operation	Description	Returns
<code>add(element)</code>	Insert element (no effect if already present)	Boolean (was it new?)
<code>remove(element)</code>	Delete element	Boolean (was it present?)
<code>contains(element)</code>	Check membership	Boolean
<code>size()</code>	Number of elements	Integer

4.3 ADT vs Implementation

The exam tests whether you understand the distinction:

- **ADT** = the logical contract (what operations are available and what they mean). A map provides `put/get/containsKey/remove`.
- **Implementation** = the concrete data structure used to fulfill the contract. A map can be implemented as a hash table OR as a BST.

Choosing a different implementation changes **performance** but not the **interface**. This is the same distinction as "stack" (ADT) vs "array-backed stack" or "list-backed stack" (implementations) from Ch 5.

4.4 Set as a special Map

A set can be implemented as a map whose values are all dummy (e.g., all 1 or all `NULL`). The keys are the set elements. This is how many standard libraries implement sets -- as hash maps or tree maps with placeholder values.

```
Set: {"apple", "cat", "dog"}
Map: {"apple" -> 1, "cat" -> 1, "dog" -> 1}
```

5. Operations -- Step-by-Step Mechanics

5.1 Map operations on a hash table

State: an array of m buckets, each bucket is a linked list of $\{key, value\}$ nodes. Hash function: $h(key) = key \% m$.

put("cat", 9):

1. Compute $h("cat")$. Suppose it equals 3.
2. Walk the chain at bucket 3. If "cat" is found, update its value to 9. If not found, prepend a new node $\{"cat", 9\}$ to the chain.

get("cat"):

1. Compute $h("cat") = 3$.
2. Walk the chain at bucket 3, comparing keys. If "cat" is found, return its value. If the chain ends (NULL), return "not found".

remove("cat"):

1. Compute $h("cat") = 3$.
2. Walk the chain at bucket 3. If "cat" is found, unlink the node and free it. If not found, do nothing.

5.2 Map operations on a BST

State: a BST where each node holds $\{key, value\}$ and is ordered by key.

put("cat", 9):

1. Start at root. Compare "cat" to root's key.
2. If "cat" < root.key, go left. If "cat" > root.key, go right. If equal, update the value.
3. Repeat until you reach a NULL child -- insert the new node there.

get("cat"):

1. Start at root. Compare "cat" to current node's key.
2. If "cat" < current.key, go left. If "cat" > current.key, go right. If equal, return the value.
3. If you reach NULL, "cat" is not in the map.

5.3 Set operations

Set operations follow the same insertion/lookup logic as maps, just without values. The interesting part is **set algebra**:

Union (A union B): all elements that are in A, or in B, or in both.

- Hash-based: insert all elements from B into a copy of A. $O(|A| + |B|)$ average.
- Tree-based: merge two sorted traversals. $O(|A| + |B|)$.

Intersection (A intersect B): elements that are in both A and B.

- Hash-based: for each element in the smaller set, check if it exists in the larger set. $O(\min(|A|, |B|))$ average.
- Tree-based: simultaneous in-order traversal, advancing the pointer that is behind. $O(|A| + |B|)$.

Difference (A - B): elements in A that are not in B.

- Hash-based: for each element in A, check if it is in B. If not, add it to the result. $O(|A|)$ average.
- Tree-based: simultaneous in-order traversal. $O(|A| + |B|)$.

6. Trace-Through Example

6.1 Building a map from 5 key-value pairs (hash table)

Table size $m = 7$. Hash function: $h(\text{key}) = \text{key} \% 7$. Collision resolution: chaining.

Step	Operation	Key	$h(\text{key})$	Bucket 0	1	2	3	4	5	6
1	put(k=10, v="ten")	10	3				[10,"ten"]			
2	put(k=22, v="twenty two")	22	1		[22,"tt"]		[10,"ten"]			
3	put(k=31, v="thirtyone")	31	3		[22,"tt"]		[10,"ten"] -> [31,"to"]			
4	put(k=4, v="four")	4	4		[22,"tt"]		[10,"ten"] -> [31,"to"]	[4,"four"]		
5	put(k=15, v="fifteen")	15	1		[22,"tt"] -> [15,"fif"]		[10,"ten"] -> [31,"to"]	[4,"four"]		

Collision at step 3: $h(31) = 3$, same as $h(10) = 3$. Both entries go in bucket 3's chain.

get(31): $h(31) = 3$ -> walk chain at bucket 3: compare 10 != 31 (next), compare 31 == 31 (found) -> return "thirtyone".

6.2 Building two sets and computing union and intersection

Set A = {3, 7, 1, 9}

Set B = {7, 2, 9, 5}

Union (A union B): start with A, add elements from B that are not already present.

Step	Element	Already in result?	Result
--	(start with A)	--	{3, 7, 1, 9}
1	7 (from B)	yes	{3, 7, 1, 9}
2	2 (from B)	no	{3, 7, 1, 9, 2}
3	9 (from B)	yes	{3, 7, 1, 9, 2}
4	5 (from B)	no	{3, 7, 1, 9, 2, 5}

A union B = {1, 2, 3, 5, 7, 9}

Intersection (A intersect B): elements present in both A and B.

Step	Element	In B?	Result
1	3	no	{}
2	7	yes	{7}
3	1	no	{7}
4	9	yes	{7, 9}

--	--	--	--

A intersect B = {7, 9}

7. Operations + Big-O Table

Operation	Hash-based (avg)	Hash-based (worst)	Tree-based (balanced)
put / add	O(1)	O(n)	O(log n)
get / contains	O(1)	O(n)	O(log n)
remove	O(1)	O(n)	O(log n)
size	O(1)	O(1)	O(1)
Union (A,B)	O(nA + nB)	O(nA * nB)	O(nA + nB)
Intersection (A,B)	O(min(nA,nB))	O(nA * nB)	O(nA + nB)
Difference (A-B)	O(nA)	O(nA * nB)	O(nA + nB)

Notes:

- Hash worst case (O(n)) happens when all keys collide into one bucket -- the chain becomes a linked list.
- Tree complexity assumes a **balanced** BST (AVL or red-black). An unbalanced BST degrades to O(n).
- Union/intersection/difference complexity for hash-based assumes O(1) average lookup and that the smaller set is iterated.

8. C Code Reference

8.1 Simple map via hash table (chaining)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 7

typedef struct Entry {
    char key[32];
    int value;
    struct Entry* next;
} Entry;

Entry* table[TABLE_SIZE];

unsigned int hash(const char* key) {
    unsigned int h = 0;
    for (int i = 0; key[i]; i++)
        h = h * 31 + key[i];
    return h % TABLE_SIZE;
}

void put(const char* key, int value) {
    unsigned int idx = hash(key);
    Entry* e = table[idx];
    while (e) {
        if (strcmp(e->key, key) == 0) {
            e->value = value;
            return;
        }
    }

```

```

        e = e->next;
    }
    e = (Entry*)malloc(sizeof(Entry));
    strcpy(e->key, key);
    e->value = value;
    e->next = table[idx];
    table[idx] = e;
}

int get(const char* key, int* out) {
    unsigned int idx = hash(key);
    Entry* e = table[idx];
    while (e) {
        if (strcmp(e->key, key) == 0) {
            *out = e->value;
            return 1;
        }
        e = e->next;
    }
    return 0;
}

int main(void) {
    put("apple", 5);
    put("cat", 9);
    put("dog", 2);
    put("apple", 12);
    int val;
    if (get("cat", &val))
        printf("cat -> %d\n", val);
    if (get("apple", &val))
        printf("apple -> %d\n", val);
    return 0;
}
/* Output:
cat -> 9
apple -> 12
*/

```

8.2 Simple set via sorted array

```

#include <stdio.h>
#include <string.h>

#define MAX 100

char* set[MAX];
int set_size = 0;

int find_index(const char* e) {
    for (int i = 0; i < set_size; i++)
        if (strcmp(set[i], e) == 0) return i;
    return -1;
}

void add(const char* e) {
    if (find_index(e) >= 0) return;
    set[set_size++] = (char*)e;
}

void remove_elem(const char* e) {
    int i = find_index(e);
    if (i < 0) return;
    set[i] = set[set_size - 1];
    set_size--;
}

```

```

}

int contains(const char* e) {
    return find_index(e) >= 0;
}

```

This is $O(n)$ for all operations -- acceptable for small sets. For $O(1)$ average, use a hash table instead of an array.

8.3 Set union and intersection

```

#include <stdio.h>
#include <string.h>

#define MAX 100

typedef struct {
    char* elements[MAX];
    int size;
} Set;

void set_add(Set* s, const char* e) {
    for (int i = 0; i < s->size; i++)
        if (strcmp(s->elements[i], e) == 0) return;
    s->elements[s->size++] = (char*)e;
}

int set_contains(Set* s, const char* e) {
    for (int i = 0; i < s->size; i++)
        if (strcmp(s->elements[i], e) == 0) return 1;
    return 0;
}

Set set_union(Set* a, Set* b) {
    Set result = {{0}, 0};
    for (int i = 0; i < a->size; i++)
        set_add(&result, a->elements[i]);
    for (int i = 0; i < b->size; i++)
        set_add(&result, b->elements[i]);
    return result;
}

Set set_intersection(Set* a, Set* b) {
    Set result = {{0}, 0};
    for (int i = 0; i < a->size; i++)
        if (set_contains(b, a->elements[i]))
            set_add(&result, a->elements[i]);
    return result;
}

int main(void) {
    Set A = {"apple", "cat", "dog", 3};
    Set B = {"cat", "dog", "fish", 3};
    Set U = set_union(&A, &B);
    Set I = set_intersection(&A, &B);
    printf("Union: ");
    for (int i = 0; i < U.size; i++) printf("%s ", U.elements[i]);
    printf("\nIntersection: ");
    for (int i = 0; i < I.size; i++) printf("%s ", I.elements[i]);
    printf("\n");
    return 0;
}

/* Output:
Union: apple cat dog fish
Intersection: cat dog
*/

```

8.4 Map via BST node (skeleton)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct MapNode {
    char key[32];
    int value;
    struct MapNode* left;
    struct MapNode* right;
} MapNode;

MapNode* bst_put(MapNode* root, const char* key, int value) {
    if (root == NULL) {
        MapNode* n = (MapNode*)malloc(sizeof(MapNode));
        strcpy(n->key, key);
        n->value = value;
        n->left = n->right = NULL;
        return n;
    }
    int cmp = strcmp(key, root->key);
    if (cmp < 0)
        root->left = bst_put(root->left, key, value);
    else if (cmp > 0)
        root->right = bst_put(root->right, key, value);
    else
        root->value = value;
    return root;
}

int bst_get(MapNode* root, const char* key, int* out) {
    if (root == NULL) return 0;
    int cmp = strcmp(key, root->key);
    if (cmp < 0) return bst_get(root->left, key, out);
    if (cmp > 0) return bst_get(root->right, key, out);
    *out = root->value;
    return 1;
}
```

9. ASCII Diagram

9.1 Hash table map with chaining

Keys: 10, 22, 31, 4, 15. Table size 7. $h(k) = k \% 7$.

```
Bucket array (size 7)
+-----+
| 0 | -> NULL
+-----+
| 1 | -> [22, "twentytwo"] -> [15, "fifteen"] -> NULL
+-----+
| 2 | -> NULL
+-----+
| 3 | -> [10, "ten"] -> [31, "thirtyone"] -> NULL
+-----+
| 4 | -> [4, "four"] -> NULL
+-----+
| 5 | -> NULL
+-----+
| 6 | -> NULL
+-----+
```

```
h(10)=3, h(22)=1, h(31)=3 (collision), h(4)=4, h(15)=1 (collision)
```

9.2 Hash table set with chaining

Same structure, but entries store only keys (no values).

```
+-----+
|  0  | -> NULL
+-----+
|  1  | -> "cherry" -> "banana" -> NULL
+-----+
|  2  | -> NULL
+-----+
|  3  | -> "apple" -> "fig" -> NULL
+-----+
|  4  | -> "elderberry" -> NULL
+-----+
```

9.3 BST map (keys in alphabetical order)

```
      ["mango", 7]
     /      \
["banana", 3] ["peach", 5]
  /      \      \
["apple", 1] ["date", 9] ["tangerine", 2]
```

In-order traversal yields keys sorted:
apple -> banana -> date -> mango -> peach -> tangerine

10. When to Use / When to Avoid

Use a map when:

- You need to associate keys with values (names with phone numbers, words with definitions, IDs with records).
- Lookup by key must be fast -- $O(1)$ average with a hash table.
- You do not need the entries in sorted order (hash table).

Use a map (tree-based) when:

- You need entries in sorted key order (range queries, finding min/max key, in-order traversal).
- You need guaranteed $O(\log n)$ worst-case performance.

Use a set when:

- You need to track membership of unique elements (visited nodes, unique words, distinct IDs).
- You need fast contains/add/remove -- $O(1)$ average with a hash set.
- You need set operations: union, intersection, difference.

Use a set (tree-based) when:

- You need elements in sorted order (finding the smallest or largest element).
- You need guaranteed $O(\log n)$ worst-case performance.

Avoid maps/sets when:

- You only need sequential access by index -- use an array (Ch 3).
- You need to iterate in insertion order -- use a hash table with a separate linked list (ordered map pattern).
- Memory is very tight -- the overhead of hash buckets or tree pointers may not be justified for small collections.

Key C-specific note: since C has no built-in map or set, you must implement one or use a library. For exams, the implementation choices are: hash table (Ch 6) for $O(1)$ average, or BST (Ch 9) for $O(\log n)$ ordered access.

11. Exam Hot Spots

This chapter has **0 MCQ corpus questions assigned to it directly**. Map and set concepts are tested **indirectly** through Ch 6 (hash tables) and Ch 9 (BSTs). The patterns below describe how these questions appear.

Pattern A: Hash table as a map

Q: "A hash table stores key-value pairs. What is the average time to look up a value by key?"

- **Correct:** $O(1)$ average.
- **Wrong:** $O(\log n)$ (that's BST), $O(n)$ (worst case only, not average).

Pattern B: BST as a sorted map

Q: "A BST stores key-value pairs in sorted order. What is the time to find the minimum key?"

- **Correct:** $O(\log n)$ -- follow left pointers from root.
- **Wrong:** $O(1)$ (that's a heap or hash table with a min-heap auxiliary structure).

Pattern C: Set implemented as a hash table

Q: "A set is implemented using a hash table. What is the average time for contains?"

- **Correct:** $O(1)$.
- **Wrong:** $O(n)$ (only worst case with all collisions).

Pattern D: Set operations on two small sets

Q: "Given $A = \{1,3,5\}$ and $B = \{2,3,4\}$, what is $A \cup B$?"

- **Correct:** $\{1,2,3,4,5\}$.
- **Wrong:** $\{3\}$ (that's the intersection), $\{1,5\}$ (that's $A - B$).

Cross-references

- Ch 6 (Hash Tables): hash functions, collisions, chaining, open addressing, load factor. All directly relevant to map/set implementation.
- Ch 9 (BSTs): tree operations, balanced vs unbalanced. Directly relevant to ordered map/set implementation.
- Ch 5 (Stacks & Queues): set operations on tree-based sets use in-order traversal, which uses a stack (implicit or explicit).

12. Common Traps

Wrong	Why it's wrong				
"Maps and sets are the same thing"	A map stores key-value pairs; a set stores keys only. A set can be implemented as a map with dummy values, but they are distinct ADTs.				
"A map can have duplicate keys"	No. Inserting a duplicate key replaces the old value. Keys are				

	always unique.				
"A set can contain duplicate elements"	No. A set stores only unique elements. <code>add("x")</code> twice has the same effect as <code>add("x")</code> once.				
"Hash-based maps keep insertion order"	No. The hash function scatters keys across buckets. Insertion order is lost. (Some implementations add a separate linked list to preserve order, but that is not standard.)				
"Tree-based maps are always faster than hash-based"	No. Tree-based is $O(\log n)$; hash-based is $O(1)$ average. Hash is faster on average; tree is slower but gives sorted order.				
"C has a built-in map type"	No. C has no built-in map, set, dictionary, or associative array. You must implement one or use a library. (Java has <code>HashMap/TreeMap</code> ; C does not.)				
"Set union requires both sets to be the same size"	No. Union works on any two sets regardless of size. The result has at most	A	+	B	elements.
"Intersection of two empty sets is undefined"	No. The intersection of two empty sets is the empty set. It is well-defined.				
"A map's get operation returns a copy of the value"	In C, if the value is a pointer, get returns the pointer (alias), not a deep copy. Modifying the returned pointer modifies the map's internal state. Be careful with ownership.				
"Hash tables	No. $O(1)$ is average				

guarantee $O(1)$ worst case"	case. Worst case is $O(n)$ when all keys collide into one bucket.				
------------------------------	--	--	--	--	--

13. Quick-Recall Card

MAP (Dictionary, Associative Array)

Key-value pairs. Keys unique. Values can duplicate.

put(k,v) -> insert or update

get(k) -> retrieve value or "not found"

containsKey(k) -> boolean

remove(k) -> delete entry

SET

Unique elements. No values.

add(e) -> insert (no effect if present)

remove(e) -> delete

contains(e) -> boolean membership

ADT vs IMPLEMENTATION

ADT = what operations are available (contract)

Implementation = how those operations work (hash table or BST)

Same ADT, different perf. Same interface, different internals.

SET AS MAP

A set can be implemented as a map with dummy values:

Set: {"a","b","c"} == Map: {"a"->1, "b"->1, "c"->1}

HASH-BASED (avg $O(1)$)

put/get/contains: $O(1)$ average, $O(n)$ worst

No ordering guarantee

Extra memory for bucket array + chains

TREE-BASED ($O(\log n)$)

put/get/contains: $O(\log n)$ guaranteed (balanced BST)

Sorted by key

Each node has left/right pointers

SET OPERATIONS

A union B = elements in A or B (or both)

A intersect B = elements in both A and B

A - B = elements in A but not in B

Union: $O(n_A + n_B)$ average (hash), $O(n_A + n_B)$ (tree)

Intersection: $O(\min(n_A, n_B))$ average (hash)

C HAS NO BUILT-IN MAP OR SET

Implement via hash table (Ch 6) or BST (Ch 9).

EXAM GOLD

- Map keys are ALWAYS unique; duplicate put replaces value
- Set elements are ALWAYS unique; add is idempotent
- Hash-based: $O(1)$ average, no ordering
- Tree-based: $O(\log n)$, sorted order
- Set union/intersection/difference are fundamental
- A set is a map with dummy values

\newpage

Chapter 9 -- Binary Trees & Binary Search Trees

Core data structure chapter. Binary trees introduce the recursive, pointer-based world of nonlinear structures. The Binary Search Tree (BST) adds an ordering invariant that gives $O(\log n)$ average search, insert, and delete. The chapter

has **38 MCQ corpus questions** assigned to it, covering BST operations, traversals, tree applications, deletion mechanics, and height analysis.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define a binary tree (node, left/right child, root, leaf, height, depth) and distinguish it from a generic (N-ary) tree.
- State the BST invariant (left < parent < right) and verify whether a given tree satisfies it.
- Trace through insert, search, and delete (all three cases) on a BST and predict the resulting structure.
- Execute inorder, preorder, postorder, and level-order traversals and explain why inorder produces sorted output for a BST.
- Distinguish between balanced, complete, and degenerate binary trees and explain the impact on operation complexity.
- Implement BST node operations in compilable C code.
- Identify when a BST is the right data structure and when a sorted array or balanced tree (AVL) is preferable.

2. Mental Model & Analogy

Think of a **binary tree** as a family tree where each person has at most two children: a left child and a right child. The person at the top is the *root*, people with no children are *leaves*, and each person's *height* is the number of generations between them and the deepest descendant.

A **Binary Search Tree** is like a phonebook where you always split the remaining pages in half. When you open the book, you see a name in the middle: if the name you want comes first alphabetically, you go left; if it comes after, you go right. You repeat this halving until you find the name or run out of pages. The BST invariant -- every left descendant is smaller, every right descendant is larger -- is exactly this "split in half" rule encoded in pointers.

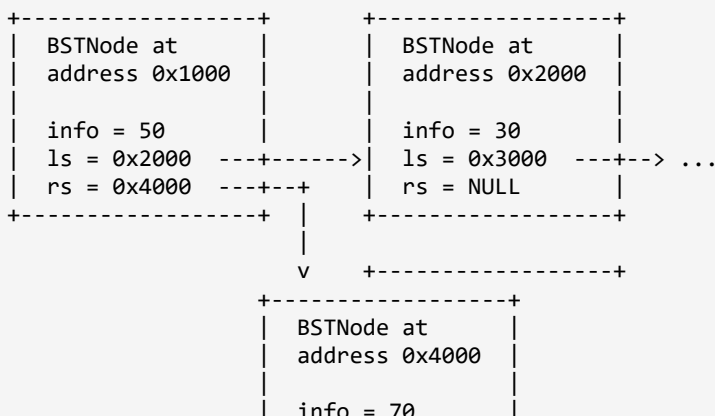
Three rules to internalize:

1. **Every node has at most two children** -- left and right. A node may have zero, one, or two children.
2. **The BST invariant is a global constraint** -- it must hold for every node, not just the root. If even one node violates it, the entire tree is not a valid BST.
3. **Inorder traversal of a BST produces sorted output** -- this is the defining operational property. Left subtree, then root, then right subtree visits keys in ascending order.

3. Memory Layout

Unlike heaps (Chapter 7), which fit into a contiguous array, binary tree nodes are **scattered** across the heap memory. Each node is individually `malloc`'d, and parent-child relationships are maintained by **pointers**, not by index arithmetic.

Node struct (each node on the heap):



```

|  ls = NULL  |
|  rs = NULL  |
+-----+

```

Key facts the exam will test:

- **Scattered heap allocation:** Each node lives at an arbitrary heap address. The tree is connected by pointers, not by physical proximity. This means cache performance is worse than arrays.
- **Left child -- right sibling representation:** For generic (N-ary) trees, the course uses `leftChild` / `rightSibling` pointers. In a binary tree, we use `left` and `right` directly because each node has at most two children.
- **Complete binary trees can use arrays:** If the tree is complete (all levels full except possibly the last, filled left to right), it can be stored in an array using the index formulas from Chapter 7. But a BST is generally *not* complete, so we use pointers.

4. Definition & Properties

Binary tree -- a tree data structure in which each node has at most two children, referred to as the *left child* and the *right child*.

Binary Search Tree (BST) -- a binary tree in which each node follows the **BST invariant**:

For every node N:
 All values in N's left subtree < N's value
 All values in N's right subtree > N's value

This is a total order relation with the standard properties: equality, completeness, consistency, and transitivity (as defined in the course material).

Terminology:

Term	Definition
Root	The topmost node; has no parent
Parent	The node directly above a given node
Child	A node directly below a given node (left or right)
Leaf (terminal node)	A node with no children
Internal node	A node with at least one child
Height of a node	Length of the longest path from that node to a leaf
Height of the tree	Height of the root = length of longest root-to-leaf path
Depth of a node	Length of the path from the root to that node
Subtree	A tree rooted at one of the node's children
Sibling	Nodes sharing the same parent

Tree types:

Type	Definition
Full binary tree	Every node has 0 or 2 children (no node has exactly 1 child)
Complete binary tree	Every level is fully filled except possibly the last, which is filled left to right
Perfect binary tree	All internal nodes have two children and all leaves are at the same

	level
Degenerate tree	Every internal node has exactly one child -- equivalent to a linked list
Balanced tree	Height is $O(\log n)$; e.g., AVL trees (Chapter 12)

Properties of BST:

- A BST is a **dynamic structure** -- nodes can be added or removed at any time without knowing the count in advance.
- A BST is **not linear** -- it is not an array or linked list; elements are not accessed sequentially or by direct index.
- BST elements are **not accessed sequentially** (that is a property of arrays/linked lists).
- BST provides **logarithmic average** performance for search, insert, and delete -- but only when the tree is reasonably balanced.

5. Operations -- Step-by-Step Mechanics

5.1 Search

Goal: Find whether a value x exists in the BST.

1. Start at the root.
2. If root is NULL, x is not in the tree. Return NULL.
3. If $x == \text{root} \rightarrow \text{info}$, found it. Return root.
4. If $x < \text{root} \rightarrow \text{info}$, search the left subtree (recurse left).
5. If $x > \text{root} \rightarrow \text{info}$, search the right subtree (recurse right).

Why $O(\log n)$ average: Each comparison eliminates half the remaining nodes (in a balanced tree). Worst case $O(n)$ if the tree is degenerate.

5.2 Insert

Goal: Insert a new value x into the BST.

1. Start at the root.
2. If the tree is empty, create a new node and make it the root.
3. If $x < \text{current} \rightarrow \text{info}$:
If $\text{current} \rightarrow \text{left}$ is NULL, create a new node there.
Else, recurse into the left subtree.
4. If $x > \text{current} \rightarrow \text{info}$:
If $\text{current} \rightarrow \text{right}$ is NULL, create a new node there.
Else, recurse into the right subtree.
5. If $x == \text{current} \rightarrow \text{info}$, duplicate. Do nothing (or handle per policy).

Key fact: A new value is always inserted as a **leaf** node. The insert operation walks from the root to a leaf position, so it does not rearrange existing nodes.

5.3 Delete

Delete is the most complex BST operation. Three cases depending on the number of children of the node to be deleted.

Case 1: Deleting a leaf (0 children)

Simply remove the node. Set the parent's pointer to NULL.

Case 2: Deleting a node with exactly 1 child

Replace the node with its only child.
The child "takes the place" of the deleted node.

Case 3: Deleting a node with 2 children

This is the exam's favorite. Two valid strategies exist:

Strategy A -- In-order successor (most common):

1. Find the in-order successor of the node to delete:
 - Go to the right child once.
 - Then go as far left as possible.
 - This gives the smallest value in the right subtree.
2. Copy the successor's value into the node to be deleted.
3. Delete the successor node (which has 0 or 1 children).

Strategy B -- In-order predecessor:

1. Find the in-order predecessor:
 - Go to the left child once.
 - Then go as far right as possible.
 - This gives the largest value in the left subtree.
2. Copy the predecessor's value into the node to be deleted.
3. Delete the predecessor node (which has 0 or 1 children).

Both strategies preserve the BST invariant. The exam accepts either the maximum of the left subtree (predecessor) or the minimum of the right subtree (successor) as valid.

Important: The in-order successor never has a left child. The in-order predecessor never has a right child. This is why their deletion always falls into Case 1 or Case 2.

5.4 Traversals

Four standard traversals for binary trees:

In-order (Left -- Root -- Right):

1. Traverse the left subtree recursively.
2. Visit the root (process its value).
3. Traverse the right subtree recursively.

For a BST, inorder produces **ascending sorted order**. This is the most important traversal.

Pre-order (Root -- Left -- Right):

1. Visit the root.
2. Traverse the left subtree recursively.
3. Traverse the right subtree recursively.

Used to copy/serialize a tree (root is recorded first).

Post-order (Left -- Right -- Root):

1. Traverse the left subtree recursively.
2. Traverse the right subtree recursively.
3. Visit the root.

Root nodes appear last. Used for deleting a tree (children deleted before parent).

Level-order (BFS with queue):

1. Enqueue the root.
2. While the queue is not empty:
 - Dequeue a node, process it.
 - Enqueue its left child (if any).
 - Enqueue its right child (if any).

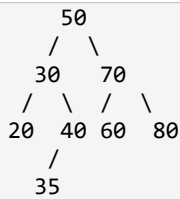
Processes nodes level by level, left to right. Uses a queue (not a stack).

Exam fact: In a postorder traversal, the root node appears at the end. In an inorder traversal of a BST, the output is sorted in ascending order. A preorder traversal visits the root first.

6. Trace-Through Example

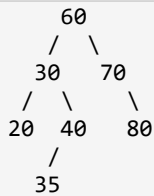
Build a BST by inserting 8 keys: 50, 30, 70, 20, 40, 60, 80, 35

After all insertions:



Delete node 50 (root, 2 children) using in-order successor:

1. Successor of 50 = min of right subtree = go right to 70, then left to 60, then left again -- 60 has no left child. Successor = 60.
2. Copy 60 into root position.
3. Delete node 60 (a leaf):



All four traversals on the final tree:

In-order: 20 30 35 40 60 70 80 (sorted!)
 Pre-order: 60 30 20 40 35 70 80
 Post-order: 20 35 40 30 80 70 60
 Level-order: 60 30 70 20 40 35 80

7. Operations + Big-O Table

Operation	Average	Worst Case	Notes
Search	$O(\log n)$	$O(n)$	Worst case: degenerate tree
Insert	$O(\log n)$	$O(n)$	Always inserts as a leaf
Delete	$O(\log n)$	$O(n)$	Must find successor/predecessor
In-order traversal	$O(n)$	$O(n)$	Visits every node once
Pre-order traversal	$O(n)$	$O(n)$	Visits every node once
Post-order traversal	$O(n)$	$O(n)$	Visits every node once
Level-order traversal	$O(n)$	$O(n)$	Uses a queue, visits every node once
Find min / max	$O(\log n)$	$O(n)$	Min = leftmost, Max = rightmost
Height	$O(n)$	$O(n)$	Must visit every node
Check if BST	$O(n)$	$O(n)$	Use min/max bounds recursively

Key insight: The average case assumes a randomly built BST, where height is approximately $1.44 * \log_2(n)$. The worst case (sorted input) produces a degenerate tree with height $n-1$, giving $O(n)$ per operation. This is why balanced BSTs (AVL, Red-Black) exist.

8. C Code Reference

8.1 Node struct

```
#include <stdio.h>
#include <stdlib.h>
```

```

struct BSTNode {
    int info;
    struct BSTNode* ls;    /* left subtree */
    struct BSTNode* rs;    /* right subtree */
};

```

8.2 Insert (iterative)

```

struct BSTNode {
    int info;
    struct BSTNode* ls;
    struct BSTNode* rs;
};
#include <stdlib.h>
struct BSTNode* bst_insert(struct BSTNode* root, int x) {
    struct BSTNode* new = (struct BSTNode*)malloc(sizeof(struct BSTNode));
    new->info = x;
    new->ls = NULL;
    new->rs = NULL;

    if (root == NULL)
        return new;

    struct BSTNode* curr = root;
    struct BSTNode* parent = NULL;
    while (curr != NULL) {
        parent = curr;
        if (x < curr->info)
            curr = curr->ls;
        else
            curr = curr->rs;
    }
    if (x < parent->info)
        parent->ls = new;
    else
        parent->rs = new;
    return root;
}
int main() {
    return 0;
}

```

8.3 Search

```

#include <stdio.h>
#include <stdlib.h>
struct BSTNode {
    int info;
    struct BSTNode* ls;
    struct BSTNode* rs;
};
struct BSTNode* bst_search(struct BSTNode* root, int x) {
    struct BSTNode* curr = root;
    while (curr != NULL) {
        if (x == curr->info)
            return curr;
        else if (x < curr->info)
            curr = curr->ls;
        else
            curr = curr->rs;
    }
    return NULL;
}

```

```
int main() {

    return 0;

}
```

8.4 Delete (3 cases)

```
struct BSTNode {
    int info;
    struct BSTNode* ls;
    struct BSTNode* rs;
};
#include <stdlib.h>
struct BSTNode* bst_find_min(struct BSTNode* root) {
    while (root->ls != NULL)
        root = root->ls;
    return root;
}
struct BSTNode* bst_delete(struct BSTNode* root, int x) {
    if (root == NULL)
        return NULL;

    if (x < root->info) {
        root->ls = bst_delete(root->ls, x);
    } else if (x > root->info) {
        root->rs = bst_delete(root->rs, x);
    } else {
        /* Case 1 & 2: Leaf or one child */
        if (root->ls == NULL) {
            struct BSTNode* temp = root->rs;
            free(root);
            return temp;
        }
        if (root->rs == NULL) {
            struct BSTNode* temp = root->ls;
            free(root);
            return temp;
        }
        /* Case 3: two children -- use in-order successor */
        struct BSTNode* succ = bst_find_min(root->rs);
        root->info = succ->info;
        root->rs = bst_delete(root->rs, succ->info);
    }
    return root;
}
int main() {

    return 0;

}
```

8.5 In-order traversal (recursive)

```
struct BSTNode {
    int info;
    struct BSTNode* ls;
    struct BSTNode* rs;
};
#include <stdio.h>
void bst_inorder(struct BSTNode* root) {
    if (root != NULL) {
        bst_inorder(root->ls);
        printf("%d ", root->info);
        bst_inorder(root->rs);
    }
}
```

```

}
int main() {

    return 0;
}

```

8.6 Level-order traversal (BFS with queue)

```

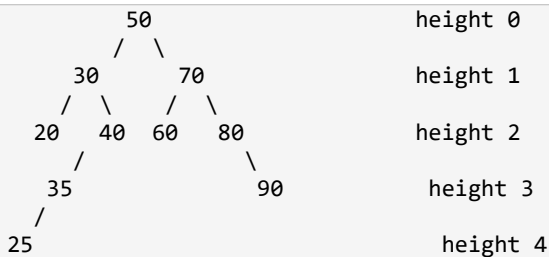
struct BSTNode {
    int info;
    struct BSTNode* ls;
    struct BSTNode* rs;
};
#include <stdio.h>
#define MAX 100
void bst_level_order(struct BSTNode* root) {
    if (root == NULL) return;
    struct BSTNode* queue[MAX];
    int front = 0, rear = 0;
    queue[rear++] = root;
    while (front != rear) {
        struct BSTNode* curr = queue[front++];
        printf("%d ", curr->info);
        if (curr->ls != NULL)
            queue[rear++] = curr->ls;
        if (curr->rs != NULL)
            queue[rear++] = curr->rs;
    }
}
int main() {

    return 0;
}

```

9. ASCII Diagram

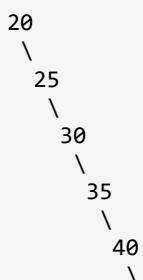
Balanced BST with 10 nodes (height = 3):



Height = 4 (root at height 0, deepest leaf at height 4)
Depth of 35 = 3 (root -> 30 -> 40 -> 35)

Degenerate (skewed) tree -- same 6 keys, sorted input:

Insert 20, 25, 30, 35, 40, 50 in order:



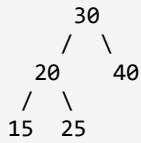
50

Height = 5 (linear chain, equivalent to a linked list)

Search for 50: must visit all 6 nodes --> $O(n)$

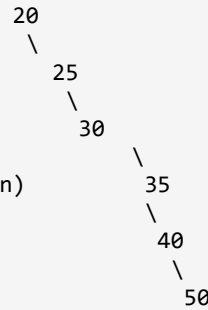
Comparison:

Balanced BST (6 nodes):



Height = 2, search = $O(\log n)$

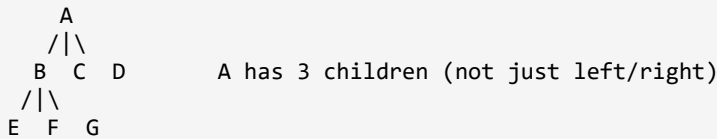
Degenerate tree (6 nodes):



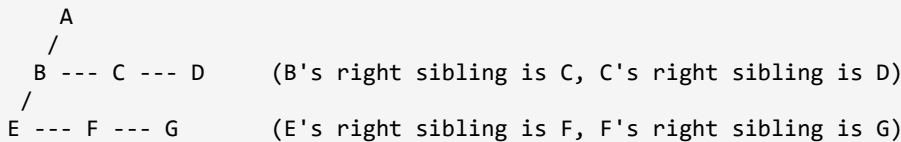
Height = 5, search = $O(n)$

Generic (N-ary) tree representation:

A generic tree node has a list of children:



Binary representation (left child -- right sibling):



10. When to Use / When to Avoid

Use a BST when:

- You need **dynamic sorted data** -- insertions and deletions while maintaining sort order.
- You need **range queries** -- find all keys between x and y using inorder traversal.
- You need **successor/predecessor** operations -- find the next larger or smaller element.
- The input is **randomly ordered** -- the tree stays approximately balanced.
- You need a **foundation for more advanced structures** -- AVL trees, Red-Black trees, B-trees are all BST variants.

Avoid a BST when:

- The input is **already sorted** -- a naive BST will become a degenerate linked list ($O(n)$ operations). Use an AVL tree or a sorted array instead.
- You need **guaranteed $O(\log n)$** -- a plain BST has no balance guarantee. Use AVL or Red-Black trees.
- You need **$O(1)$ lookup by index** -- BST does not support random access. Use an array.
- The data is **static** (no insertions/deletions) -- a sorted array with binary search gives $O(\log n)$ lookup with better cache performance.
- You need **priority queue semantics** -- use a heap (Chapter 7).

Application domains (from the corpus):

- **File system organization** -- directories and subdirectories form a tree.

- **Website navigation structure** -- sitemaps are trees.
- **Organizational charts** -- hierarchical company structures.
- **Expression trees** -- each leaf is an operand, each internal node is an operator.
- **Decision trees / taxonomy trees** -- each non-leaf is a decision, each leaf is a conclusion.
- **Database indexing** -- B-trees (BST generalization) are the standard index structure.

11. Exam Hot Spots

This chapter has **38 MCQ corpus questions** assigned to it. They cluster into the following patterns.

Pattern A: Applications of trees

Q30, Q97, Q115, Q225, Q471, Q973, Q996, Q3958 (8 identical duplicates): "In which domains are trees used as applied data structures?"

- **Correct:** "Defining a website's tree structure", "Organizing files in an operating system", "Organizing hierarchical structures in companies"
- **Wrong:** "Implementing sequential calculations" -- trees are nonlinear.

Pattern B: BST deletion -- root replacement

Q35, Q71, Q105, Q113, Q222, Q473, Q975, Q998, Q3959 (9 duplicates): "What values can replace the root after deleting node 55 from BST with level-order 55-33-77-11-44-66-88?"

- **Correct:** **44** and **66** -- in-order predecessor (max of left subtree) and in-order successor (min of right subtree).
- **Wrong:** 77, 88 -- neither is the immediate predecessor or successor of 55.

Pattern C: BST operations

Q40, Q110, Q1003, Q3957: "Operations applicable to BSTs include:" -- Correct: "Search node by key value", "Postorder traversal", "Insert new node", "Clustering nodes based on a key value". Wrong: "Topological sort" (graph operation, not BST).

Pattern D: Tree traversal order

Q73: "In postorder traversal, the order is:" -- Correct: "Left subtree -- right subtree -- current node". Wrong: "Current node -- subtrees" (preorder).

Q62: "What traversal does saveflightsVector use?" (code-based) -- Correct: **preorder** (root processed before children).

Q7950: "What traversal visits left, node, then right?" -- Correct: **In-order**. Wrong: Pre-order (root first), Post-order (root last), Level-order (BFS).

Pattern E: BST properties -- true/false

Q246, Q501: "Which statements are true about BST?" -- Correct: "BST is a dynamic structure (nodes added/removed without knowing count in advance)". Wrong: "linear structure", "accessed sequentially", "accessed by direct index".

Q3961: "Binary search trees are:" -- Correct: "Binary trees". Wrong: "Generic trees" (generic trees allow N children; BSTs allow at most 2).

Q3973: "All BSTs are balanced." -- **False**. Sorted input creates a degenerate tree. Only AVL/Red-Black guarantee $O(\log n)$.

Pattern F: BST inorder traversal sorted output

Q3964, Q3965: "Which traversal displays BST elements in ascending sorted order?" / "What can be said about the inorder traversal of a BST?" -- Correct: **In-order**. Wrong: Pre-order, Post-order, Level-order -- none produce sorted output.

Q3967: "In a postorder traversal, root nodes appear at the end." -- **True**. Postorder visits left-right-root.

Pattern G: BST structural properties

Q7949: "In a BST, where are values smaller than a node usually stored?" -- Correct: **In the left subtree**. Wrong: "Right subtree", "Only at the root", "Outside the tree".

Q3968: "In a BST, a new value is always inserted as:" -- Correct: **A leaf**. Wrong: "Anywhere", "Root", "Next to the largest element".

Q3969: "In a BST, the right child's value is always:" -- Correct: **Greater than the parent's value** (strictly greater, not $\geq$).

Q3962: "Given node X with 2 children and Y its in-order successor, which is true?" -- Correct: **Y has no left child**. The successor is the leftmost node in the right subtree.

Pattern H: BST height and recursion

Q3966: "The height of a randomly built BST is:" -- Correct: **$O(\log n)$** . Wrong: vague distance definitions.

Q3990: "The height of a tree is:" -- Correct: "Length of the longest path from root to a leaf node". Wrong: "path from root to a node" (that is depth), "path from one node to another" (too vague).

Q3971: "The main method for solving binary tree problems is:" -- Correct: **Recursion**. Wrong: "Permutations", "Backtracking".

12. Common Traps

Wrong	Why it's wrong
"A BST is a linear structure"	BST is nonlinear. Arrays and linked lists are linear.
"All BSTs are balanced"	Sorted input creates a degenerate linked list. Only AVL/Red-Black guarantee balance.
"BST search is always $O(\log n)$ "	Only on average. Worst case (degenerate) is $O(n)$.
"Delete with 2 children: just remove the node"	Must replace with successor or predecessor first.
"Delete with 2 children: only successor works"	Both successor (min of right subtree) and predecessor (max of left subtree) are valid.
"The in-order successor has a left child"	Successor is the leftmost node in the right subtree -- by definition it has no left child.
"In-order traversal of any tree gives sorted output"	Only for a BST. Generic binary trees produce arbitrary order.
"Postorder visits the root first"	Postorder: left-right-root (root last). Preorder: root-left-right (root first).
"Level-order uses a stack"	Level-order (BFS) uses a queue . DFS traversals use a stack.
"Height of a tree with 1 node is 1"	Height = edges on longest root-to-leaf path. Single node = height 0. Empty = -1.
"Topological sort is a BST operation"	Topological sort is a graph algorithm for DAGs.
"Left child is always smaller than right child"	BST invariant: left < parent < right. No direct sibling constraint.
"Insert creates a new root"	Insert creates a new leaf . Root only changes if tree was empty.

13. Quick-Recall Card

DEFINITION

Binary tree = each node has at most 2 children (left, right).

BST invariant = for every node: left subtree < node < right subtree.

Leaf = node with no children.
Height = longest root-to-leaf path (in edges).
Depth = path length from root to that node.

BST INVARIANT (must hold for EVERY node)

left subtree values < node value < right subtree values

OPERATIONS

Search: walk left or right by comparison $O(\log n)$ avg, $O(n)$ worst
Insert: walk to leaf position, create node $O(\log n)$ avg, $O(n)$ worst
Delete: 3 cases --
Case 1: leaf --> remove it
Case 2: one child --> replace with child
Case 3: two children --> replace with successor or predecessor,
then delete the successor/predecessor

DELETE WITH 2 CHILDREN -- TWO VALID STRATEGIES

Strategy A: Replace with in-order successor (min of right subtree).
Strategy B: Replace with in-order predecessor (max of left subtree).
The successor never has a left child. The predecessor never has a right child.

TRAVERSALS

Inorder: Left -- Root -- Right (BST => sorted output)
Preorder: Root -- Left -- Right (root first, used for serialization)
Postorder: Left -- Right -- Root (root last, used for deletion)
Level-order: BFS with queue (level by level, left to right)

TREE TYPES

Full: every node has 0 or 2 children
Complete: all levels full except last, filled left to right
Perfect: all levels full, all leaves at same depth
Degenerate: each node has 1 child --> linked list, height = $n-1$
Balanced: height = $O(\log n)$ --> AVL, Red-Black trees

C STRUCT

```
struct BSTNode {  
    int info;  
    struct BSTNode* ls; /* left subtree */  
    struct BSTNode* rs; /* right subtree */  
};
```

EXAM GOLD

- Inorder traversal of a BST produces SORTED output.
- BST delete with 2 children: use successor OR predecessor.
- In-order successor has NO left child.
- Not all BSTs are balanced (sorted input => degenerate).
- Level-order uses a QUEUE, not a stack.
- Topological sort is a GRAPH operation, not a BST operation.
- BST is a dynamic structure (nodes added/removed freely).
- New values are always inserted as LEAVES.
- Right child value > parent value (strictly greater).

WHEN TO USE

Dynamic sorted data, range queries, successor/predecessor.
Random input expected. Foundation for AVL/Red-Black trees.

WHEN TO AVOID

Sorted input (use AVL or sorted array).
Need guaranteed $O(\log n)$ (use balanced BST).
Static data (use sorted array + binary search).
Priority queue (use a heap, Ch 7).

\newpage

Chapter 10 -- AVL Trees

Core data structure chapter. AVL trees fix the fatal weakness of plain BSTs -- sorted input degenerating to $O(n)$ -- by enforcing a balance invariant after every modification. The chapter has **49 MCQ corpus questions** assigned to it, covering rotation mechanics, balance-factor triggers, structural properties, and rebalancing after insert and delete.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define an AVL tree and state the balance invariant (height difference at most 1 for every node).
- Compute the balance factor (GE) of any node and determine whether the tree is valid.
- Identify the four rotation cases (LL, RR, LR, RL) from a given imbalance configuration.
- Trace through insert and delete operations, applying the correct rotation(s) to restore balance.
- Explain why AVL trees guarantee $O(\log n)$ worst-case for search, insert, and delete.
- Implement AVL node operations in compilable C code, including rotation helpers.
- Distinguish AVL trees from plain BSTs and other balanced structures (Red-Black, B-trees).

2. Mental Model & Analogy

Think of an AVL tree as a **librarian who constantly reshelves books**. In a plain BST (Chapter 9), the librarian shelves books in order but never checks whether one side of the shelf is growing taller than the other. Eventually, if books arrive in sorted order, the left side of the shelf collapses into a single tall column while the right side is empty -- every lookup now means climbing the entire column.

An AVL librarian is different. After every single book is placed or removed, the librarian walks back up the shelf, checking the height on each side. If one side has grown more than one unit taller than the other, the librarian performs a quick rearrangement -- sometimes rotating a section of books to the left, sometimes to the right, sometimes both -- so the shelf stays roughly level. The result: no matter how many books arrive, the shelf never grows more than about $1.44 * \log_2(n)$ levels tall, and every lookup is guaranteed fast.

Three rules to internalize:

1. **The balance invariant is global** -- it must hold for *every* node, not just the root. A single node with height difference of 2 anywhere in the tree means the tree is not a valid AVL tree.
2. **Rebalancing happens bottom-up** -- after inserting or deleting, you walk back from the modified node toward the root, checking and fixing balance at each ancestor.
3. **At most two rotations fix any imbalance** -- regardless of how the imbalance occurred, a single or double rotation (at most two $O(1)$ pointer operations) is always sufficient.

3. Memory Layout

AVL tree nodes live in heap memory, identical to BST nodes (Chapter 9), with one additional field: the **balance factor** (called `eqDegree` or `GE` in the course material).

AVL Node struct (each node on the heap):

```
+-----+
| AVLNode at |      | AVLNode at |
| 0x1000     |      | 0x2000     |
| info = 50  |      | info = 30  |
| left = 0x2000 ---+-----> | left = 0x3000 ---+-----> ...
| right = 0x4000 -+-- |      | right = NULL   |
| GE = -1     |      | GE = 0     |
+-----+          +-----+
```

```

      v      |  AVLNode at
+-----+ 0x4000
|  info = 70
|  left = NULL
|  right = NULL
|  GE = 0
+-----+

```

Key facts the exam will test:

- **Extra field vs. BST:** The only structural difference from a BST node is the `GE` field. Every other aspect -- scattered heap allocation, pointer-based parent-child links -- is identical.
- **Balance factor storage:** The course material uses a stored `eqDegree` field in the node struct (Q3989). Some implementations compute it from subtree heights on the fly.
- **Node structure:** `info` (the key), `left` pointer, `right` pointer, and `GE` (balance factor) -- four fields total.

4. Definition & Properties

AVL Tree -- a binary search tree (BST) in which, for every node, the heights of the left and right subtrees differ by at most one unit. Named after inventors Georgy Adelson-Velsky and Evgenii Landis (1962).

Balance Factor (GE):

$GE = H(\text{left subtree}) - H(\text{right subtree})$

Where $H(T)$ is the height of subtree T (the length of the longest path from the subtree's root to a leaf, in edges). The height of an empty (NULL) subtree is 0.

The AVL invariant:

For every node N :
 $GE(N) \in \{-1, 0, +1\}$

If any node has `GE` outside this set, the tree is **not** a valid AVL tree and must be rebalanced.

Properties:

Property	Value
Balance factor range	$\{-1, 0, +1\}$ for every node
Height of AVL with n nodes	$\leq 1.44 * \log_2(n + 2) - 0.328$
Search (worst case)	$O(\log n)$
Insert (worst case)	$O(\log n)$
Delete (worst case)	$O(\log n)$
Rotation per fix	$O(1)$ -- at most 2 rotations
When to check balance	After every insertion and every deletion

Balance factor interpretation:

GE Value	Meaning
0	Left and right subtrees are the same height -- perfectly balanced
+1	Left subtree is one unit taller -- minor left-heavy imbalance
-1	Right subtree is one unit taller -- minor right-heavy imbalance
+2	Left subtree is two units taller -- strong imbalance (not AVL)

-2	Right subtree is two units taller -- strong imbalance (not AVL)
----	--

When does rebalancing trigger? A node's left and right subtree heights differ by more than 1 (Q232, Q480). This means GE is +2 or -2.

The balance invariant applies to: Every node -- not just the root, not just leaves, not just the largest element (Q3994).

AVL trees are also called: Balanced trees (Q3977). They are also known as "self-balanced binary search trees."

Key characteristics (Q242, Q251, Q490, Q496):

- The difference between the heights of the left and right subtrees of any node is at most 1.
- Insertion of a node requires rebalancing operations to maintain the AVL property.
- AVL trees allow search, insert, and delete in $O(\log n)$ worst-case time.
- AVL trees are NOT always complete (not all levels are full except possibly the last).

5. Operations -- Step-by-Step Mechanics

5.1 Insert with Rebalance

1. BST insert (root to leaf position).
2. Walk BACK UP toward the root.
3. At each ancestor, recompute GE.
4. If GE in $\{-2, +2\}$, that node is the PIVOT.
5. Identify rotation type from pivot GE and child GE (see table).
6. Apply rotation. At most 2 rotations per insert.

Traversal direction: Insertion traverses from root to the insertion position (downward), then traverses in reverse from the inserted node back to the root (upward) for rebalancing (Q3980).

5.2 Delete with Rebalance

1. BST delete (3 cases from Chapter 9).
2. Walk BACK UP toward the root, recomputing GE at each ancestor.
3. If GE in $\{-2, +2\}$, apply rotation.
4. Deletion may cause MULTIPLE imbalances -- continue checking up the path. More complex than insert.

5.3 The Four Rotation Types

The rotation to apply depends on two factors: the **pivot's GE** and the **child's GE** on the imbalance direction.

Rotation Decision Table (from course material):

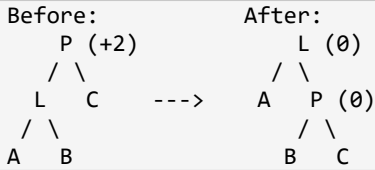
Pivot GE	Child GE	Rotation	Name
+2	+1	Single right rotation	RR (left-left case)
+2	-1	Double: left on child, right on pivot	LR (left-right case)
-2	-1	Single left rotation	LL (right-right case)
-2	+1	Double: right on child, left on pivot	RL (right-left case)

Important: When the child is NULL or has GE = 0 (which happens during deletion), treat it as having the same sign as the pivot for rotation selection.

Exam trap (Q46, Q90, Q124, Q209, Q460, Q964, Q986, Q3974): The RL rotation is triggered when the **root has balance factor +2 and the right child has balance factor -1** -- not "left imbalance with right child" or "root GE = -2 with left child +1".

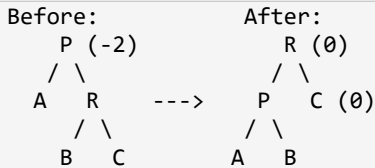
5.3.1 Single Right Rotation (RR -- left-left imbalance)

Used when: **Pivot GE = +2, left child GE = +1.**



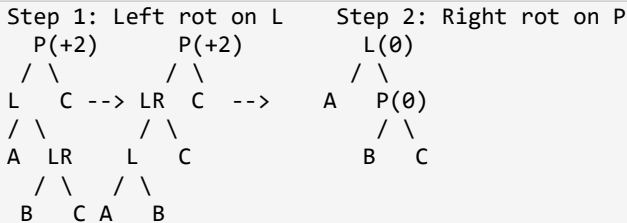
5.3.2 Single Left Rotation (LL -- right-right imbalance)

Used when: **Pivot GE = -2, right child GE = -1.**



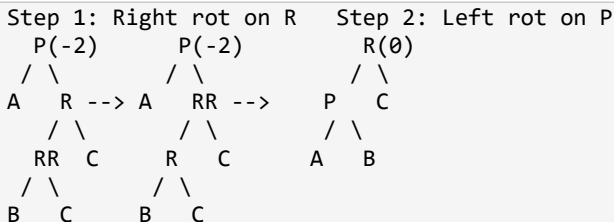
5.3.3 Double Rotation: Left-Right (LR case)

Used when: **Pivot GE = +2, left child GE = -1.**



5.3.4 Double Rotation: Right-Left (RL case)

Used when: **Pivot GE = -2, right child GE = +1.**



Before: After: P (GE=+2) L (GE=0) /\ /\ L C ---> A P (GE=0 or -1) /\ /\ A B B C

- P rotates to the right.
- L becomes the new root of this subtree.
- L's right child B becomes P's left child.
- P's right child C stays as is.

5.3.2 Single Left Rotation (LL case -- right-right imbalance)

Used when: ****Pivot GE = -2, right child GE = -1**.**

The pivot node is too heavy on the right, and the right child is also heavy on the right. A single left rotation fixes this.

Before: After: P (GE=-2) R (GE=0) /\ /\ A R ---> P C (GE=0 or +1) /\ /\ B C A B

- P rotates to the left.
- R becomes the new root of this subtree.
- R's left child B becomes P's right child.
- P's left child A stays as is.

5.3.3 Double Rotation: Left-Right (LR case)

Used when: ****Pivot GE = +2, left child GE = -1****.

The pivot is heavy on the left, but the left child is heavy on the right. A single rotation won't fix this -- you need two rotations.

Step 1: Left rotation on child L Step 2: Right rotation on pivot P P (GE=+2) P (GE=+2) L (GE=0) /\ /\
/\ L C ---> LR C ---> A P (GE=0) /\ /\ /\ A LR L C B C /\ /\ B C
A B

After both rotations, the subtree is balanced with GE in {-1, 0, +1} for all nodes.

5.3.4 Double Rotation: Right-Left (RL case)

Used when: ****Pivot GE = -2, right child GE = +1****.

The pivot is heavy on the right, but the right child is heavy on the left. A single rotation won't fix this -- you need two rotations.

Step 1: Right rotation on child R Step 2: Left rotation on pivot P P (GE=-2) P (GE=-2) R (GE=0) /\ /\
/\ A R ---> A RR ---> P C /\ /\ /\ RR C R C A B /\ /\ B C B
C

5.4 Height Function

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct AVLNode { int info; struct AVLNode left; struct AVLNode right; int GE; }; int max(int a, int b) { return a > b ? a : b; }
int height(struct AVLNode root) { if (root == NULL) return 0; return 1 + max(height(root->left), height(root->right)); }
int computeGE(struct AVLNode node) { if (node == NULL) return 0; return height(node->left) - height(node->right); }
int H(struct AVLNode root) { return (root == NULL) ? 0 : 1 + max(H(root->left), H(root->right)); }
int main() { struct AVLNode rotateRight(struct AVLNode pivot); struct AVLNode rotateLeft(struct AVLNode pivot); struct AVLNode rebalance(struct AVLNode* node);

return 0; }
```

The height of a void subtree is 0 (not -1 as in some conventions). The course material explicitly states: "The height of a void subtree is 0."

6. Trace-Through Example

6.1 RR Case (Single Right Rotation) -- Insert 30, 20, 10

Insert 30: 30(0)

Insert 20: 30(+1) / 20(0)

Insert 10: 30(+2) <-- PIVOT / 20(+1) / 10(0)

Pivot=30, GE=+2. Left child=20, GE=+1. Left-left imbalance. Apply single right rotation on 30:

20(0) / \ 10 30 (0) (0)

6.2 LL Case (Single Left Rotation) -- Insert 30, 50, 60

Insert 30: 30(0)

Insert 50: 30(-1) \ 50(0)

Insert 60: 30(-2) <-- PIVOT \ 50(-1) \ 60(0)

Pivot=30, GE=-2. Right child=50, GE=-1. Right-right imbalance. Apply single left rotation on 30:

50(0) / \ 30 60 (0) (0)

30(GE=0)

Insert 50: Goes right of 30.

30(GE=-1)

\
50(GE=0)

Insert 60: Goes right of 50.

30(GE=-2) <-- PIVOT

\
50(GE=-1)
\
60(GE=0)

Pivot = 30, GE = -2. Right child = 50, GE = -1. This is **right-right imbalance**.

Apply single left rotation on 30:

50(GE=0)
/
30 60
(0) (0)

6.3 LR Case (Double Rotation) -- Insert 30, 10, 20

Insert 30: 30(0)

Insert 10: 30(+1)
/
10(0)

Insert 20: 30(+2) <-- PIVOT
/
10(-1)
\
20(0)

Pivot=30, GE=+2. Left child=10, GE=-1. Left-right imbalance.

Step 1: Left rotation on child 10:

30(+2)
/
20(0)
/
10(0)

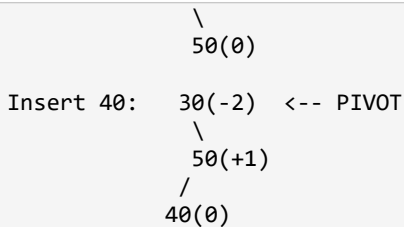
Step 2: Right rotation on pivot 30:

20(0)
/
10 30
(0) (0)

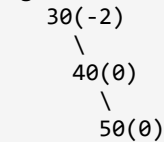
6.4 RL Case (Double Rotation) -- Insert 30, 50, 40

Insert 30: 30(0)

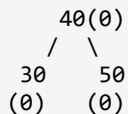
Insert 50: 30(-1)



Pivot=30, GE=-2. Right child=50, GE=+1. Right-left imbalance.
 Step 1: Right rotation on child 50:



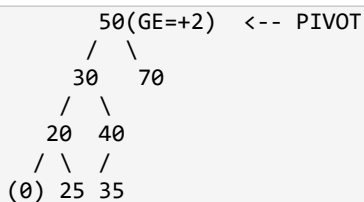
Step 2: Left rotation on pivot 30:



6.5 Combined Trace: Insert 8 Keys Triggering a Rotation

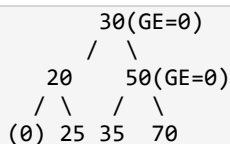
Insert sequence: 50, 30, 70, 20, 40, 25, 35, 28

After inserting 50, 30, 70, 20, 40, 25, 35 -- the tree is:

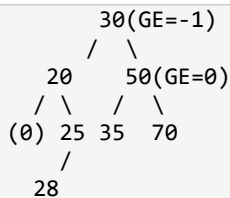


Pivot = 50, GE = +2. Left child = 30, GE = +1. Left-left imbalance.

Apply single right rotation on 50:



Insert 28 (goes right of 25):



No node has GE outside {-1, 0, +1}. All valid.

Final tree (in-order: 20, 25, 28, 30, 35, 50, 70)

7. Operations + Big-O Table

Operation	Average	Worst Case	Notes
Search	$O(\log n)$	$O(\log n)$	Guaranteed -- unlike plain BST
Insert	$O(\log n)$	$O(\log n)$	Walk down + walk up + at most 2 rotations

Delete	$O(\log n)$	$O(\log n)$	Walk down + walk up + possibly multiple rebalancing
Rotation	$O(1)$	$O(1)$	At most 2 rotations per insert, possibly more per delete
Find min / max	$O(\log n)$	$O(\log n)$	Min = leftmost, Max = rightmost
Height	$O(n)$	$O(n)$	Must visit every node (or $O(1)$ if stored)
In-order traversal	$O(n)$	$O(n)$	Visits every node once -- produces sorted output
Compute GE	$O(1)$	$O(1)$	If stored in node; $O(\text{height})$ if computed from heights

Key insight: AVL trees transform BST's **average** $O(\log n)$ into **worst-case** $O(\log n)$. The height is bounded by $\sim 1.44 * \log_2(n)$, so even in the worst case (sorted input), operations remain logarithmic. This is the entire point of AVL trees.

Comparison with plain BST:

Operation	BST (average)	BST (worst)	AVL (worst)
Search	$O(\log n)$	$O(n)$	$O(\log n)$
Insert	$O(\log n)$	$O(n)$	$O(\log n)$
Delete	$O(\log n)$	$O(n)$	$O(\log n)$

8. C Code Reference

8.1 Node Struct

```
#include <stdio.h>
#include <stdlib.h>

struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE; /* grade of equilibrium / balance factor */
};
```

8.2 Height and Balance Factor Helpers

```
#include <stdio.h>
#include <stdlib.h>
typedef struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE;
} AVLNode;

int height(AVLNode* root) {
    if (root == NULL) return 0;
    int h1 = height(root->left);
    int hr = height(root->right);
    return 1 + (h1 > hr ? h1 : hr);
}

int computeGE(AVLNode* node) {
```

```

    if (node == NULL) return 0;
    return height(node->left) - height(node->right);
}

```

8.3 Right Rotation (RR case)

```

#include <stdio.h>
#include <stdlib.h>
struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE;
};
int max(int a, int b) { return a > b ? a : b; }
int height(struct AVLNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}
int computeGE(struct AVLNode* node) {
    if (node == NULL) return 0;
    return height(node->left) - height(node->right);
}
struct AVLNode* rotateRight(struct AVLNode* pivot) {
    struct AVLNode* child = pivot->left;
    struct AVLNode* subtreeB = child->right;
    child->right = pivot;
    pivot->left = subtreeB;
    pivot->GE = computeGE(pivot);
    child->GE = computeGE(child);
    return child;
}
int main() {
    struct AVLNode* rotateRight(struct AVLNode* pivot);
    struct AVLNode* rotateLeft(struct AVLNode* pivot);
    struct AVLNode* rebalance(struct AVLNode* node);

    return 0;
}

```

8.4 Left Rotation (LL case)

```

#include <stdio.h>
#include <stdlib.h>
struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE;
};
int max(int a, int b) { return a > b ? a : b; }
int height(struct AVLNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}
int computeGE(struct AVLNode* node) {
    if (node == NULL) return 0;
    return height(node->left) - height(node->right);
}
struct AVLNode* rotateLeft(struct AVLNode* pivot) {
    struct AVLNode* child = pivot->right;
    struct AVLNode* subtreeB = child->left;
    child->left = pivot;
    pivot->right = subtreeB;
    pivot->GE = computeGE(pivot);
}

```

```

    child->GE = computeGE(child);
    return child;
}
int main() {
    struct AVLNode* rotateRight(struct AVLNode* pivot);
    struct AVLNode* rotateLeft(struct AVLNode* pivot);
    struct AVLNode* rebalance(struct AVLNode* node);

    return 0;
}

```

8.5 Rebalance

```

#include <stdio.h>
#include <stdlib.h>
typedef struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE;
} AVLNode;
int height(AVLNode* root);
int computeGE(AVLNode* node);
AVLNode* rotateRight(AVLNode* pivot);
AVLNode* rotateLeft(AVLNode* pivot);

AVLNode* rebalance(AVLNode* node) {
    node->GE = computeGE(node);
    if (node->GE == 2) {
        if (computeGE(node->left) >= 0)
            return rotateRight(node);          /* RR case */
        else {
            node->left = rotateLeft(node->left);
            return rotateRight(node);          /* LR case */
        }
    }
    if (node->GE == -2) {
        if (computeGE(node->right) <= 0)
            return rotateLeft(node);           /* LL case */
        else {
            node->right = rotateRight(node->right);
            return rotateLeft(node);           /* RL case */
        }
    }
    return node;
}

```

8.6 Insert with Rebalance

```

#include <stdio.h>
#include <stdlib.h>
typedef struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE;
} AVLNode;
int computeGE(AVLNode* node);
AVLNode* rebalance(AVLNode* node);

AVLNode* avl_insert(AVLNode* root, int x) {
    if (root == NULL) {
        AVLNode* n = (AVLNode*)malloc(sizeof(AVLNode));
        n->info = x; n->left = NULL; n->right = NULL; n->GE = 0;
        return n;
    }
}

```

```

}
if (x < root->info)
    root->left = avl_insert(root->left, x);
else if (x > root->info)
    root->right = avl_insert(root->right, x);
return rebalance(root);
}

```

8.7 Delete with Rebalance

```

#include <stdio.h>
#include <stdlib.h>
typedef struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE;
} AVLNode;
int computeGE(AVLNode* node);
AVLNode* rebalance(AVLNode* node);

AVLNode* findMin(AVLNode* root) {
    while (root->left != NULL) root = root->left;
    return root;
}

AVLNode* avl_delete(AVLNode* root, int x) {
    if (root == NULL) return NULL;
    if (x < root->info)
        root->left = avl_delete(root->left, x);
    else if (x > root->info)
        root->right = avl_delete(root->right, x);
    else {
        if (root->left == NULL) {
            AVLNode* t = root->right; free(root); return t;
        }
        if (root->right == NULL) {
            AVLNode* t = root->left; free(root); return t;
        }
        AVLNode* succ = findMin(root->right);
        root->info = succ->info;
        root->right = avl_delete(root->right, succ->info);
    }
    return rebalance(root);
}

```

9. ASCII Diagram

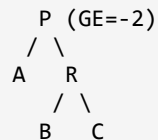
9.1 Single Right Rotation (RR -- left-left case)



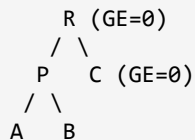
Pivot P: GE = +2 (left-heavy)
 Child L: GE = +1 (left-heavy on the same side)
 Fix: rotate P to the right

9.2 Single Left Rotation (LL -- right-right case)

BEFORE:



AFTER:

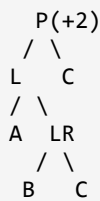


---->

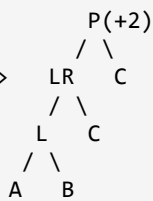
Pivot P: GE = -2 (right-heavy)
Child R: GE = -1 (right-heavy on the same side)
Fix: rotate P to the left

9.3 Double Rotation: Left-Right (LR case)

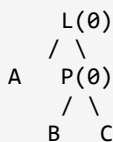
BEFORE:



STEP 1 (left rot L):



STEP 2 (right rot P):



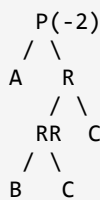
---->

---->

Pivot P: GE=+2, Child L: GE=-1 (opposite side)
Fix: left rotation on L, then right rotation on P

9.4 Double Rotation: Right-Left (RL case)

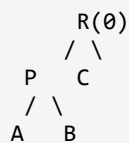
BEFORE:



STEP 1 (right rot R):



STEP 2 (left rot P):



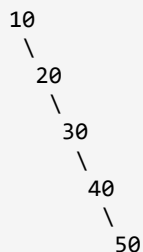
---->

---->

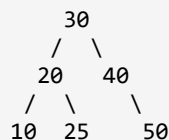
Pivot P: GE=-2, Child R: GE=+1 (opposite side)
Fix: right rotation on R, then left rotation on P

9.5 AVL vs. Degenerate Tree (same keys: 10, 20, 30, 40, 50)

Degenerate BST (sorted input):



AVL tree (same keys):



Height = 5, search = $O(n)$

Height = 3, search = $O(\log n)$

10. When to Use / When to Avoid

Use an AVL tree when:

- You need **guaranteed $O(\log n)$** for search, insert, delete -- plain BSTs can't promise this.
- The input is **sorted or nearly sorted** -- worst case for BSTs but fine for AVL.
- You need **sorting with guaranteed $O(n \log n)$ upper bound** (Q238, Q489).

- Read-heavy workload with occasional modifications -- AVL is more strictly balanced than Red-Black.

Avoid an AVL tree when:

- The data is **static** -- sorted array + binary search is simpler.
- **Insertions/deletions are very frequent** -- AVL rebalances more aggressively than Red-Black.
- You need **O(1) index access** -- use an array.
- Priority queue semantics -- use a heap (Chapter 7).
- Data is small -- overhead of balance factors isn't worth it.

11. Exam Hot Spots

This chapter has **49 MCQ corpus questions** assigned to it. They cluster into the following patterns.

Pattern A: Four rotation types (20+ questions)

Q29, Q36, Q57, Q104, Q116, Q221, Q476, Q974, Q997, Q3975: "Which rotations are used for AVL rebalancing?"

- **Correct:** All four: **Single left, Single right, Left-right, Right-left.**
- **Wrong:** Any answer omitting one of the four. These appear in both English and Romanian. Select ALL four.

Pattern B: RL rotation trigger (10 questions)

Q46, Q90, Q124, Q209, Q460, Q964, Q986, Q3974: "When is a right-left rotation used?"

- **Correct:** "Root GE = +2 and right child GE = -1."
- **Wrong:** "Left imbalance with right child" (that's LR), "Root GE = -2 with left child +1" (that's LL), "GE = 0" (not an imbalance).
- **Trap:** RL = GE +2 (not -2), right child GE -1 (not +1). The naming is counterintuitive.

Pattern C: AVL definition and balance invariant (10 questions)

Q230, Q481: "Best description of an AVL tree?"

- **Correct:** "BST where the height of left and right subtrees of any node differs by at most 1."
- **Wrong:** "Every level completely filled" (complete tree), "depth of left always greater" (not the invariant), "splay operations" (splay tree).

Q3976, Q3979, Q3993: Balance factor values -- "All GE in {-1, 0, +1}" is correct.

Q3978: AVL balance property -- "Height of left subtree differs from right by at most one unit" is correct. Both directions (left-right and right-left) apply.

Q3982: False statement about perfectly balanced trees -- "Leaf nodes on levels 1..h-2 must have the maximum number of children" is the FALSE statement (only height difference matters, not node count at each level).

Q3984: "NOT a property of perfectly balanced trees?" -- "Node count difference is 0" (only height difference matters, not node count).

Pattern D: Rebalancing trigger (4 questions)

Q232, Q480: "What triggers rebalancing?"

- **Correct:** "A node's left and right subtree heights differ by more than 1."
- **Wrong:** "Height exceeds $\log_2(n)$ ", "duplicate values", "in-order not sorted", "leaf depth is 1".

Pattern E: When to check balance (2 questions)

Q3987: Balance must be checked **"after each deletion and insertion"** -- not just insert, not just delete.

Pattern F: AVL applications (4 questions)

Q238, Q489: "Problems solvable with AVL?"

- **Correct:** "Sorting in $O(n \log n)$ with guaranteed upper bound" and "Fast search in dynamic dataset with guarantee."
- **Wrong:** " $O(\log n)$ only if static" (AVL works for dynamic), "Search without guarantee" (AVL's purpose is the guarantee).

Pattern G: AVL characteristics (7 questions)

Q242, Q490: "Characteristics of AVL?"

- **Correct:** "Height difference at most 1" and "Insertion requires rebalancing."
- **Wrong:** "All leaves same level" (not required), "Height always $O(n)$ " (height is $O(\log n)$).

Q251, Q496: "True about AVL?" -- "Height diff at most 1," "Rebalancing needed," " $O(\log n)$ worst-case." Wrong: "AVL is always complete."

Q3981: "Which statement is correct about AVL?" -- **All variants correct:** GE in $\{-1, 0, +1\}$, AVL is balanced BST, rotation can be left or right.

Q3983: "Which statement about balance characteristics is correct?" -- **All variants correct:** GE = height difference, balanced when GE in $\{-1, 0, +1\}$, height = longest root-to-leaf path.

Q3996: For the sequence 23, 8, 28, 10, 5, 3 (root = 23), the FALSE statement is: "Height of right subtree equals height of left subtree" (they are not equal -- the tree is left-heavy).

Pattern H: Balance factor direction (4 questions)

Q3985: "GE = -2 means" -- **Imbalance on the left side** (right subtree is taller).

Q3986: "Right-side imbalance resolved by" -- **Rotation to the left.**

Q3988: "Balance factor can be calculated" -- **From left to right and from right to left** (both directions of height difference).

Q3997: "Balance factor is" -- **Difference between height of right subtree and height of left subtree.**

Pattern I: Insert traversal and node structure (2 questions)

Q3980: "Insertion traversal" -- **Root to insertion position, then inserted node back to root** (downward walk + upward rebalancing).

Q3989: "Element structure contains" -- **Info, left pointer, right pointer** (with GE as the fourth field).

Pattern J: GE value interpretation (2 questions)

Q3991: "GE in $\{-2, 2\}$ " -- **Strongly imbalanced.**

Q3992: "GE = 0" -- **Balanced.**

Pattern K: Rebalancing methods (1 question)

Q3995: "Rebalancing done through" -- **All variants correct** (all four rotation types are valid).

12. Common Traps

Wrong	Why it's wrong
"AVL tree is always complete"	AVL is balanced (height $\leq 1.44 \cdot \log_2(n)$), but NOT necessarily complete.

"All leaves are on the same level"	Only true for perfect binary trees, not AVL.
"Height is always $O(n)$ "	AVL height is $O(\log n)$. $O(n)$ is for degenerate BSTs.
"GE = 0 means imbalance"	GE = 0 means perfectly balanced. Only GE = +2 or -2 is imbalance.
"RL: root GE=-2, left child +1"	Wrong. RL = root GE= +2 , right child GE=-1.
"LR: root GE=-2, right child -1"	Wrong. LR = root GE= +2 , left child GE=-1.
"Check balance only after insert"	Must check after both insert and delete (Q3987).
"Once balanced, stays balanced"	Every insertion/deletion can unbalance the tree.
"Rebalancing from root to leaf"	Rebalancing walks from modified node back to root (bottom-up).
"AVL guarantees $O(1)$ "	AVL guarantees $O(\log n)$, not $O(1)$. Only rotation is $O(1)$.
"Balance factor = node count difference"	Balance factor = height difference, NOT node count.
"Rotation changes in-order traversal"	Rotation preserves BST invariant -- sorted order unchanged.
"Left-right fixes right-right"	LR fixes left-right imbalance. Right-right needs single left rotation.

13. Quick-Recall Card

DEFINITION

AVL tree = a BST where, for every node, the heights of the left and right subtrees differ by at most 1.

Named after: Adelson-Velsky and Landis (1962).

Also called: "self-balanced BST" or "balanced tree."

BALANCE FACTOR (GE)

$GE = H(\text{left subtree}) - H(\text{right subtree})$

Valid AVL: GE in $\{-1, 0, +1\}$ for EVERY node.

Imbalanced: GE = +2 or -2 at any node.

HEIGHT

$H(\text{NULL}) = 0$

$H(\text{node}) = 1 + \max(H(\text{left}), H(\text{right}))$

AVL with n nodes: height $\leq 1.44 * \log_2(n)$

FOUR ROTATIONS

RR (single right): pivot GE=+2, left child GE=+1 (left-left case)

LL (single left): pivot GE=-2, right child GE=-1 (right-right case)

LR (double): pivot GE=+2, left child GE=-1 (left-right case)

Step 1: left rotation on child

Step 2: right rotation on pivot

RL (double): pivot GE=-2, right child GE=+1 (right-left case)

Step 1: right rotation on child

Step 2: left rotation on pivot

ROTATION DECISION TABLE

Pivot GE	Child GE	Rotation
+2	+1	Single right (RR)
+2	-1	Double LR
-2	-1	Single left (LL)
-2	+1	Double RL

INSERT REBALANCE

1. BST insert (root to leaf)
2. Walk back up to root
3. At each ancestor: recompute GE
4. If GE in $\{-2, +2\}$: apply rotation
5. At most 1 node becomes imbalanced, at most 2 rotations fix it

DELETE REBALANCE

1. BST delete (3 cases)
2. Walk back up to root
3. At each ancestor: recompute GE
4. If GE in $\{-2, +2\}$: apply rotation
5. Multiple ancestors may need rebalancing (more complex than insert)

COMPLEXITY (WORST CASE -- GUARANTEED)

Search: $O(\log n)$
Insert: $O(\log n)$
Delete: $O(\log n)$
Rotation: $O(1)$

C STRUCT

```
struct AVLNode {
    int info;
    struct AVLNode* left;
    struct AVLNode* right;
    int GE; /* balance factor */
};
```

EXAM GOLD

- Balance invariant applies to EVERY node, not just root.
- Check balance after BOTH insert and delete.
- RL: pivot $GE=+2$, right child $GE=-1$ (not the other way around).
- Height of NULL subtree is 0.
- AVL guarantees $O(\log n)$ worst-case; plain BST only $O(\log n)$ average.
- AVL is NOT the same as a complete binary tree.
- Rotation preserves in-order traversal (sorted order).
- Balance factor = height difference, NOT node count difference.

WHEN TO USE

Guaranteed $O(\log n)$ needed. Sorted input expected. Dynamic dataset.
Sorting with guaranteed $O(n \log n)$ upper bound.

WHEN TO AVOID

Static data (use sorted array + binary search).
Write-heavy workload (use Red-Black tree).
Need $O(1)$ index access (use array).
Priority queue semantics (use heap, Ch 7).

\newpage

Chapter 11 — B-Trees

Concept chapter. B-trees generalize binary search trees to multi-way branching, trading depth for width. Each node holds multiple keys and children, keeping all leaves at the same height. This makes B-trees the structure of choice for disk-based storage (databases, file systems) where minimizing disk reads matters more than minimizing comparisons. The chapter has **0 MCQ corpus questions assigned to it directly** — B-tree concepts appear in theory questions and cross-referencing with AVL / BST (Ch 9/10). The patterns here are what make those conceptual questions solvable.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define a B-tree of order m and state the exact constraints on the number of keys and children per node.
- Explain why all leaves in a B-tree must be at the same height and why this guarantees balance.
- Describe the search, insert, and delete algorithms at a conceptual level, including when splitting, merging, or redistributing occurs.
- Compare a B-tree to a B+ tree and explain the practical advantage of the B+ tree variant for database indexing.
- State the height bound of a B-tree and explain why $O(\log_m n)$ disk reads are sufficient for any operation.

- Contrast B-trees with binary search trees and explain why high branching factor reduces disk I/O.
- Trace through a B-tree construction from a given sequence of insertions, showing each split.

2. Mental Model & Analogy

Think of a B-tree as a **multi-level filing cabinet**. A regular binary search tree is like a filing cabinet where each drawer holds one folder and has exactly two sub-drawers — you keep digging deeper, one drawer at a time. A B-tree of order 5 is like a filing cabinet where each drawer can hold up to 4 folders (keys) and has up to 5 sub-drawers (children). You open a drawer, scan the folders inside to decide which sub-drawer to open next, and keep going until you find what you need.

Why this matters for databases: Opening a drawer costs time (a disk read). A binary tree with 1 million items needs about 20 drawers opened ($\log_2 1,000,000 \sim 20$). A B-tree of order 1000 needs only about 3 ($\log_{1000} 1,000,000 \sim 2$). Fewer drawer opens = faster retrieval.

The B+ tree variant is like the **index at the back of a textbook**. The textbook itself (internal nodes) tells you which chapter to look in. The index (leaf nodes) is where you actually find the page numbers (data). All the real information is in the index; the internal nodes are just signposts.

B+ tree analogy reinforced: In a B+ tree, the internal nodes hold *copies* of keys for routing purposes only. The actual data records live exclusively in the leaves. The leaves are linked together in a sorted list, so range queries ("find all records between A and M") can scan the leaf chain without climbing back up the tree.

3. Memory Layout

In practice, each B-tree node is stored on a **disk page** (typically 4 KB or 8 KB). This is the fundamental reason B-trees exist: one disk read retrieves an entire node, including all its keys and child pointers. The branching factor (how many children fit in one page) determines the order m .

For exam purposes, conceptualize each node as a C struct containing:

```
+-----+
| n | key[0] | key[1] | ... | key[n-1] |
|   | child[0] | child[1] | ... | child[n] |
+-----+

n = number of keys currently in the node
keys are sorted: key[0] < key[1] < ... < key[n-1]
child[i] = pointer to subtree with keys < key[i]
child[n] = pointer to subtree with keys > key[n-1]
```

Key layout facts:

- A node with n keys has exactly $n+1$ children (pointers to subtrees).
- Keys within a node are stored in sorted order for binary search.
- Child pointers are interleaved with keys: $child[0]$ is to the left of $key[0]$, $child[1]$ is between $key[0]$ and $key[1]$, and so on.
- In a B+ tree, internal nodes contain only keys and child pointers (no data). Leaf nodes contain keys and actual data records, and are linked together.

4. Definition & Properties

B-tree of order m (sometimes called "minimum degree t " where $m = 2t$):

A B-tree of order m is a balanced search tree with the following properties:

Property	Constraint
Every node has at most	m children
Every non-root internal node has at least	$\lceil m/2 \rceil$ children

The root has at least	2 children (if not a leaf)
Every node has at most	$m - 1$ keys
Every non-root internal node has at least	$\text{ceil}(m/2) - 1$ keys
All leaves appear at the same level	(perfectly balanced)
A node with n keys has	$n + 1$ children

Height bound:

For a B-tree of order m containing n keys:

$$h \leq \log_{\text{ceil}(m/2)} ((n+1) / 2)$$

In simpler terms: $h = O(\log_m n)$. With $m = 1000$ and $n = 1,000,000$, the height is at most 3. This is why B-trees are so efficient for disk storage — very few disk reads are needed.

B+ tree variant:

A B+ tree of order m differs from a standard B-tree in two ways:

1. **All data resides in the leaves.** Internal nodes hold only copies of keys for routing. A key that appears in an internal node also appears in exactly one leaf.
2. **Leaves are linked.** Each leaf contains a pointer to the next leaf in sorted order, forming a sorted linked list at the leaf level.

The practical consequence: internal nodes are smaller (no data payloads), so more keys fit per disk page, further reducing tree height. Range queries become sequential leaf scans instead of repeated tree traversals.

5. Operations — Step-by-Step Mechanics

5.1 Search

Search in a B-tree follows the same logic as search in a BST, but at each node you scan through all keys to find the right subtree:

Step 1. Start at the root.

Step 2. At the current node, scan keys left to right (or use binary search within the node). If the target key is found, return it.

Step 3. If not found, determine which child pointer to follow: if $\text{target} < \text{key}[0]$, follow $\text{child}[0]$; if $\text{key}[i] < \text{target} < \text{key}[i+1]$, follow $\text{child}[i+1]$; if $\text{target} > \text{key}[n-1]$, follow $\text{child}[n]$.

Step 4. Repeat from Step 2 at the child node. If you reach a leaf without finding the key, the key does not exist.

Cost: At most h disk reads (one per level), where h is the tree height.

5.2 Insertion

Insertion always happens at a leaf node. The critical rule: **if the leaf is full, split it first (bottom-up).**

Step 1. Search for the correct leaf to insert into.

Step 2. If the leaf has fewer than $m-1$ keys, insert the new key in sorted order. Done.

Step 3. If the leaf has $m-1$ keys (it is full), split it:

- The leaf contains keys: $k_1, k_2, \dots, k_{m-1}$, and the new key k_{new} .

- Sort all m keys together.
- Create a new right sibling node.
- The original node keeps the first $\text{ceil}((m-1)/2)$ keys.
- The new node gets the remaining keys.
- The middle key (at position $\text{ceil}((m-1)/2)$) is **promoted** to the parent.
- If the parent is also full, split it too. This cascade can propagate up to the root.

Step 4. If the root itself splits, a new root is created with the promoted middle key. This is the only way the tree grows taller.

Split rule for m -ary B-trees: When a node with $m-1$ keys splits, the middle key at position $\text{ceil}((m-1)/2)$ goes up to the parent. The left node keeps keys before it, the right node keeps keys after it.

5.3 Deletion

Deletion is more complex because removing a key from an internal node may require borrowing from siblings or merging nodes.

Case 1: Deleting from a leaf that has more than the minimum number of keys. Simply remove the key. No structural changes needed.

Case 2: Deleting from a leaf that has the minimum number of keys. Try to **redistribute** (borrow a key from a sibling) or **merge** the leaf with a sibling.

- **Redistribute:** If a left or right sibling has more than the minimum number of keys, rotate a key from the sibling through the parent into this node.
- **Merge:** If both siblings have the minimum, merge the leaf with one sibling and the separating key from the parent. This reduces the parent's key count, which may trigger Case 2 recursively on the parent.

Case 3: Deleting a key from an internal node. Replace the key with its **in-order predecessor** (the largest key in the left subtree) or **in-order successor** (the smallest key in the right subtree), then delete that predecessor/successor from the appropriate leaf. This reduces the problem to Case 1 or Case 2.

Case 4: The root becomes empty after deletion. If the root loses its last key, the tree height decreases by 1. The sole child becomes the new root.

Redistribute vs. Merge decision: Redistribution is preferred because it avoids changing the tree structure. Merge is used only when no sibling has excess keys.

6. Trace-Through Example

Build a B-tree of order 5 (max 4 keys per node, max 5 children) from the following 10 keys inserted in order:

Insert: 10, 20, 30, 40, 50, 60, 70, 80, 90, 100

Step 1: Insert 10, 20, 30, 40 — all go into the root. No split needed (4 keys < 5 max).

```
[10, 20, 30, 40]
```

Step 2: Insert 50 — root is full (4 keys). Split.

Sort all 5 keys: 10, 20, 30, 40, 50. Middle key = 30 (position 2 of 5, 0-indexed). Promote 30 to new root.

```

      [30]
     /  \
  [10, 20] [40, 50]

```

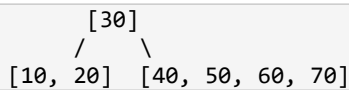
Step 3: Insert 60 — goes into right leaf [40, 50]. No split (3 keys < 5 max).

```

      [30]
     /  \
  [10, 20] [40, 50, 60]

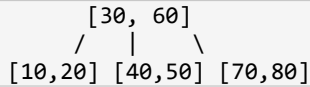
```

Step 4: Insert 70 — goes into right leaf [40, 50, 60]. No split (4 keys = max).

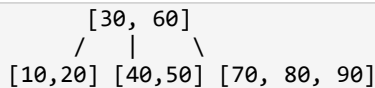


Step 5: Insert 80 — right leaf is full (4 keys). Split.

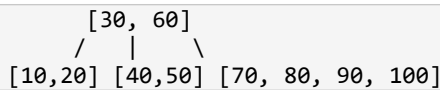
Sort all 5 keys in right leaf: 40, 50, 60, 70, 80. Middle key = 60. Promote 60 to parent.



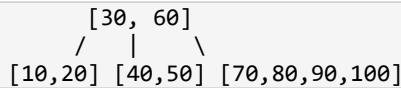
Step 6: Insert 90 — goes into rightmost leaf [70, 80]. No split.



Step 7: Insert 100 — rightmost leaf [70, 80, 90] becomes [70, 80, 90, 100]. No split (4 keys = max).



Final B-tree of order 5 after inserting all 10 keys:



Observations:

- Height = 1 (root + leaves). A BST with these 10 keys in sorted order would have height 9.
- All leaves are at the same depth (property satisfied).
- Root has 3 children, 2 keys (minimum for non-root internal: $\text{ceil}(5/2)-1 = 2$ keys, satisfied).
- Left leaves have 2 keys (minimum for non-root leaf: $\text{ceil}(5/2)-1 = 2$, satisfied).
- Rightmost leaf has 4 keys (maximum $m-1 = 4$, satisfied).

7. Operations + Big-O Table

Operation	Complexity	Notes
Search	$O(\log_m n)$	At most h disk reads, one per level
Insert	$O(\log_m n)$	May split nodes up the path to root
Delete	$O(\log_m n)$	May merge/redistribute nodes up the path
Traversal (in-order)	$O(n)$	Visit all leaves; B+ tree leaves are linked
Space	$O(n)$	Each key stored once (B-tree) or twice (B+ tree, as copy in internal)
Height	$O(\log_m n)$	With $m=1000$, height ~ 3 for 1 billion keys

Comparison with BST/AVL:

Structure	Search	Insert	Delete	Height
BST (unbalanced)	$O(n)$ worst	$O(n)$ worst	$O(n)$ worst	$O(n)$ worst
AVL tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\sim 1.44 \log n$
B-tree (order m)	$O(\log_m n)$	$O(\log_m n)$	$O(\log_m n)$	$\log_m n$

The B-tree wins on disk I/O because $\log_m n \ll \log_2 n$ when $m \gg 2$.

8. C Code Reference

8.1 B-tree node structure

```
#include <stdio.h>
#include <stdlib.h>

#define M 5 /* order of the B-tree */

typedef struct BTreeNode {
    int keys[M - 1]; /* up to m-1 keys */
    struct BTreeNode* children[M]; /* up to m children */
    int n; /* current number of keys */
    int leaf; /* 1 if leaf, 0 otherwise */
} BTreeNode;

BTreeNode* createNode(int leaf) {
    BTreeNode* node = (BTreeNode*)malloc(sizeof(BTreeNode));
    node->n = 0;
    node->leaf = leaf;
    int i;
    for (i = 0; i < M; i++)
        node->children[i] = NULL;
    return node;
}
```

Key points: `keys` array holds up to $m-1$ sorted integers. `children` array holds up to m child pointers. `n` tracks how many keys are currently stored. `leaf` flag distinguishes leaf nodes from internal nodes. In a production system, keys would be generic (or pointers to data), but `int` suffices for exam examples.

8.2 B-tree search

```
#include <stdio.h>
#define M 5
typedef struct BTreeNode {
    int keys[M - 1];
    struct BTreeNode* children[M];
    int n;
    int leaf;
} BTreeNode;

BTreeNode* search(BTreeNode* root, int key) {
    if (root == NULL) return NULL;

    int i = 0;
    while (i < root->n && key > root->keys[i])
        i++;

    if (i < root->n && key == root->keys[i])
        return root;

    if (root->leaf)
        return NULL;

    return search(root->children[i], key);
}
```

Logic: Scan keys left to right until you find the target or pass it. If found, return the node. If it is a leaf and not found, return NULL. Otherwise, recurse into the appropriate child. Iterative version would use a loop instead of recursion to save stack space on deep trees (though B-tree depth is small in practice).

8.3 B-tree insert concept (split logic)

```
#include <stdio.h>
#include <stdlib.h>
#define M 5
typedef struct BTreeNode {
    int keys[M - 1];
    struct BTreeNode* children[M];
    int n;
    int leaf;
} BTreeNode;

BTreeNode* createNode(int leaf) {
    BTreeNode* node = (BTreeNode*)malloc(sizeof(BTreeNode));
    node->n = 0;
    node->leaf = leaf;
    int i;
    for (i = 0; i < M; i++)
        node->children[i] = NULL;
    return node;
}

/* When a node is full during insert, split the child at index i of parent */
void splitChild(BTreeNode* parent, int i) {
    BTreeNode* full = parent->children[i];
    int mid = (M - 1) / 2; /* index of middle key */

    BTreeNode* right = createNode(full->leaf);
    right->n = full->n - mid - 1;

    int j;
    /* copy keys [mid+1 .. n-1] to right node */
    for (j = 0; j < right->n; j++)
        right->keys[j] = full->keys[mid + 1 + j];

    /* copy children [mid+1 .. n] to right node (if not leaf) */
    if (!full->leaf) {
        for (j = 0; j <= right->n; j++)
            right->children[j] = full->children[mid + 1 + j];
    }

    /* shift parent's children and keys to make room */
    for (j = parent->n; j > i; j--)
        parent->children[j + 1] = parent->children[j];
    parent->children[i + 1] = right;

    for (j = parent->n - 1; j >= i; j--)
        parent->keys[j + 1] = parent->keys[j];
    parent->keys[i] = full->keys[mid];

    full->n = mid;
    parent->n++;
}
```

Split rule: The middle key at position `mid` moves up to the parent. Keys after `mid` go to the new right sibling. Children after `mid` go to the new right sibling. The parent absorbs the promoted key and gains one new child pointer.

8.4 B-tree in-order traversal

```
#define M 5
typedef struct BTreeNode {
    int keys[M - 1];
    struct BTreeNode* children[M];
    int n;
    int leaf;
}
```

```

} BTreeNode;
#include <stdio.h>
void traverse(BTreeNode* root) {
    if (root == NULL) return;

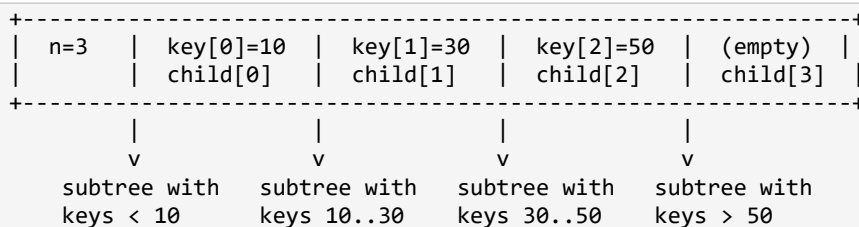
    int i;
    for (i = 0; i < root->n; i++) {
        if (!root->leaf)
            traverse(root->children[i]);
        printf("%d ", root->keys[i]);
    }
    if (!root->leaf)
        traverse(root->children[root->n]);
}
int main() {
    return 0;
}

```

Logic: For each key, traverse the left child first (if not a leaf), print the key, then continue. After the last key, traverse the rightmost child. This produces keys in sorted order, the same as in-order traversal of a BST. In a B+ tree, you would simply follow the leaf pointers from left to right instead of recursing.

9. ASCII Diagram

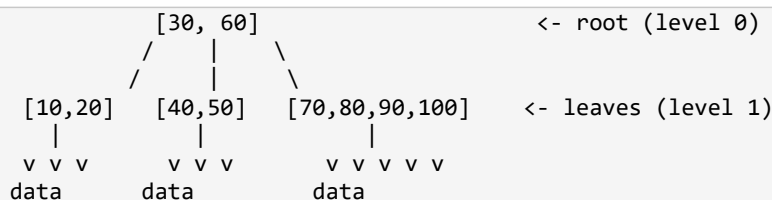
9.1 B-tree node layout (order 5, m-1=4 keys max)



Key facts:

- n = number of keys currently stored ($0 \leq n \leq m-1$).
- Keys are sorted left to right.
- There are exactly $n+1$ child pointers.
- $child[i]$ points to the subtree containing keys between $key[i-1]$ and $key[i]$.

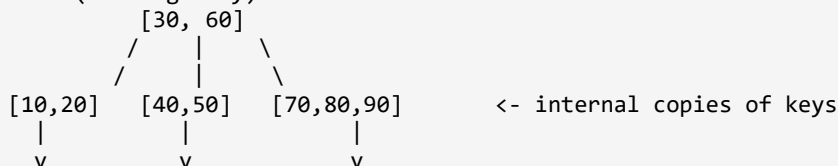
9.2 B-tree with 3 levels (order 5)



All leaves at the same height (level 1). Height = 2. In a binary tree with the same data, the height would be up to 10.

9.3 B+ tree structure (order 5)

INTERNAL NODES (routing only):



LEAF NODES (contain actual data, linked):

```
+-----+-----+-----+-----+ +-----+-----+-----+-----+ +-----+-----+-----+-----+
| 10,d | 20,d | NULL | NEXT---->| 40,d | 50,d | NULL | NEXT---->| 70,d | 80,d | 90,d |100,d|
+-----+-----+-----+-----+ +-----+-----+-----+-----+ +-----+-----+-----+-----+
leaf 1                                leaf 2                                leaf 3
```

In a B+ tree:

- Internal nodes hold only keys for routing (no data).
- All data records (d) live in the leaf nodes.
- Leaf nodes are linked in sorted order (NEXT pointers).
- A key that appears in an internal node is a copy; the "real" entry is in the leaf.
- Range queries scan the leaf chain sequentially without climbing back up.

9.4 Node split during insertion (order 5)

Before split (leaf is full with 4 keys + 1 new):

```
[10, 20, 30, 40] + insert 25
```

After split (middle key promoted to parent):

```
Parent:  [..., 30, ...]
         /      \
       [10, 20] [25, 40]
```

The key 30 (middle of sorted 10, 20, 25, 30, 40) moves up. Left node keeps [10, 20]. Right node gets [25, 40] (or [30, 40] depending on the split convention — the exact middle key goes up).

10. When to Use / When to Avoid

Use B-trees (or B+ trees) when:

- Data is stored on disk and each disk read is expensive (database indexes, file system directories).
- The dataset is very large (millions or billions of records) and must fit in a structure with predictable $O(\log n)$ access.
- Range queries are frequent (B+ tree leaves form a sorted linked list for sequential scanning).
- The branching factor should be tuned to match the disk page size (e.g., 4 KB page with 4-byte keys = order ~ 1000).
- You need guaranteed worst-case performance (B-trees are always balanced by construction).

Avoid B-trees when:

- Data fits entirely in memory and m is small (an AVL or Red-Black tree may be simpler and faster due to better cache locality for small structures).
- The key space is not naturally ordered (a hash table would be faster for point lookups).
- You need $O(1)$ average-case lookup (B-trees give $O(\log n)$, which is slower than hash tables for exact-match queries).
- Insertions and deletions are extremely frequent and the overhead of maintaining B-tree invariants is not justified.

Exam context: The course lists B-trees alongside AVL and Red-Black trees as types of balanced trees (see part3_nonlinear.txt, line 1310). The key distinction is that B-trees are optimized for external storage (disk), while AVL and Red-Black trees are optimized for in-memory operation.

11. Exam Hot Spots

This chapter has **0 MCQ corpus questions assigned to it directly**. B-tree concepts are tested in theory questions that cross-reference BST/AVL material from Ch 9/10. The patterns below are the concept patterns the exam uses.

Pattern A: B-tree vs BST properties

Q: "What is the maximum height of a B-tree of order 5 with 31 keys?"

- **Correct reasoning:** A B-tree of order 5 has at most 4 keys per node and at least 2 keys per non-root internal node. Height is $O(\log_5 n)$. With 31 keys and branching factor ~ 2.5 (minimum), height is at most 3. With maximum branching (5), height is at most 2.
- **Wrong:** "31" (that would be a degenerate BST), " $\log_2 31 = 5$ " (that's a binary tree, not a B-tree).

Pattern B: B+ tree differences

Q: "What is the main advantage of a B+ tree over a B-tree for database indexing?"

- **Correct:** "All data is in the leaves, so internal nodes are smaller and more keys fit per page, reducing tree height."
- **Wrong:** "B+ trees are faster for point lookups" (roughly the same), "B+ trees use less total memory" (they actually use slightly more due to key duplication).

Pattern C: Node splitting mechanics

Q: "In a B-tree of order 5, a leaf node with 4 keys receives a 5th key. How many keys go to the new right sibling?"

- **Correct:** 2 keys. The 5 sorted keys are split: parent gets 1 (middle), left keeps 2, right gets 2.
- **Wrong:** "All 4 original keys stay, new key goes to a new node" (incorrect — the split distributes keys evenly).

Pattern D: Why B-trees reduce disk I/O

Q: "A database with 1 billion records uses a B-tree of order 1000. What is the maximum number of disk reads to find any record?"

- **Correct:** 3 reads ($\log_{1000} 1,000,000,000 \sim 3$).
- **Wrong:** "1 billion" (that would be a linear scan), "30" (that's a binary tree).

12. Common Traps

The exam has a consistent set of wrong-answer patterns. Memorize why each is wrong so you don't pick them.

Wrong	Why it's wrong
"A B-tree is a binary tree with more children"	A B-tree is NOT a binary tree. Binary trees have at most 2 children; B-trees of order m have up to m children. The structures are fundamentally different.
"B-tree nodes can have different heights"	ALL leaves in a B-tree are at the same height. This is the defining property that guarantees balance. If leaves are at different heights, it is not a B-tree.
"Node splitting happens top-down during insertion"	Splitting happens bottom-up. You insert into a leaf, and if it overflows, you split it and possibly propagate the split upward. You do not pre-split on the way down.
"B+ tree internal nodes contain actual data records"	Internal nodes in a B+ tree contain only keys for routing. All data records live exclusively in the leaf nodes.
"m-ary means each node has exactly m children"	m -ary means each node has AT MOST m children. A node can have fewer children (but non-root nodes must have at least $\lceil m/2 \rceil$ children).
"B-trees are slower than AVL trees for in-memory search"	For in-memory operation with small datasets, B-trees may be slower due to more comparisons per node. But for disk-based operation, B-trees are dramatically faster due to fewer disk reads. Context matters.
"A B-tree of order 3 is the same as an AVL tree"	Not true. A B-tree of order 3 (2-3 tree) is a different structure from an AVL tree, though both are balanced. AVL trees are strictly binary; B-

	trees of order 3 can have nodes with 2 children.
"In-order traversal of a B+ tree requires climbing back to the root"	In a B+ tree, leaf nodes are linked. In-order traversal follows the leaf pointers sequentially, never climbing back up. This is a key advantage for range queries.
"Deletion in a B-tree always removes from a leaf"	Deletion from an internal node is handled by replacing the key with its in-order predecessor or successor (from a leaf), then deleting that leaf entry. The leaf deletion triggers any needed merges/redistributes.
"The root of a B-tree can have only 1 child"	The root must have at least 2 children if it is not a leaf. A root with 1 child means the tree should have been restructured (the child should have become the new root).

13. Quick-Recall Card

B-TREE OF ORDER m

Every node: at most m children, at most $m-1$ keys.
Non-root node: at least $\lceil m/2 \rceil$ children, at least $\lceil m/2 \rceil - 1$ keys.
Root (if not leaf): at least 2 children.
All leaves: same height (perfectly balanced).
Node with n keys: exactly $n+1$ children.

HEIGHT BOUND

$h \leq \log_{\lceil m/2 \rceil} ((n+1)/2) = O(\log_m n)$.
Example: $m=1000$, $n=10^9 \Rightarrow h \leq 3$.

INSERTION

1. Find the correct leaf.
2. If leaf has room ($< m-1$ keys), insert in sorted order.
3. If leaf is full, split: sort m keys, promote middle to parent.
4. If parent overflows, split it too (cascade to root).
5. If root splits, new root is created (tree grows taller).

DELETION

- Case 1: Leaf has $>$ min keys $\Rightarrow$ just remove.
Case 2: Leaf has min keys $\Rightarrow$ redistribute from sibling or merge.
Case 3: Internal node $\Rightarrow$ replace with predecessor/successor, delete from leaf.
Case 4: Root empties $\Rightarrow$ root's child becomes new root (tree shrinks).

B+ TREE DIFFERENCES

- All data in leaves only (internal nodes = routing keys).
- Leaves are linked (sorted linked list).
- Range queries scan leaf chain (no tree climbing).
- Internal nodes are smaller $\Rightarrow$ more keys per page $\Rightarrow$ shorter tree.

B-TREE vs BST vs AVL

BST: $O(n)$ worst case, height up to n .
AVL: $O(\log n)$ always, height $\sim 1.44 \log n$.
B-tree: $O(\log_m n)$ always, much shorter height on disk.

WHEN TO USE

Disk-based storage (databases, file systems).
Large datasets with frequent range queries.
When disk I/O is the bottleneck.

EXAM GOLD

- B-tree is NOT a binary tree.
- All leaves at the same height (mandatory).
- Split is bottom-up, not top-down.
- B+ tree: data only in leaves, leaves are linked.
- m -ary = AT MOST m children, not exactly m .
- Height $O(\log_m n)$ with $m \gg 2$ is much less than $O(\log_2 n)$.

\newpage

Chapter 12 — Graphs

Foundational chapter. Graphs are the most general non-linear data structure in this course. A tree is a special case of a graph (connected, acyclic, directed). Understanding graph representations, traversals, and terminology is essential for answering questions on trees, topological sort, shortest path, and connectivity. Graph concepts also appear indirectly in questions about BSTs, AVL trees, and linked structures.

1. Learning Objectives

After reading this chapter, you will be able to:

- Define a graph $G = (V, E)$ and distinguish directed vs. undirected, weighted vs. unweighted, connected vs. disconnected, dense vs. sparse, and complete graphs.
- Represent a graph using an adjacency matrix ($O(V^2)$ space) and an adjacency list ($O(V + E)$ space), and explain the tradeoffs.
- Execute BFS (breadth-first search) using a queue and DFS (depth-first search) using a stack or recursion, and predict the discovery order.
- Compute degree, in-degree, and out-degree for vertices in directed and undirected graphs.
- Identify applications of graphs: shortest path, connectivity, topological sort on DAGs, cycle detection, connected components, and minimum spanning tree.
- Recognize that trees are special graphs and that tree traversal questions often test graph concepts indirectly.

2. Mental Model & Analogy

Think of a **social network** as a graph. Each person is a **vertex** (node). A friendship between two people is an **edge** (link). If the friendship is mutual (both follow each other), the edge is **undirected**. If it is a one-way follow (you follow a celebrity but they do not follow you back), the edge is **directed**.

Think of **roads between cities** as a **weighted undirected graph**. Each city is a vertex. Each road is an edge, and the road's length or travel time is the **weight**. Finding the fastest route from city A to city B is the **shortest path** problem.

Key distinctions to internalize:

1. **Directed vs. undirected:** In an undirected graph, edge (u, v) is the same as (v, u) . In a directed graph (digraph), edge (u, v) goes from u to v only — u is the source, v is the destination.
2. **Weighted vs. unweighted:** In an unweighted graph, every edge is equal. In a weighted graph, each edge carries a numeric value (distance, cost, capacity).
3. **Connected vs. disconnected:** A connected graph has a path between every pair of vertices. A disconnected graph has at least one isolated vertex or component.
4. **A tree is a graph:** Specifically, a tree is a connected, acyclic, directed graph with a single root. Every tree is a graph, but not every graph is a tree.

3. Memory Layout

There are two primary representations of a graph in memory.

Adjacency Matrix — a 2D array:

For a graph with V vertices, allocate a $V \times V$ matrix.

```
A[i][j] = 1 (or weight) if edge (i, j) exists
A[i][j] = 0             if no edge
```

```
Undirected graph => matrix is symmetric: A[i][j] == A[j][i]
Directed graph   => matrix may not be symmetric
Weighted graph   => A[i][j] stores the weight instead of 1
```

Adjacency List — array of linked lists:

Allocate an array of V linked lists.

Each list at index i contains all vertices j such that edge (i, j) exists.

```
adj[0] -> [1] -> [3] -> NULL
adj[1] -> [0] -> [2] -> [4] -> NULL
adj[2] -> [1] -> NULL
adj[3] -> [0] -> [4] -> NULL
adj[4] -> [1] -> [3] -> NULL
```

Space: $O(V)$ for the array + $O(E)$ for all linked list nodes = $O(V + E)$

Space comparison:

Representation	Space	Best for
Adjacency matrix	$O(V^2)$	Dense graphs (E close to V^2)
Adjacency list	$O(V + E)$	Sparse graphs (E close to V)

4. Definition & Properties

Graph — a non-linear data structure consisting of a finite set of **vertices** (also called nodes) and a finite set of **edges** (links between pairs of vertices). Formally, $G = (V, E)$ where V is the set of vertices and E is the set of edges.

Key properties:

- **Directed graph (digraph):** Edge (u, v) has direction: u is source, v is destination. (v, u) is a different edge.
- **Undirected graph:** Edge (u, v) is bidirectional. (u, v) and (v, u) are the same edge.
- **Weighted graph:** Each edge has a numeric weight (distance, cost, capacity).
- **Unweighted graph:** All edges are equal (weight 1 or no weight).
- **Connected graph:** A path exists between every pair of vertices.
- **Disconnected graph:** At least one vertex is unreachable from another.
- **Dense graph:** E is close to V^2 . Adjacency matrix is efficient.
- **Sparse graph:** E is much less than V^2 . Adjacency list saves space.
- **Complete graph:** Every pair of vertices is connected. Undirected: $V(V-1)/2$ edges.
- **Loop:** An edge connecting a vertex to itself: (v, v) .
- **Path:** A sequence $p_0, p_1, \dots, p_m$ where each consecutive pair is connected by an edge.
- **Cycle:** A path that starts and ends at the same vertex.
- **Acyclic:** No cycles. A directed acyclic graph (DAG) supports topological sort.
- **Degree:** In undirected graphs, number of incident edges. In directed graphs: **in-degree** (incoming) + **out-degree** (outgoing).
- **Bipartite graph:** Vertices split into two sets; every edge crosses between sets (none within).

Handshaking lemma: The sum of all degrees in an undirected graph equals $2|E|$ (each edge contributes 1 to the degree of each of its two endpoints).

5. Operations — Step-by-Step Mechanics

5.1 Adjacency Matrix — Insert Edge

```
/* Add edge (u, v) to undirected graph */
void addEdgeMatrix(int** adj, int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1;    /* symmetric for undirected */
}
```

Step 1. Set $\text{adj}[u][v] = 1$ to record the edge from u to v .

Step 2. For undirected graphs, also set $\text{adj}[v][u] = 1$ to record the symmetric edge.

Step 3. For weighted graphs, replace 1 with the weight value.

5.2 Adjacency Matrix — Edge Lookup

```
/* Check if edge (u, v) exists */
int hasEdgeMatrix(int** adj, int u, int v) {
    return adj[u][v] != 0;
}
```

Step 1. Read $\text{adj}[u][v]$. If non-zero, the edge exists. $O(1)$ time.

5.3 Adjacency List — Insert Edge

```
struct Graph {
    int V;
    struct Node** adj;
};
struct Node {
    int vertex;
    struct Node* next;
};
#include <stdlib.h>
void addEdgeList(struct Node** adj, int u, int v) {
    /* Add v to u's list */
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = adj[u];
    adj[u] = newNode;

    /* Add u to v's list (undirected) */
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = u;
    newNode->next = adj[v];
    adj[v] = newNode;
}
int main() {
    /* Add edge (u, v) to undirected graph using adjacency list */

    return 0;
}
```

Step 1. Allocate a new node with vertex id v .

Step 2. Prepend it to the linked list at $\text{adj}[u]$. This is $O(1)$.

Step 3. For undirected graphs, also prepend a node with vertex id u to $\text{adj}[v]$.

5.4 Adjacency List — Edge Lookup

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int vertex;
    struct Node* next;
};
/* Check if edge (u, v) exists in adjacency list */
int hasEdgeList(struct Node** adj, int u, int v) {
    struct Node* curr = adj[u];
    while (curr != NULL) {
        if (curr->vertex == v)
            return 1;
    }
}
```

```

        curr = curr->next;
    }
    return 0;
}

```

Step 1. Walk the linked list at `adj[u]`, looking for vertex `v`.

Step 2. If found, return 1 (edge exists). If the list ends, return 0. Time: $O(\text{degree}(u))$.

5.5 BFS — Breadth-First Search

```

struct Graph {
    int V;
    struct Node** adj;
};
struct Node {
    int vertex;
    struct Node* next;
};
#include <stdlib.h>
#include <stdio.h>
void bfs(struct Node** adj, int start, int V) {
    int* visited = (int*)calloc(V, sizeof(int));
    int* queue = (int*)malloc(V * sizeof(int));
    int front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);

        struct Node* curr = adj[v];
        while (curr != NULL) {
            if (!visited[curr->vertex]) {
                visited[curr->vertex] = 1;
                queue[rear++] = curr->vertex;
            }
            curr = curr->next;
        }
    }
    free(visited);
    free(queue);
}
int main() {
    return 0;
}

```

Step 1. Mark the start vertex as visited and enqueue it.

Step 2. While the queue is not empty: dequeue the front vertex `v`, print/process it.

Step 3. For each unvisited neighbor `u` of `v`: mark `u` as visited and enqueue `u`.

Step 4. BFS explores vertices **level by level** — all vertices at distance 1 from the start are visited before any at distance 2, and so on.

5.6 DFS — Depth-First Search (Recursive)

```

struct Graph {
    int V;
    struct Node** adj;
};

```

```

struct Node {
    int vertex;
    struct Node* next;
};
#include <stdlib.h>
#include <stdio.h>
void dfsVisit(struct Node** adj, int v, int* visited) {
    visited[v] = 1;
    printf("%d ", v);

    struct Node* curr = adj[v];
    while (curr != NULL) {
        if (!visited[curr->vertex])
            dfsVisit(adj, curr->vertex, visited);
        curr = curr->next;
    }
}
void dfs(struct Node** adj, int start, int V) {
    int* visited = (int*)calloc(V, sizeof(int));
    dfsVisit(adj, start, visited);
    free(visited);
}
int main() {

    return 0;
}

```

Step 1. Mark the current vertex v as visited, print/process it.

Step 2. For each unvisited neighbor u of v : recursively call `dfsVisit(adj, u, visited)`.

Step 3. DFS goes as **deep as possible** along each branch before backtracking. It uses the call stack (recursion) implicitly, or an explicit stack data structure.

5.7 Topological Sort (DAG)

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int vertex;
    struct Node* next;
};
void topologicalSortUtil(struct Node** adj, int v, int* visited, int* stack, int* top) {
    visited[v] = 1;
    struct Node* curr = adj[v];
    while (curr != NULL) {
        if (!visited[curr->vertex])
            topologicalSortUtil(adj, curr->vertex, visited, stack, top);
        curr = curr->next;
    }
    stack[( *top )++] = v;
}

void topologicalSort(struct Node** adj, int V) {
    int* visited = (int*)calloc(V, sizeof(int));
    int* stack = (int*)malloc(V * sizeof(int));
    int top = 0;

    for (int i = 0; i < V; i++) {
        if (!visited[i])
            topologicalSortUtil(adj, i, visited, stack, &top);
    }

    for (int i = top - 1; i >= 0; i--)
        printf("%d ", stack[i]);
}

```

```

    free(visited);
    free(stack);
}

```

Step 1. Perform DFS on the entire graph (some vertices may be unreachable from any single start).

Step 2. After all descendants of a vertex are processed, push that vertex onto a stack.

Step 3. Pop all vertices from the stack — this gives a topological ordering where for every directed edge (u, v), u appears before v.

Important: Topological sort only works on **DAGs** (directed acyclic graphs). A graph with a cycle has no valid topological ordering.

6. Trace-Through Example

Consider the following 6-node undirected graph:



Edges: (A,B), (A,C), (B,C), (B,D), (C,D), (C,E), (D,F), (E,F)

Degrees

Vertex	Neighbors	Degree
A	B, C	2
B	A, C, D	3
C	A, B, D, E	4
D	B, C, F	3
E	C, F	2
F	D, E	2

Total degree = 2 + 3 + 4 + 3 + 2 + 2 = 16. Number of edges = 8. Check: 16 = 2 * 8. Correct.

BFS starting from A

Queue operations and discovery order:

Step	Dequeue	Neighbors checked	Enqueue	Queue after	Discovered so far
1	A	B, C	B, C	[B, C]	A, B, C
2	B	A(vis), C(vis), D	D	[C, D]	A, B, C, D
3	C	A(vis), B(vis), D(vis), E	E	[D, E]	A, B, C, D, E
4	D	B(vis), C(vis), F	F	[E, F]	A, B, C, D, E, F
5	E	C(vis), F(vis)	--	[F]	A, B, C, D, E, F
6	F	D(vis), E(vis)	--	[]	A, B, C, D, E, F

--	--	--	--	--	--

BFS discovery order: A, B, C, D, E, F

Levels: A=0, B,C=1, D,E=2, F=3.

DFS starting from A

Recursive calls and backtracking:

Step	Visit	Action	Call stack
1	A	Go to B	[A]
2	B	Go to C	[A, B]
3	C	Go to D	[A, B, C]
4	D	Go to F	[A, B, C, D]
5	F	Go to E	[A, B, C, D, F]
6	E	All neighbors visited, backtrack	[A, B, C, D, F, E]
7-12	--	Backtrack through F, D, C, B, A	shrinking stack

DFS discovery order: A, B, C, D, F, E

Note: DFS and BFS produce different traversal orders on the same graph. The exact DFS order depends on the order in which neighbors are visited (here: alphabetical).

7. Operations + Big-O Table

Operation	Representation	Complexity	Notes
Add edge	Adjacency matrix	$O(1)$	Set $A[u][v] = 1$ (and $A[v][u]$ for undirected)
Add edge	Adjacency list	$O(1)$	Prepend to linked list
Remove edge	Adjacency matrix	$O(1)$	Set $A[u][v] = 0$
Remove edge	Adjacency list	$O(\text{degree}(u))$	Walk list to find and remove the node
Edge lookup	Adjacency matrix	$O(1)$	Read $A[u][v]$
Edge lookup	Adjacency list	$O(\text{degree}(u))$	Walk list at $\text{adj}[u]$
Add vertex	Adjacency matrix	$O(V^2)$	Must reallocate and copy the entire $V \times V$ matrix
Add vertex	Adjacency list	$O(1)$	Append a new (empty) slot to the array
BFS	Either	$O(V + E)$	Visits every vertex and edge once
DFS	Either	$O(V + E)$	Visits every vertex and edge once
Topological sort	Adjacency list	$O(V + E)$	DFS-based, requires a DAG
Find connected components	Either	$O(V + E)$	Run BFS/DFS from each unvisited

			vertex
Cycle detection	Either	$O(V + E)$	DFS with back-edge detection (directed) or union-find (undirected)
Shortest path (unweighted)	Either	$O(V + E)$	BFS from source
Shortest path (weighted)	Adjacency list	$O((V + E) \log V)$	Dijkstra with priority queue

Key insight: BFS and DFS both have the same asymptotic complexity $O(V + E)$. The choice between them depends on the application, not the speed.

8. C Code Reference

8.1 Graph struct with adjacency list

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int vertex;
    struct Node* next;
};

struct Graph {
    int V;           /* number of vertices */
    struct Node** adj; /* array of V linked lists */
};

struct Graph* createGraph(int V) {
    struct Graph* g = (struct Graph*)malloc(sizeof(struct Graph));
    g->V = V;
    g->adj = (struct Node**)malloc(V * sizeof(struct Node*));
    for (int i = 0; i < V; i++)
        g->adj[i] = NULL;
    return g;
}

void addEdge(struct Graph* g, int u, int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = g->adj[u];
    g->adj[u] = newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = u;
    newNode->next = g->adj[v];
    g->adj[v] = newNode;
}
```

8.2 BFS

```
struct Graph {
    int V;
    struct Node** adj;
};

struct Node {
    int vertex;
    struct Node* next;
};

#include <stdlib.h>
#include <stdio.h>
```

```

void bfs(struct Graph* g, int start) {
    int* visited = (int*)calloc(g->V, sizeof(int));
    int* queue = (int*)malloc(g->V * sizeof(int));
    int front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);

        struct Node* curr = g->adj[v];
        while (curr != NULL) {
            if (!visited[curr->vertex]) {
                visited[curr->vertex] = 1;
                queue[rear++] = curr->vertex;
            }
            curr = curr->next;
        }
    }
    free(visited);
    free(queue);
}

int main() {

    return 0;
}

```

8.3 DFS (recursive)

```

struct Graph {
    int V;
    struct Node** adj;
};

struct Node {
    int vertex;
    struct Node* next;
};

#include <stdlib.h>
#include <stdio.h>

void dfsVisit(struct Graph* g, int v, int* visited) {
    visited[v] = 1;
    printf("%d ", v);

    struct Node* curr = g->adj[v];
    while (curr != NULL) {
        if (!visited[curr->vertex])
            dfsVisit(g, curr->vertex, visited);
        curr = curr->next;
    }
}

void dfs(struct Graph* g, int start) {
    int* visited = (int*)calloc(g->V, sizeof(int));
    dfsVisit(g, start, visited);
    free(visited);
}

int main() {

    return 0;
}

```

8.4 Add edge (adjacency matrix version)

```
void addEdgeMatrix(int** adj, int u, int v, int V) {
    if (u < 0 || u >= V || v < 0 || v >= V) return;
    adj[u][v] = 1;
    adj[v][u] = 1;    /* undirected */
}
```

8.5 Topological sort

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int vertex;
    struct Node* next;
};
struct Graph {
    int V;
    struct Node** adj;
};
void topoUtil(struct Graph* g, int v, int* visited, int* stack, int* top) {
    visited[v] = 1;
    struct Node* curr = g->adj[v];
    while (curr != NULL) {
        if (!visited[curr->vertex])
            topoUtil(g, curr->vertex, visited, stack, top);
        curr = curr->next;
    }
    stack[( *top )++] = v;
}

void topologicalSort(struct Graph* g) {
    int* visited = (int*)calloc(g->V, sizeof(int));
    int* stack = (int*)malloc(g->V * sizeof(int));
    int top = 0;

    for (int i = 0; i < g->V; i++)
        if (!visited[i])
            topoUtil(g, i, visited, stack, &top);

    printf("Topological order: ");
    for (int i = top - 1; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");

    free(visited);
    free(stack);
}
```

9. ASCII Diagram

The example graph (6 nodes, 8 edges):

```
A --- B
|   /  |
|  /   |
| /    |
C --- D
|       |
|       |
E --- F
```

Adjacency matrix for the above graph:

A	B	C	D	E	F
---	---	---	---	---	---

- You need to iterate over neighbors of a vertex frequently.
- Memory is a constraint.
- You are doing BFS or DFS (both traverse neighbor lists).

Avoid graphs (use simpler structures) when:

- The data is strictly hierarchical (use a tree).
- The data is purely sequential (use an array or linked list).
- You only need key-value lookup (use a hash table).

11. Exam Hot Spots

This chapter has **1 MCQ corpus question** assigned to it. Graph concepts are also tested **indirectly** through tree questions, since trees are special graphs. Here is the direct question, followed by indirect patterns.

Pattern A: BST application (direct, ID 58)

Q58: "Which operation uses the data structure binary search tree?"

- **Correct:** "Clustering nodes based on a certain key value" — BSTs maintain a total order and support efficient clustering/grouping by key.
- **Wrong:** "Transitive closure" (graph algorithm), "Topological sort" (DAG algorithm, not BST-specific), "Depth-first traversal" (general graph/tree traversal, not BST-specific).

Takeaway: The exam expects you to know which algorithms belong to which data structures. BST operations include insertion, deletion, search, in-order traversal (sorted output), min/max, and clustering by key. Topological sort is a graph algorithm. DFS is a general traversal applicable to any graph or tree.

Pattern B: Trees as special graphs (indirect)

Many exam questions about trees are really testing graph concepts:

- **Tree traversal (pre/in/post-order)** is a special case of DFS on a directed acyclic graph.
- **BST search** is a special case of DFS on a binary search tree (a directed acyclic graph with ordering property).
- **AVL rotations** operate on a specific type of balanced BST, which is itself a special graph.
- **"Connected component"** in a tree is trivially the entire tree (a tree is connected). In a general graph, finding connected components requires BFS/DFS.

Pattern C: Graph terminology tested through trees

The exam may test the following graph terms using tree contexts:

Term	Tree equivalent
Vertex / node	Tree node
Edge	Parent-child link
Degree	Number of children (for general trees)
Path	Sequence from ancestor to descendant
Cycle	Does not exist in a tree (acyclic)
Connected	A tree is always connected
Root	The single root node
Leaf	Node with degree 0 (no children)

BFS	Level-order traversal
DFS	Pre-order / in-order / post-order traversal

Note: The corpus has sparse direct graph coverage (1 question). Graph concepts are predominantly tested through tree-related questions. Study trees thoroughly — they are the exam's primary vehicle for testing graph fundamentals.

12. Common Traps

The corpus and course material reveal consistent wrong-answer patterns. Memorize why each is wrong.

Wrong	Why it's wrong
"BFS uses a stack"	BFS uses a queue (FIFO). DFS uses a stack (LIFO) or recursion.
"DFS uses a queue"	DFS uses a stack (LIFO) or recursion. BFS uses a queue.
"In an undirected graph, the adjacency matrix is not symmetric"	The adjacency matrix of an undirected graph is always symmetric : $A[i][j] == A[j][i]$.
"An adjacency list uses $O(V^2)$ space"	An adjacency list uses $O(V + E)$ space. $O(V^2)$ is the adjacency matrix.
"An adjacency matrix uses $O(V + E)$ space"	An adjacency matrix uses $O(V^2)$ space, regardless of how many edges exist.
"Edge lookup in an adjacency list is $O(1)$ "	Edge lookup in an adjacency list is $O(\text{degree}(u))$, because you must walk the linked list. In a matrix it is $O(1)$.
"A tree is not a graph"	A tree is a graph — specifically a connected, acyclic, directed graph with a single root.
"A graph with a cycle can have a valid topological sort"	Topological sort only works on DAGs (directed acyclic graphs). Cycles make it impossible.
"In a directed graph, in-degree equals out-degree for every vertex"	This is only true for Eulerian circuits. In general, in-degree and out-degree can differ.
"BFS and DFS have different time complexity"	Both are $O(V + E)$ on adjacency list representations. The difference is in the order of visitation, not the speed.
"A complete graph has V edges"	A complete undirected graph has $V(V-1)/2$ edges. A complete directed graph has $V(V-1)$ edges.
"Adding a vertex to an adjacency matrix is $O(1)$ "	Adding a vertex requires reallocating and copying the entire $V \times V$ matrix, which is $O(V^2)$.
"Disconnected graphs cannot have cycles"	A disconnected graph can have cycles in any of its components. Connectivity and cycles are independent properties.
"Bipartite graphs cannot have odd-length cycles"	A graph is bipartite if and only if it has no odd-length cycles. Odd cycles break bipartiteness.
"Topological sort uses BFS"	Topological sort is typically implemented with DFS (post-order push to stack). Kahn's algorithm uses BFS, but DFS is the standard approach taught.

13. Quick-Recall Card

GRAPH DEFINITION

$G = (V, E)$ V = vertices, E = edges

Directed (digraph): edge (u,v) has direction $u \rightarrow v$

Undirected: edge (u,v) is bidirectional

Weighted: each edge has a numeric cost
Complete: every pair of vertices is connected

REPRESENTATIONS

Adjacency Matrix: $V \times V$ array, $O(V^2)$ space
 $A[i][j] = 1$ (or weight) if edge exists
 Edge lookup: $O(1)$
 Add vertex: $O(V^2)$ – must reallocate
Adjacency List: array of linked lists, $O(V+E)$ space
 adj[u] lists all neighbors of u
 Edge lookup: $O(\text{degree}(u))$
 Add vertex: $O(1)$ – append to array

TRAVERSALS

BFS: queue, level-by-level, $O(V+E)$
 Good for: shortest path (unweighted), connected components
DFS: stack/recursion, depth-first, $O(V+E)$
 Good for: cycle detection, topological sort, bridges

TOPOLOGICAL SORT

Only works on DAGs (directed acyclic graphs)
DFS-based: post-order push to stack, then pop all
Result: linear ordering where u comes before v for every edge (u,v)

DEGREE

Undirected: $\text{degree}(v)$ = number of edges incident on v
Directed: $\text{in-degree}(v) + \text{out-degree}(v) = \text{degree}(v)$
Handshaking lemma: sum of all degrees = $2|E|$

EXAM GOLD

"A tree IS a graph (connected, acyclic, directed)"
"BFS uses a queue; DFS uses a stack"
"Adjacency matrix is $O(V^2)$; adjacency list is $O(V+E)$ "
"Topological sort requires a DAG (no cycles)"
"BFS and DFS are both $O(V+E)$ "
"Edge lookup: matrix $O(1)$, list $O(\text{degree})$ "

\newpage

Chapter 13 -- Sorting Algorithms

Applications chapter. Sorting is the most fundamental algorithmic problem in computer science. Nearly every data structure course, interview, and exam tests whether you can distinguish between sorting algorithms by their complexity, stability, space cost, and best-use scenarios. This chapter covers comparison-based and non-comparison sorts, with C code, step-by-step traces, and the lower-bound proof.

1. Learning Objectives

After reading this chapter, you will be able to:

- Explain how bubble sort, selection sort, and insertion sort work, including their $O(n^2)$ worst-case behavior.
- Implement merge sort using the divide-and-conquer paradigm and analyse its $O(n \log n)$ time and $O(n)$ auxiliary space.
- Implement quick sort with a partition strategy and explain why its worst case is $O(n^2)$ while its average is $O(n \log n)$.
- Describe heap sort and its $O(n \log n)$ guarantee by building on Chapter 7 (Heaps).
- Understand counting sort and radix sort as non-comparison algorithms that break the $O(n \log n)$ lower bound.
- Distinguish stable from unstable sorts, in-place from out-of-place sorts, and know which algorithms fall into which category.
- Apply the comparison-based lower bound $\Omega(n \log n)$ to prove that no comparison sort can do better on average.

2. Mental Model & Analogy

Think of sorting as organizing a hand of playing cards. You hold five cards and want them in order from left to right. Different strategies exist:

- **Bubble sort** is like repeatedly scanning left to right, swapping adjacent cards that are out of order. After each full pass, the largest unsorted card "bubbles up" to its correct position at the right end. Simple, but you make many passes.
- **Selection sort** is like scanning the entire hand to find the smallest card, then placing it at position 1. Then scanning the remainder for the next smallest, and so on. Few swaps, but always scans the whole remaining portion.
- **Insertion sort** is like picking up cards one at a time and sliding each into its correct position among the already-sorted cards to the left. Very natural for small hands.
- **Merge sort** is like splitting the hand in half, sorting each half separately (recursively), then merging the two sorted halves back together into one sorted hand. Requires a second hand (auxiliary space) to hold the merge result.
- **Quick sort** is like picking one card as a "pivot," rearranging so everything smaller goes left and everything larger goes right, then recursively sorting each side. Fast in practice but can degenerate if you pick bad pivots.
- **Heap sort** is like dumping all cards into a heap (priority queue), then repeatedly extracting the minimum. Guarantees $O(n \log n)$ without the risk of a bad pivot.

The key insight: the strategy you choose determines not just speed, but also memory usage, whether the original order of equal elements is preserved (stability), and whether you can sort without extra memory (in-place).

3. Memory Layout

Sorting algorithms differ dramatically in how they use memory:

```
In-Place Sorts (bubble, selection, insertion, heap sort)
=====
Array:  [ 5 | 3 | 8 | 1 | 2 ]    <-- single array, swaps in place
        ^                      no auxiliary array needed
        |                      O(1) extra space
        | same array modified

Out-of-Place Sort (merge sort)
=====
Original: [ 5 | 3 | 8 | 1 | 2 ]

Divide:   [ 5 | 3 ] [ 8 | 1 | 2 ]    <-- logical split (indices)

Merge:    temp[] allocated on heap    <-- O(n) auxiliary space
          merge [3,5] and [1,2,8] into temp
          copy temp back to original

+-----+ HIGH ADDRESSES
|      Stack      | <-- local vars, recursion frames
| merge sort uses | merge sort: O(log n) stack depth
| O(log n) stack for | quick sort: O(n) worst stack
| recursive calls |
+-----+
|      ...      |
+-----+
|      Heap      | <-- temp[] for merge sort: O(n) bytes
|                | O(1) for in-place sorts
+-----+ LOW ADDRESSES
```

Key facts the exam will test:

- **In-place** means the sort uses $O(1)$ extra memory beyond the input array. Bubble, selection, insertion, and heap sort are in-place.
- **Out-of-place** means the sort requires additional memory proportional to n . Merge sort needs $O(n)$ auxiliary space.
- **Quick sort** is usually classified as in-place (the partition is done within the array), but the recursion stack can grow to $O(n)$ in the worst case.

- **Non-comparison sorts** (counting sort, radix sort) may need $O(n + k)$ auxiliary space where k is the range of values.

4. Definition & Properties

Sorting -- the problem of rearranging elements of an array so they appear in non-decreasing (or non-increasing) order according to some comparison operator.

Properties that classify sorting algorithms:

- **Stable sort:** If two elements have equal keys, their relative order in the input is preserved in the output. Bubble sort, insertion sort, merge sort, and counting sort are stable. Selection sort, heap sort, and quick sort (standard partition) are **not** stable.
- **In-place sort:** The algorithm uses $O(1)$ auxiliary memory beyond the input. Bubble, selection, insertion, and heap sort are in-place. Merge sort is **not** in-place.
- **Comparison-based sort:** The algorithm decides the order of elements solely by pairwise comparisons. Bubble, selection, insertion, merge, quick, and heap sort are comparison-based. Counting sort and radix sort are **not** comparison-based.
- **Comparison-based lower bound:** Any comparison-based sorting algorithm must make at least $\Omega(n \log n)$ comparisons in the worst case. This is proved via a decision-tree argument: there are $n!$ possible orderings, each leaf represents one, and a binary tree with $n!$ leaves has height at least $\log_2(n!)$ which is $\Omega(n \log n)$.

The lower bound matters: No comparison-based sort can guarantee better than $O(n \log n)$ in the worst case. Merge sort and heap sort achieve this bound. Quick sort achieves it on average but not in the worst case. Bubble, selection, and insertion sort are $O(n^2)$ and thus far from optimal for large inputs.

5. Operations -- Step-by-Step Mechanics

5.1 Bubble Sort

Idea: Repeatedly walk through the array, comparing adjacent elements and swapping them if out of order. After each pass, the largest unsorted element is in its final position.

```
[5,3,8,1,2]
Pass 1: swap(5,3)->[3,5,8,1,2], swap(8,1)->[3,5,1,8,2], swap(8,2)->[3,5,1,2,8]
Pass 2: swap(5,1)->[3,1,5,2,8], swap(5,2)->[3,1,2,5,8]
Pass 3: swap(3,1)->[1,3,2,5,8], swap(3,2)->[1,2,3,5,8]
Pass 4: no swaps -> sorted.
```

Optimization: If a full pass produces no swaps, the array is sorted and we stop early.

5.2 Selection Sort

Idea: Divide the array into sorted (left) and unsorted (right) portions. Find the minimum in the unsorted portion and swap it into the first unsorted position.

```
[5,3,8,1,2] -> min=1 at idx 3, swap a[0]<->a[3]: [1,3,8,5,2]
[1,3,8,5,2] -> min=2 at idx 4, swap a[1]<->a[4]: [1,2,8,5,3]
[1,2,8,5,3] -> min=3 at idx 4, swap a[2]<->a[4]: [1,2,3,5,8]
[1,2,3,5,8] -> min=5 at idx 3, no swap needed.
```

Key fact: Selection sort always makes exactly $n-1$ swaps. It is the minimum-swap sort, but it always does $O(n^2)$ comparisons regardless of input order.

5.3 Insertion Sort

Idea: Build the sorted array one element at a time. Take the next element and insert it into its correct position among the already-sorted elements to its left, shifting larger elements right.

```
[5,3,8,1,2]
Step 1: take 3, shift 5 right, insert 3: [3,5,8,1,2]
```

```
Step 2: take 8, 8>5 no shift:      [3,5,8,1,2]
Step 3: take 1, shift 8,5,3 right, insert 1: [1,3,5,8,2]
Step 4: take 2, shift 8,5 right, insert 2:  [1,2,3,5,8]
```

Best case: If the array is already sorted, insertion sort does zero shifts and runs in $O(n)$. This makes it the best choice for nearly-sorted data.

5.4 Merge Sort -- Divide and Conquer

Idea: Divide the array into two halves, recursively sort each half, then merge the two sorted halves.

```
DIVIDE: [5,3,8,1,2] -> [5,3,8] [1,2] -> [5,3] [8] [1] [2] -> [5] [3] [8] [1] [2]
MERGE:  [5]+[3]->[3,5] | [3,5]+[8]->[3,5,8] | [1]+[2]->[1,2] | [3,5,8]+[1,2]->[1,2,3,5,8]
```

Master theorem: $T(n) = 2T(n/2) + O(n)$. By Master theorem ($a=2$, $b=2$, $f(n)=n$): $T(n) = O(n \log n)$. $\log n$ levels, $O(n)$ merge work per level.

5.5 Quick Sort -- Partition

Idea: Pick a pivot, partition so smaller elements go left and larger go right, then recursively sort each partition.

```
Quick sort [5,3,8,1,2], pivot = 5 (Lomuto):
  3<5 swap->[3,5,8,1,2], 8>5 skip, 1<5 swap->[3,5,1,8,2], 2<5 swap->[3,5,1,2,8]
  Swap pivot into place: [3,1,2,5,8]
  Left=[3,1,2], Right=[8] -> recurse -> [1,2,3,5,8]
```

Lomuto partition: Indices i (boundary) and j (scanner). If $a[j] \leq \text{pivot}$, advance i and swap $a[i]$ with $a[j]$. Finally swap pivot into position $i+1$.

Worst case: Pivot is always min or max (e.g., sorted array with first-element pivot): $T(n) = T(n-1) + O(n) = O(n^2)$.

Best/average case: Good pivots produce balanced partitions: $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

5.6 Heap Sort

Idea: Build a max-heap, then repeatedly extract the maximum and place it at the end.

```
Build max-heap from [5,3,8,1,2]: -> [8,3,5,1,2]
Extract 8: swap with last, heapify -> [5,3,2,1 | 8]
Extract 5: swap with last, heapify -> [3,1,2 | 5,8]
Extract 3: swap with last, heapify -> [2,1 | 3,5,8]
Extract 2: swap with last      -> [1 | 2,3,5,8]
```

Heap sort is $O(n \log n)$ in all cases and in-place, but not stable.

5.7 Counting Sort (Non-Comparison)

Idea: If elements are integers in $[0, k]$, count occurrences, use prefix sums to place elements directly.

```
Input: [4,2,2,8,3,3,1]
Count: count[1]=1, count[2]=2, count[3]=2, count[4]=1, count[8]=1
Prefix: count[1]=1, count[2]=3, count[3]=5, count[4]=6, count[8]=7
Place elements (right to left for stability) -> [1,2,2,3,3,4,8]
```

Complexity: $O(n + k)$ time, $O(n + k)$ space. Beats $\Omega(n \log n)$ because it is NOT comparison-based.

5.8 Radix Sort (Non-Comparison)

Idea: Sort digit by digit (LSD to MSD), using a stable sub-sort (counting sort) at each position.

```
Input: [170, 45, 75, 90, 802, 24, 2, 66]
LSD sort: [170, 90, 802, 2, 24, 45, 75, 66]
Tens sort: [802, 2, 24, 45, 66, 170, 75, 90]
MSD sort: [2, 24, 45, 66, 75, 90, 170, 802]
```

Complexity: $O(d * (n + k))$ where d = digits, n = elements, k = base. If d is constant, this is $O(n)$. Radix sort MUST be stable at each pass.

6. Trace-Through Example

Sort [5, 3, 8, 1, 2] with bubble sort and merge sort.

Bubble Sort

Initial: [5,3,8,1,2]
Pass 1: swap(5,3)->[3,5,8,1,2], swap(8,1)->[3,5,1,8,2], swap(8,2)->[3,5,1,2,8]
Pass 2: swap(5,1)->[3,1,5,2,8], swap(5,2)->[3,1,2,5,8]
Pass 3: swap(3,1)->[1,3,2,5,8], swap(3,2)->[1,2,3,5,8]
Pass 4: no swaps. Done.
Comparisons: 10, Swaps: 7

Merge Sort

DIVIDE: [5,3,8,1,2] -> [5,3,8] [1,2] -> [5,3] [8] [1] [2] -> [5] [3] [8] [1] [2]
MERGE: [5]+[3]->[3,5] | +[8]->[3,5,8] | [1]+[2]->[1,2] | [3,5,8]+[1,2]->[1,2,3,5,8]
Comparisons: 10, Aux space: up to 5 elements

7. Operations + Big-O Table

Algorithm	Best Case	Average Case	Worst Case	Space	Stable	In-Place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$	No	Yes**
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes	No
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n + k)$	Yes	No

* Quick sort uses $O(\log n)$ stack space on average for recursion, $O(n)$ in the worst case.

** Quick sort is in-place by the standard definition (partition within the array), though the recursion depth is not $O(1)$.

Lower bound reminder: The bottom three rows (counting sort, radix sort) achieve better than $O(n \log n)$ because they are NOT comparison-based. They exploit the structure of the keys (integers in a bounded range) to bypass the comparison lower bound.

8. C Code Reference

8.1 Bubble Sort

```
void bubbleSort(int a[], int n) {
    int i, j, temp;
    int swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
                swapped = 1;
            }
        }
    }
}
```

```

        if (!swapped) break;  /* already sorted */
    }
}

```

8.2 Selection Sort

```

void selectionSort(int a[], int n) {
    int i, j, minIdx, temp;
    for (i = 0; i < n - 1; i++) {
        minIdx = i;
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[minIdx])
                minIdx = j;
        }
        if (minIdx != i) {
            temp = a[i];
            a[i] = a[minIdx];
            a[minIdx] = temp;
        }
    }
}

```

8.3 Insertion Sort

```

void insertionSort(int a[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

```

8.4 Merge Sort

```

void merge(int a[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    int i, j, k;

    for (i = 0; i < n1; i++)
        L[i] = a[left + i];
    for (j = 0; j < n2; j++)
        R[j] = a[mid + 1 + j];

    i = 0; j = 0; k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])  /* <= ensures stability */
            a[k++] = L[i++];
        else
            a[k++] = R[j++];
    }
    while (i < n1)
        a[k++] = L[i++];
    while (j < n2)
        a[k++] = R[j++];
}

void mergeSort(int a[], int left, int right) {

```

```

    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(a, left, mid);
        mergeSort(a, mid + 1, right);
        merge(a, left, mid, right);
    }
}

```

8.5 Quick Sort (Lomuto Partition)

```

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    int j, temp;

    for (j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }
    temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

```

8.6 Heap Sort

```

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i) {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;
        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    int i, temp;
    for (i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);
    for (i = n - 1; i > 0; i--) {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
    }
}

```

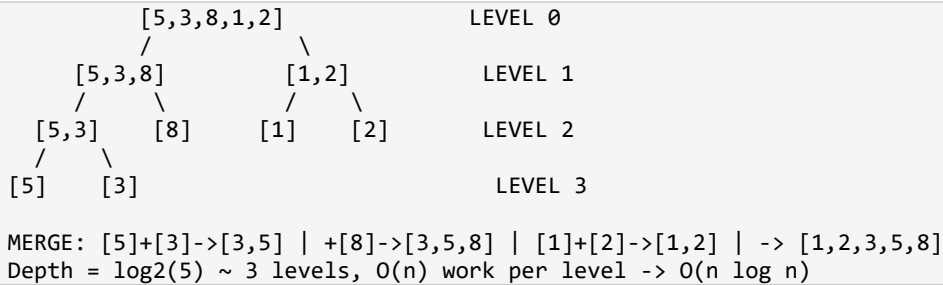
```

    heapify(a, i, 0);
}
}

```

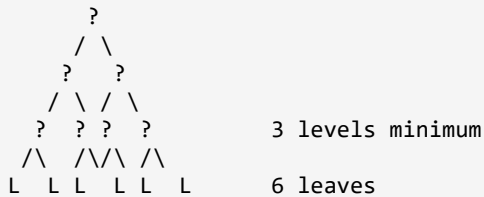
9. ASCII Diagram

Merge sort tree (divide/merge):



Decision tree (lower bound proof):

For $n=3$, there are $3! = 6$ orderings. A binary tree with 6 leaves:



Height $\geq \log_2(n!) = \Omega(n \log n)$

10. When to Use / When to Avoid

Algorithm	Best for	Avoid when
Bubble Sort	Tiny/nearly sorted arrays, teaching	Large data, performance matters
Selection Sort	Minimizing swaps (flash memory)	Stability needed, large data
Insertion Sort	Small/nearly sorted, online sorting	Large random data
Merge Sort	Guaranteed $O(n \log n)$, stable, linked lists	Memory is tight (needs $O(n)$ space)
Quick Sort	Fastest average, cache-friendly	Worst case must be avoided, stability needed
Heap Sort	Guaranteed $O(n \log n)$ + in-place	Stability needed, cache performance matters
Counting Sort	Integer keys in small range $[0, k]$	Non-integer data, $k \gg n$
Radix Sort	Fixed-length integers/strings	Variable-length data, many digits

11. Exam Hot Spots

This chapter has **1 MCQ corpus question** directly assigned (ID 3960), plus sorting concepts appear indirectly in several other chapters.

Corpus Question

Q3960: "Ce metoda de cautare folosim pentru a gasi un nod intr-un arbore?" (What search method do we use to find a node in a tree?)

- **Correct:** "Divide et Impera" (Divide and Conquer)

- **Wrong:** "Bubble Sort", "Metoda selectiei", "Metoda numararii"
- **Note:** This question tests whether you recognize that BST search (Chapter 9/10) uses the divide-and-conquer paradigm -- the same paradigm that powers merge sort and quick sort. At each step, you eliminate half the search space by comparing with the current node. The connection is: **binary search in a BST is $O(\log n)$ because it divides the problem in half at each step, just like merge sort divides the array in half.**

Indirect Sorting Appearances

- **Heap sort (Chapter 7):** Building a heap is $O(n)$, extracting n elements is $O(n \log n)$. The heap data structure from Chapter 7 directly enables heap sort. Exam questions about heaps often test whether you know heap sort exists and is $O(n \log n)$ in-place.
- **BST inorder traversal (Chapters 9-10):** An inorder traversal of a BST produces sorted output. This is sometimes tested as "what traversal gives sorted order in a BST?" The answer is inorder (left-root-right).
- **AVL trees (Chapter 12):** Maintaining balance ensures $O(\log n)$ operations, which is relevant to understanding why $O(n \log n)$ is the target for sorting -- the same $\log n$ depth that makes balanced trees fast makes comparison sorts limited.

12. Common Traps

Wrong	Why it's wrong
"Bubble sort is $O(n \log n)$ "	It is $O(n^2)$ average/worst. Only best case (sorted + early termination) is $O(n)$.
"Selection sort is stable"	Selection sort swaps across long distances, changing relative order of equal elements. NOT stable.
"Insertion sort is always $O(n^2)$ "	It is $O(n)$ on sorted/nearly-sorted input. Only $O(n^2)$ worst case (reverse-sorted).
"Quick sort is always $O(n \log n)$ "	$O(n^2)$ worst case with bad pivot (e.g., sorted input, first-element pivot).
"Merge sort is in-place"	Requires $O(n)$ auxiliary space for the temp array. NOT in-place.
"Heap sort is not $O(n \log n)$ "	$O(n \log n)$ in ALL cases -- best, average, worst. Heap guarantees $\log n$ height.
"Counting sort is comparison-based"	Uses direct addressing, not comparisons. Achieves $O(n + k)$, bypassing $\Omega(n \log n)$.
"Radix sort compares elements"	Processes digits using counting sort. Does NOT compare whole elements.
" $\Omega(n \log n)$ applies to all sorts"	Only to COMPARISON-BASED sorts. Counting/radix sort are not comparison-based.
"Stable sort means error-free execution"	"Stable" means equal elements keep their original relative order. Nothing to do with correctness.
"In-place means zero extra memory"	Means $O(1)$ extra memory. Even in-place sorts use temporary swap variables.
"Heap sort and heap data structure are unrelated"	Heap sort USES the heap. Building max-heap + extract max IS heap sort.

13. Quick-Recall Card

ALGORITHM COMPLEXITIES

Bubble: best $O(n)$, avg $O(n^2)$, worst $O(n^2)$, $O(1)$, stable

Selection: best $O(n^2)$, avg $O(n^2)$, worst $O(n^2)$, $O(1)$, unstable
 Insertion: best $O(n)$, avg $O(n^2)$, worst $O(n^2)$, $O(1)$, stable
 Merge: best $O(n \log n)$, avg $O(n \log n)$, worst $O(n \log n)$, $O(n)$, stable
 Quick: best $O(n \log n)$, avg $O(n \log n)$, worst $O(n^2)$, $O(\log n)$, unstable
 Heap: best $O(n \log n)$, avg $O(n \log n)$, worst $O(n \log n)$, $O(1)$, unstable
 Counting: $O(n + k)$, $O(n + k)$ space, stable
 Radix: $O(d(n+k))$, $O(n + k)$ space, stable

KEY DEFINITIONS

Stable: equal elements keep original relative order
 In-place: $O(1)$ extra memory
 Comparison-based: decides order by pairwise comparisons only

LOWER BOUND

Omega($n \log n$) for comparison sorts (decision tree: $n!$ leaves $\rightarrow$ height $\geq \log_2(n!)$)
 Counting/radix bypass this (NOT comparison-based)

WHEN TO USE

Nearly sorted $\rightarrow$ Insertion | Small $\rightarrow$ Insertion | Guaranteed $O(n \log n)$ stable $\rightarrow$ Merge
 Guaranteed $O(n \log n)$ in-place $\rightarrow$ Heap | Fastest avg $\rightarrow$ Quick | Integers small range $\rightarrow$ Counting
 Integers many digits $\rightarrow$ Radix | Minimize swaps $\rightarrow$ Selection

QUICK SORT PARTITION (Lomuto)

pivot=a[high]; i=low-1; for j=low..high-1: if a[j]<=pivot: i++, swap a[i],a[j]
 swap a[i+1],a[high]

EXAM GOLD

Bubble best $O(n)$ | Selection always $n-1$ swaps | Insertion best for nearly sorted
 Merge stable but $O(n)$ space | Quick worst $O(n^2)$ bad pivot | Heap $O(n \log n)$ all cases
 Omega($n \log n$) for comparison sorts only | BST inorder = sorted | Divide et Impera

\newpage

Chapter 14 -- Applications: Compression, String Processing & Library Functions

Capstone chapter. This chapter connects every data structure studied so far to real-world applications: string processing uses char arrays and pattern matching, compression uses trees and priority queues, base conversion uses stacks, and the C standard library provides the building blocks for all of them. This is a concept-driven chapter with 0 MCQ corpus questions; the exam tests these topics through code-trace and fill-in-the-blank questions scattered across earlier chapters.

1. Learning Objectives

After reading this chapter, you will be able to:

- Represent strings as null-terminated char arrays and perform basic operations (length, copy, compare) with correct boundary handling.
- Explain pattern matching and understand why KMP achieves $O(n+m)$ instead of naive $O(n*m)$.
- Describe run-length encoding (RLE) and Huffman coding as two different compression strategies.
- Convert decimal numbers to binary using a stack, and trace the remainder-collecting algorithm step by step.
- Convert infix expressions to postfix notation using Dijkstra's Shunting-Yard algorithm.
- Use C standard library functions from `<string.h>`, `<stdlib.h>`, and `<ctype.h>` correctly.
- Identify which earlier data structure is required for each application.

2. Mental Model & Analogy

String processing is word search in a book. The naive approach reads every word on every page; KMP is like a bookmark that skips ahead when a partial match already matched.

Compression is packing a suitcase. RLE says "12 identical socks in one bag" -- simple, works with repetition. Huffman says "frequent small items in easy-to-reach pockets, rare large items at the bottom" -- optimal, but requires building a priority tree first.

Base conversion is a stack of receipts. Dividing 13 by 2 produces remainders in reverse order. A stack collects them bottom-up; popping gives the correct digit sequence.

3. Memory Layout

3.1 Strings as char arrays

A C string is a contiguous array of `char` values, terminated by `'\0'` (value 0).

```
char str[] = "Hello";

+-----+-----+-----+-----+-----+
| 'H'   | 'e'   | 'l'   | 'l'   | 'o'   | '\0'  |
+-----+-----+-----+-----+-----+
| [0]   | [1]   | [2]   | [3]   | [4]   | [5]   |

sizeof(str) = 6 (includes the null terminator)
strlen(str) = 5 (does NOT include the null terminator)
```

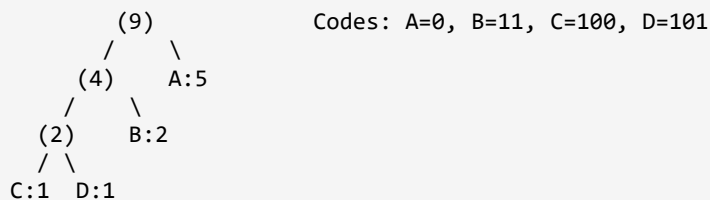
Key facts: `str` decays to `&str[0]` in most contexts (Chapter 1). String literals are read-only. Without a null terminator, `strlen` reads past the array boundary.

3.2 Huffman tree

A Huffman tree is a binary tree where each leaf holds a character and its frequency. Built bottom-up using a priority queue (min-heap):

Frequency table: A=5, B=2, C=1, D=1

Leaves (sorted): [C:1] [D:1] [B:2] [A:5]
Merge C+D -> CD:2 [CD:2] [B:2] [A:5]
Merge CD+B -> CDB:4 [A:5] [CDB:4]
Merge -> root:9



The priority queue used during construction is typically a min-heap (Chapter 7).

4. Definition & Properties

String -- a null-terminated sequence of characters in a contiguous byte array. Fixed at declaration time or heap-allocated. Mutable, but size cannot grow without reallocation. Length is $O(n)$ to compute.

Compression -- reduces data size for storage or transmission. **Lossless** (RLE, Huffman) allows perfect recovery; **lossy** (JPEG, MP3) discards data (not covered here). **Compression ratio** = original size / compressed size.

RLE replaces consecutive repeated characters with (character, count) pairs. Simple, $O(n)$, but only effective on data with long runs.

Huffman Coding assigns variable-length bit codes based on frequency. More frequent characters get shorter codes. Requires a binary tree built via a priority queue. Produces optimal prefix-free codes.

Base systems:

Base	Digits	Used in
Binary (2)	0, 1	Computer hardware
Octal (8)	0-7	Unix file permissions
Decimal (10)	0-9	Everyday counting
Hexadecimal (16)	0-9, A-F	Memory addresses, color codes

Expression notations:

- **Infix:** operators between operands ($3 + 4$). Requires precedence rules and parentheses.
- **Prefix (Polish):** operators before operands ($+ 3 4$). No parentheses needed.
- **Postfix (Reverse Polish):** operators after operands ($3 4 +$). Evaluated with a stack in a single left-to-right pass.

5. Operations -- Step-by-Step Mechanics

5.1 String operations

- **strlen(s):** walk `s` counting bytes until `'\0'`. Returns count excluding terminator.
- **strcpy(dst, src):** copy each byte from `src` to `dst` including `'\0'`. Caller must ensure `dst` is large enough.
- **strcmp(a, b):** compare character by character. Return `<0` if `a < b`, `0` if equal, `>0` if `a > b`. Stops at first `'\0'`.
- **strcat(dst, src):** find `'\0'` in `dst`, overwrite it with `src[0]`, copy all of `src` including its terminator.

5.2 Base conversion with a stack

To convert decimal `N` to binary:

1. While `N > 0`:
 1. Push (`N % 2`) onto the stack.
 2. `N = N / 2`.
2. Pop all elements from the stack. They form the binary representation from most significant to least significant bit.

Why a stack? The remainders are generated in reverse order (least significant first). A stack reverses them.

5.3 Infix to postfix (Dijkstra Shunting-Yard)

Scan the infix expression left to right. For each token:

Token type	Action
Operand (number)	Append directly to output
Left parenthesis (	Push onto operator stack
Right parenthesis)	Pop operators to output until (is found; discard the (
Operator	Pop operators with $\geq$ precedence to output, then push this operator

When scanning is done, pop all remaining operators to output.

Operator precedence (higher number = higher priority):

Precedence	Operators
------------	-----------

1 (lowest)	+, -
2	*, /
3 (highest)	^

5.4 Run-Length Encoding

Scan the string left to right. For each run of identical characters:

1. Record the character.
2. Count how many consecutive occurrences follow.
3. Emit (character, count).

If the count is 1, some implementations emit just the character without a count.

5.5 Huffman coding

1. Count frequency of each character in the input.
2. Create a leaf node for each character and insert into a min-heap.
3. While the heap has more than one node:
 3. Extract two nodes with the smallest frequencies.
 4. Create a new internal node with frequency = sum of the two.
 5. Set the two extracted nodes as left and right children.
 6. Insert the new node back into the heap.
4. The remaining node is the root. Traverse left = append '0' to code, right = append '1'.

6. Trace-Through Example

6.1 Decimal 13 to binary using a stack

```
N = 13: push 13%2=1. N=6. Stack: [1]
N = 6: push 6%2=0. N=3. Stack: [1, 0]
N = 3: push 3%2=1. N=1. Stack: [1, 0, 1]
N = 1: push 1%2=1. N=0. Stack: [1, 0, 1, 1]
Pop all: 1, 1, 0, 1 => "1101". Verify: 8+4+0+1=13.
```

6.2 Compress "AAABBBCC" with RLE

```
A A A B B B C C => A3 B3 C2
Original: 8 bytes | Compressed: 6 bytes | Ratio: 1.33
RLE is not effective here because the runs are short.
```

6.3 Infix "3+4*2" to postfix

Token	Action	Output	Stack
3	operand -> output	3	(empty)
+	push onto stack	3	+
4	operand -> output	3 4	+
*	precedence(*)=2 > precedence(+)=1, push	3 4	+ *
2	operand -> output	3 4 2	+ *
(end)	pop all operators	3 4 2 * +	(empty)

Postfix: "3 4 2 * +"

Evaluation:

```
push 3      Stack: [3]
push 4      Stack: [3, 4]
```

```

push 2      Stack: [3, 4, 2]
*: pop 2,4 -> 4*2=8, push 8   Stack: [3, 8]
+: pop 8,3 -> 3+8=11, push 11 Stack: [11]
Result: 11

```

7. Operations + Big-O Table

Operation	Complexity	Notes
<code>strlen(s)</code>	$O(n)$	Must scan until <code>'\0'</code>
<code>strcpy(dst, src)</code>	$O(n)$	Copies $n+1$ bytes including terminator
<code>strcmp(a, b)</code>	$O(\min(n,m))$	Stops at first difference or <code>'\0'</code>
<code>strcat(dst, src)</code>	$O(n+m)$	Scans <code>dst</code> for <code>'\0'</code> , then copies <code>src</code>
Naive pattern search	$O(n*m)$	For each position, compare entire pattern
KMP pattern search	$O(n+m)$	Precompute failure function, then single scan
RLE compression/decompression	$O(n)$	Single left-to-right scan
Huffman build	$O(n \log k)$	k = unique characters; heap operations dominate
Base conversion	$O(\log_b N)$	b = target base, N = number
Infix to postfix	$O(n)$	Each token pushed/popped at most once
<code>qsort(arr, n, sz, cmp)</code>	$O(n \log n)$	Quicksort; comparator called $O(n \log n)$ times
<code>bsearch(key, arr, n, sz, cmp)</code>	$O(\log n)$	Binary search on sorted array

8. C Code Reference

8.1 Basic string operations

```

#include <stdio.h>
#include <string.h>

int main()
{
    char a[] = "Hello";
    char b[20];

    /* strlen: count until '\0' */
    printf("Length of \"%s\" = %lu\n", a, strlen(a));
    /* Output: Length of "Hello" = 5 */

    /* strcpy: copy including '\0' */
    strcpy(b, a);
    printf("b = \"%s\"\n", b);
    /* Output: b = "Hello" */

    /* strcmp: lexicographic comparison */
    printf("cmp(Hello, Hello) = %d\n", strcmp("Hello", "Hello"));
    printf("cmp(Hello, World) = %d\n", strcmp("Hello", "World"));
    /* Output: cmp(Hello, Hello) = 0 */
    /* Output: cmp(Hello, World) = -15 (diff of 'H' - 'W') */

    /* strcat: append to end */
    strcat(b, " World");
}

```

```

printf("b = \"%s\\n\"", b);
/* Output: b = "Hello World" */

return 0;
}

```

8.2 Base conversion: decimal to binary using a stack

```

#include <stdio.h>

#define MAX 64

void decimalToBinary(int n)
{
    int stack[MAX];
    int top = -1;

    if (n == 0) {
        printf("0\\n");
        return;
    }

    while (n > 0) {
        stack[++top] = n % 2;
        n /= 2;
    }

    printf("Binary: ");
    while (top >= 0) {
        printf("%d", stack[top--]);
    }
    printf("\\n");
}

int main()
{
    decimalToBinary(13);    /* Output: Binary: 1101 */
    decimalToBinary(255);  /* Output: Binary: 11111111 */
    decimalToBinary(0);    /* Output: 0 */
    return 0;
}

```

8.3 Infix to postfix conversion

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
char peek() { return stack[top]; }
int isEmpty() { return top == -1; }

int precedence(char op)
{
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
}

```

```

    return 0;
}

void infixToPostfix(const char* infix)
{
    char postfix[MAX];
    int j = 0;

    for (int i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];

        if (isdigit(c) || isalpha(c)) {
            postfix[j++] = c;
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (!isEmpty() && peek() != '(') {
                postfix[j++] = pop();
            }
            pop(); /* discard '(' */
        } else {
            /* operator */
            while (!isEmpty() && precedence(peek()) >= precedence(c)) {
                postfix[j++] = pop();
            }
            push(c);
        }
    }

    while (!isEmpty()) {
        postfix[j++] = pop();
    }
    postfix[j] = '\0';

    printf("Infix:   %s\n", infix);
    printf("Postfix: %s\n", postfix);
}

int main()
{
    infixToPostfix("3+4*2"); /* Output: 342*+ */
    infixToPostfix("2*(4+3)+9/3"); /* Output: 243+*93/+ */
    return 0;
}

```

8.4 qsort and bsearch

```

#include <stdio.h>
#include <stdlib.h>

int compare(const void* a, const void* b)
{
    return (*(int*)a - *(int*)b);
}

int main()
{
    int arr[] = {42, 15, 8, 23, 4, 16};
    int n = sizeof(arr) / sizeof(arr[0]);

    /* qsort: O(n Log n), comparator receives const void* pointers */
    qsort(arr, n, sizeof(int), compare);
    printf("Sorted: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

```

```

/* Output: Sorted: 4 8 15 16 23 42 */

/* bsearch: O(log n), array MUST be sorted by the same comparator */
int key = 23;
int* found = (int*)bsearch(&key, arr, n, sizeof(int), compare);
if (found)
    printf("Found %d at index %ld\n", key, found - arr);
/* Output: Found 23 at index 4 */

return 0;
}

```

Key: The comparator receives `const void*` pointers. You must cast them to the actual type before comparing. Return convention: negative if first < second, zero if equal, positive if first > second. `bsearch` returns a pointer into the array (or NULL), and you can compute the index via pointer subtraction.

9. ASCII Diagram

9.1 Stack during decimal-to-binary conversion of 13

N=13: push 13%2=1	N=6: push 6%2=0	N=3: push 3%2=1	N=1: push 1%2=1
+-----+ 1 <-- top +-----+	+-----+ 0 <-- top +-----+ 1 +-----+	+-----+ 1 <-- top +-----+ 0 +-----+ 1 +-----+	+-----+ 1 <-- top +-----+ 0 +-----+ 1 +-----+ 1 +-----+

Pop order: 1, 1, 0, 1 => binary "1101"

9.2 Infix-to-postfix stack trace for "3+4*2"

Input: 3 + 4 * 2

Scan '3':	operand -> output.	Output: 3	Stack: (empty)
Scan '+':	push '+'.	Output: 3	Stack: +
Scan '4':	operand -> output.	Output: 3 4	Stack: +
Scan '*':	precedence(*)=2 > (+)=1, push '*'.	Output: 3 4	Stack: + *
Scan '2':	operand -> output.	Output: 3 4 2	Stack: + *
End:	pop all.	Output: 3 4 2 * +	Stack: (empty)

Stack visualization during scan of '*':

```

+-----+
|  *  | <-- top (higher precedence, sits on top)
+-----+
|  +  | (lower precedence, waiting below)
+-----+

```

9.3 Huffman tree construction for "AAABBBCC"

Leaves: [C:2] [B:3] [A:5]

Merge C+B -> (5):	Merge (5)+A -> (10):
(5)	(10)
/ \	/ \
C:2 B:3	A:5 (5)
	/ \
	C:2 B:3

Codes: A=0 (1 bit), B=11 (2 bits), C=10 (2 bits)

Original: 8 chars * 8 bits = 64 bits | Encoded: 14 bits | Ratio: 4.57

10. When to Use / When to Avoid

Scenario	Recommended approach	Avoid
Short fixed-size text	Char arrays (stack-allocated)	<code>strcpy</code> with untrusted source length
Data with long repeated runs	RLE -- simple, $O(n)$	Huffman when speed matters and ratio is secondary
Non-uniform character distribution	Huffman -- optimal prefix-free codes	RLE on random data (it expands the output)
Decimal to binary/hex conversion	Stack-based remainder collection	Manual string concatenation
Expression parsing with precedence	Shunting-Yard with operator stack	Hand-written recursive descent for simple cases
Sorting a generic array	<code>qsort</code> with a comparator function	Bubble sort when $O(n \log n)$ is needed
Searching a sorted array	<code>bsearch</code> -- $O(\log n)$	Linear scan -- $O(n)$

11. Exam Hot Spots

This chapter has 0 MCQ corpus questions. It is a concept-driven capstone. The exam tests these topics indirectly through:

- **Stack questions** (Chapter 5): if you see a stack used with `%` and `/`, it is base conversion. If you see an operator-precedence table, it is infix-to-postfix.
- **Tree questions** (Chapters 9-10): Huffman coding as an application of binary trees and priority queues.
- **Library function questions**: `<string.h>`, `<stdlib.h>`, `<ctype.h>` signatures and behaviors.

Cross-references:

Application	Depends on	Chapter
String as char array	Pointers, arrays, memory layout	Ch 1, Ch 3
Pattern matching (KMP)	Failure function from prefix table	--
RLE compression	Linear scan, simple counting	--
Huffman coding	Binary trees, priority queue (min-heap)	Ch 7, Ch 9
Base conversion	Stack push/pop	Ch 5
Infix to postfix	Stack, operator precedence	Ch 5
<code>qsort</code> / <code>bsearch</code>	Comparator function pointer, pointer arithmetic	Ch 1, Ch 3

12. Common Traps

Wrong assumption	Why it is wrong
<code>strlen("Hello")</code> returns 6	It returns 5. <code>strlen</code> does NOT count the null terminator. <code>sizeof("Hello")</code> returns 6.
<code>strcpy</code> checks buffer size	It does not. If <code>dst</code> is smaller than <code>src</code> , you get a buffer overflow. Use <code>strncpy</code> or verify sizes.
<code>strcmp("abc", "abcd")</code> returns -1	It returns a negative value, but not necessarily -1. It returns the difference of the first differing characters.

The <code>qsort</code> comparator <code>return a - b</code> always works	This overflows when <code>a</code> and <code>b</code> have very different signs. Use <code>(a > b) - (a < b)</code> instead.
Base conversion remainder order: read top-to-bottom	Remainders are generated LSB first. You must reverse them (pop from stack) to get the correct number.
<code>strncpy(dst, src, n)</code> always null-terminates	If <code>src</code> is longer than <code>n</code> , <code>strncpy</code> does NOT add <code>'\0'</code> . You must manually set <code>dst[n-1] = '\0'</code> .
RLE always compresses	RLE can expand data with no runs. "ABCD" becomes "A1B1C1D1" -- 8 bytes instead of 4.
<code>bsearch</code> works on unsorted arrays	It requires the array to be sorted in the same order the comparator defines.
<code>isdigit(c)</code> works with any <code>int</code> value	If <code>c</code> is negative (signed char > 127), behavior is undefined. Cast to unsigned char first.
<code>^</code> is left-associative in infix-to-postfix	<code>^</code> (exponentiation) is right-associative. <code>2^3^4</code> parses as <code>2^(3^4)</code> , not <code>(2^3)^4</code> .

13. Quick-Recall Card

STRING BASICS

```
"Hello" -> {'H','e','l','l','o','\0'}    sizeof=6, strlen=5
strlen(s)      count until \0           O(n)
strcpy(d,s)    copy bytes incl \0       O(n)
strcmp(a,b)    char-by-char compare     O(min(n,m))
strcat(d,s)    find \0 in dst, append   O(n+m)
strncpy(d,s,n) copy at most n bytes; MANUALLY add \0 if needed
```

BASE CONVERSION (stack)

```
while (N > 0): push(N % base), N /= base
pop all -> digits in correct order (MSB first)
13 -> binary -> push 1,0,1,1 -> pop 1,1,0,1 -> "1101"
```

INFIX TO POSTFIX (Shunting-Yard)

```
operand -> output | '(' -> push | ')' -> pop until '('
operator -> pop higher-or-equal precedence, then push | end -> pop all
Precedence: +,- = 1; *,/ = 2; ^ = 3
```

POSTFIX EVALUATION

```
operand -> push | operator -> pop y, pop x, push (x op y)
```

RLE: count runs, emit (char, count). Good: repeated data. Bad: random data.

HUFFMAN: count freq -> min-heap -> merge two smallest -> left=0, right=1

C LIBRARY

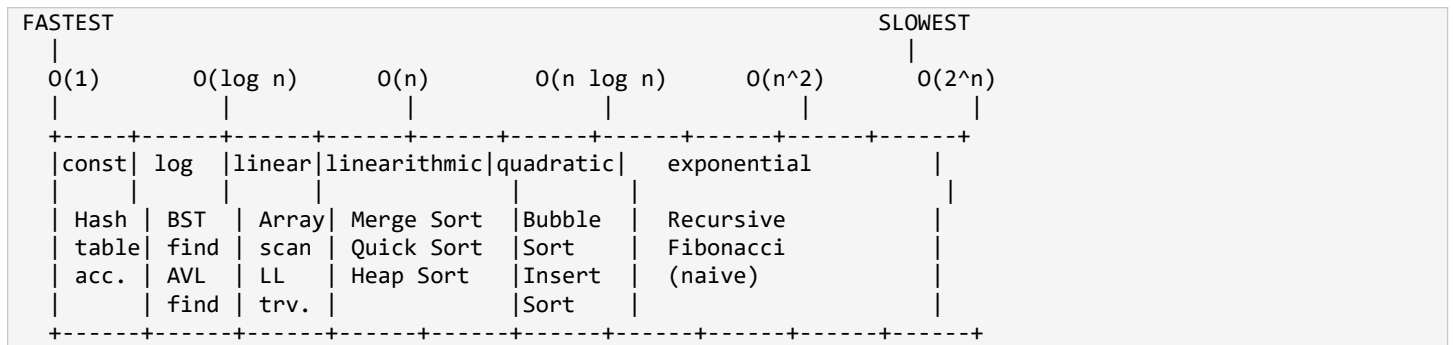
```
<string.h>  strlen, strcpy, strncpy, strcmp, strncmp, strcat
<stdlib.h>  malloc, free, atoi, qsort, bsearch
<ctype.h>   isdigit, isalpha, toupper, tolower
qsort(base,n,sz,cmp)  compar: <0 if a<b, 0 if equal, >0 if a>b
bsearch(key,base,n,sz,cmp)  array MUST be sorted; returns pointer or NULL
```

\newpage

Appendix A -- Master Big-O Cheat Sheet

Quick-reference table of every complexity claim across all 14 chapters. Use it for last-minute revision and for cross-checking answers during the exam.

1. Visual Complexity Class Chart



2. Data Structure Operation Table

2.1 Static Array (Ch 3)

Operation	Average	Worst	Space	Notes
Access by index	$O(1)$	$O(1)$	$O(n)$	Contiguous memory
Search (unsorted)	$O(n)$	$O(n)$	$O(n)$	Linear scan
Search (sorted, binary)	$O(\log n)$	$O(\log n)$	$O(n)$	Binary search
Insert at end	$O(1)^*$	$O(n)$	$O(n)$	*amortized if capacity allows
Insert at middle	$O(n)$	$O(n)$	$O(n)$	Shift elements right
Delete at end	$O(1)$	$O(1)$	$O(n)$	Decrement length
Delete at middle	$O(n)$	$O(n)$	$O(n)$	Shift elements left

2.2 Dynamic Array / Vector (Ch 3)

Operation	Average	Worst	Space	Notes
Access by index	$O(1)$	$O(1)$	$O(n)$	Same as static
Search (unsorted)	$O(n)$	$O(n)$	$O(n)$	Linear scan
Insert at tail	$O(1)$	$O(n)$	$O(n)$	Amortized; worst = reallocate
Insert at middle	$O(n)$	$O(n)$	$O(n)$	Shift + possible realloc
Delete at tail	$O(1)$	$O(1)$	$O(n)$	
Delete at middle	$O(n)$	$O(n)$	$O(n)$	Shift elements left

2.3 Singly Linked List (Ch 4)

Operation	Average	Worst	Space	Notes
Access by index	$O(n)$	$O(n)$	$O(n)$	Sequential traversal
Search	$O(n)$	$O(n)$	$O(n)$	Walk from head
Insert at head	$O(1)$	$O(1)$	$O(n)$	Update head pointer

Insert at tail	$O(n)$	$O(n)$	$O(n)$	Walk to end (no tail ptr)
Insert after given node	$O(1)$	$O(1)$	$O(n)$	When node pointer is given
Delete at head	$O(1)$	$O(1)$	$O(n)$	
Delete by value	$O(n)$	$O(n)$	$O(n)$	Search + unlink

2.4 Doubly Linked List (Ch 4)

Operation	Average	Worst	Space	Notes
Access by index	$O(n)$	$O(n)$	$O(n)$	Can search from either end
Search	$O(n)$	$O(n)$	$O(n)$	Walk from head or tail
Insert at head	$O(1)$	$O(1)$	$O(n)$	
Insert at tail	$O(1)$	$O(1)$	$O(n)$	With tail pointer
Insert after given node	$O(1)$	$O(1)$	$O(n)$	
Delete at head	$O(1)$	$O(1)$	$O(n)$	
Delete by value	$O(n)$	$O(n)$	$O(n)$	Search + unlink both links

2.5 Hash Table (Ch 6)

Operation	Average	Worst	Space	Notes
Search	$O(1)$	$O(n)$	$O(n)$	Worst = all keys collide
Insert	$O(1)$	$O(n)$	$O(n)$	Amortized with resizing
Delete	$O(1)$	$O(n)$	$O(n)$	
Resize (rehash)	$O(n)$	$O(n)$	$O(n)$	Triggered by load factor

2.6 Binary Heap (Ch 7)

Operation	Average	Worst	Space	Notes
Peek (find min/max)	$O(1)$	$O(1)$	$O(n)$	Root element
Insert	$O(\log n)$	$O(\log n)$	$O(n)$	Bubble up
Extract min/max	$O(\log n)$	$O(\log n)$	$O(n)$	Replace root, sift down
Build heap (heapify)	$O(n)$	$O(n)$	$O(n)$	Bottom-up construction
Decrease key	$O(\log n)$	$O(\log n)$	$O(n)$	
Delete arbitrary	$O(\log n)$	$O(\log n)$	$O(n)$	Swap to root, extract

2.7 Binary Search Tree (Ch 9)

Operation	Average	Worst	Space	Notes
Search	$O(\log n)$	$O(n)$	$O(n)$	Worst =

				degenerate/skewed tree
Insert	$O(\log n)$	$O(n)$	$O(n)$	
Delete	$O(\log n)$	$O(n)$	$O(n)$	
Find min / max	$O(\log n)$	$O(n)$	$O(n)$	Follow leftmost / rightmost
In-order traversal	$O(n)$	$O(n)$	$O(n)$	Visits every node

2.8 AVL Tree (Ch 10)

Operation	Average	Worst	Space	Notes
Search	$O(\log n)$	$O(\log n)$	$O(n)$	Guaranteed balanced
Insert	$O(\log n)$	$O(\log n)$	$O(n)$	At most 2 rotations
Delete	$O(\log n)$	$O(\log n)$	$O(n)$	At most $O(\log n)$ rotations
Find min / max	$O(\log n)$	$O(\log n)$	$O(n)$	
In-order traversal	$O(n)$	$O(n)$	$O(n)$	

2.9 B-Tree (Ch 11)

Operation	Average	Worst	Space	Notes
Search	$O(\log n)$	$O(\log n)$	$O(n)$	Height = $O(\log_m n)$
Insert	$O(\log n)$	$O(\log n)$	$O(n)$	Split at most $O(\log n)$ nodes
Delete	$O(\log n)$	$O(\log n)$	$O(n)$	Merge at most $O(\log n)$ nodes

2.10 Graph Algorithms (Ch 12)

Algorithm	Average	Worst	Space	Notes
BFS	$O(V + E)$	$O(V + E)$	$O(V)$	Queue-based level traversal
DFS	$O(V + E)$	$O(V + E)$	$O(V)$	Stack/recursion based
Dijkstra (binary heap)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$	Non-negative weights
Prim (binary heap)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$	MST

3. Sorting Algorithms (Ch 13)

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Adaptive with early exit
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Min swaps ($n-1$)
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Best for

						small/nearly sorted
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Divide-and-conquer
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Pivot-dependent; in-place
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	In-place, not stable
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes	Integer keys in $[0, k]$
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes	d = digit count

4. Stack and Queue Operations (Ch 5)

Data Structure	Operation	Average	Worst	Space
Stack (array)	Push	$O(1)$	$O(n)$	$O(n)$
Stack (array)	Pop	$O(1)$	$O(1)$	$O(n)$
Stack (array)	Peek/Top	$O(1)$	$O(1)$	$O(n)$
Queue (array, circular)	Enqueue	$O(1)$	$O(n)$	$O(n)$
Queue (array, circular)	Dequeue	$O(1)$	$O(1)$	$O(n)$
Queue (linked list)	Enqueue	$O(1)$	$O(1)$	$O(n)$
Queue (linked list)	Dequeue	$O(1)$	$O(1)$	$O(n)$

5. Summary -- Exam Quick-Pick

$O(1)$	: Array access, hash lookup (avg), stack/queue ops, heap peek
$O(\log n)$	: BST (avg), AVL (all), B-Tree, binary search, heap insert/extract
$O(n)$	: Linked list search, BFS, DFS, array linear scan, heap build
$O(n \log n)$	: Merge sort, heap sort, quick sort (avg)
$O(n^2)$	: Bubble, selection, insertion sort (avg/worst), quick sort worst
$O(2^n)$	: Recursive Fibonacci, power set generation

\newpage

Appendix B -- MCQ Coverage Matrix

339 questions from the exam corpus, mapped to their primary chapter. Use this to identify high-yield chapters and to verify that every question ID you encounter has a home.

Summary

Metric	Value
Total questions	339
Chapters covered	10
Chapters uncovered	4

English questions	109
Romanian questions	230

Per-Chapter Breakdown

Chapter 1 -- Pointers & the Memory Model

Field	Value
Questions	120
Topic	pointers
Question IDs	3, 12, 13, 20, 24, 26, 27, 31, 32, 33, 37, 44, 45, 47, 48, 51, 53, 56, 68, 70, 72, 74, 76, 83, 84, 86, 87, 88, 91, 94, 95, 96, 98, 99, 100, 102, 103, 106, 108, 111, 112, 114, 118, 119, 120, 121, 123, 127, 210, 212, 213, 215, 216, 217, 219, 220, 223, 224, 233, 237, 239, 241, 243, 247, 249, 250, 461, 463, 464, 465, 466, 467, 468, 470, 474, 475, 484, 488, 492, 493, 494, 498, 500, 502, 963, 965, 966, 968, 969, 970, 972, 976, 977, 978, 981, 983, 985, 987, 988, 989, 991, 992, 993, 995, 999, 1000, 1001, 1006, 1007, 3938, 3939, 3940, 3941, 3942, 3943, 3946, 3949, 3950, 3955, 3956

Chapter 2 -- Complexity & Big-O Notation

Field	Value
Questions	0
Topic	--
Question IDs	(none)

No corpus questions are assigned to this chapter. Complexity theory is tested indirectly through the Big-O columns of other chapters (e.g., Ch 3 arrays, Ch 9 BSTs).

Chapter 3 -- Arrays & Dynamic Arrays

Field	Value
Questions	45
Topic	arrays
Question IDs	1, 2, 4, 15, 25, 28, 39, 41, 55, 59, 60, 67, 81, 82, 89, 101, 107, 109, 126, 208, 226, 227, 229, 231, 459, 477, 478, 482, 483, 962, 979, 980, 984, 1002, 1004, 1005, 1968, 3944, 3945, 3947, 3948, 3951, 4003, 4007, 4011

Chapter 4 -- Linked Lists

Field	Value
Questions	40
Topic	linked_lists
Question IDs	7, 8, 10, 11, 17, 18, 21, 22, 34, 38, 49, 50, 54, 65, 66, 77, 85, 92, 117, 125, 211, 218, 240, 244, 248, 462, 472, 491, 495, 499, 961, 971, 982, 994, 1969, 1970, 3952, 3953, 3970, 3972

Chapter 5 -- Stacks & Queues

Field	Value
Questions	2
Topic	stacks_queues
Question IDs	19, 64

Chapter 6 -- Hash Tables

Field	Value
Questions	15
Topic	hash_tables
Question IDs	42, 43, 52, 75, 228, 234, 235, 236, 245, 479, 485, 486, 487, 497, 7951

Chapter 7 -- Binary Heaps

Field	Value
Questions	28
Topic	heaps
Question IDs	5, 6, 9, 14, 16, 23, 61, 63, 69, 93, 122, 214, 469, 967, 990, 3954, 3963, 3998, 3999, 4000, 4001, 4002, 4004, 4005, 4006, 4008, 4009, 4010

Chapter 8 -- Recursion

Field	Value
Questions	0
Topic	--
Question IDs	(none)

No corpus questions are assigned to this chapter. Recursion is tested implicitly through tree traversals (Ch 9) and sorting algorithms (Ch 13).

Chapter 9 -- Binary Search Trees

Field	Value
Questions	38
Topic	bst, trees
Question IDs	30, 35, 40, 62, 71, 73, 97, 105, 110, 113, 115, 222, 225, 246, 471, 473, 501, 973, 975, 996, 998, 1003, 3957, 3958, 3959, 3961, 3962, 3964, 3965, 3966, 3967, 3968, 3969, 3971, 3973, 3990, 7949, 7950

Chapter 10 -- AVL Trees

Field	Value
Questions	49
Topic	avl
Question IDs	29, 36, 46, 57, 90, 104, 116, 124, 209, 221, 230, 232, 238, 242, 251, 460, 476, 480, 481, 489, 490, 496, 964, 974, 986, 997, 3974, 3975, 3976, 3977, 3978, 3979, 3980, 3981, 3982, 3983, 3984, 3985, 3986, 3987, 3988, 3989, 3991, 3992, 3993, 3994, 3995, 3996, 3997

Chapter 11 -- B-Trees

Field	Value
Questions	0
Topic	--
Question IDs	(none)

No corpus questions are assigned to this chapter. B-Tree concepts (balanced multi-way search) are tested indirectly through AVL tree questions (Ch 10).

Chapter 12 -- Graphs

Field	Value
Questions	1
Topic	graphs
Question IDs	58

Chapter 13 -- Sorting Algorithms

Field	Value
Questions	1
Topic	sorting
Question IDs	3960

Chapter 14 -- Advanced Topics & Integration

Field	Value
Questions	0
Topic	--
Question IDs	(none)

No corpus questions are assigned to this chapter. Integration questions may draw from multiple chapters but are not tagged individually.

Coverage Notes

- **Chapters with 0 questions (2, 8, 11, 14):** These chapters are either theoretical (Ch 2 Big-O, Ch 8 recursion), covered indirectly by other chapters (Ch 11 B-Trees via Ch 10 AVL), or serve as integration/synthesis topics (Ch 14) without standalone MCQs.
- **Heaviest chapters:** Ch 1 (120 questions, 35.4%), Ch 10 (49 questions, 14.5%), Ch 3 (45 questions, 13.3%), Ch 4 (40 questions, 11.8%). These four chapters account for 75% of the corpus.
- **Lightest covered chapters:** Ch 5 (2 questions), Ch 12 (1 question), Ch 13 (1 question). These are tested but underrepresented.

\newpage

Appendix C -- C Memory Glossary

One-line definitions for every C memory concept used across all 14 chapters. Each entry includes the chapter(s) where it is first or most heavily tested.

1. Storage Class Specifiers

Term	Definition	Chapter
<code>auto</code>	Default storage class for local variables; lifetime is the enclosing block.	Ch 1
<code>static</code>	Extends variable lifetime to the entire program; preserves value across calls.	Ch 1
<code>extern</code>	Declares a variable defined in another translation unit.	Ch 1
<code>register</code>	Suggests the compiler store the variable in a	Ch 1

	CPU register (rarely used in modern code).	
--	--	--

2. Dynamic Allocation Functions

Term	Definition	Chapter
<code>malloc(n)</code>	Allocates <code>n</code> bytes of uninitialized heap memory; returns <code>void*</code> .	Ch 1
<code>calloc(count, size)</code>	Allocates <code>count * size</code> bytes, zero-initialized.	Ch 1
<code>realloc(p, n)</code>	Resizes the block at <code>p</code> to <code>n</code> bytes; may copy data to a new location.	Ch 1
<code>free(p)</code>	Returns the heap block at <code>p</code> to the OS; does not change <code>p</code> itself.	Ch 1

3. Memory Segments

Term	Definition	Chapter
Stack	LIFO region for local variables and function frames; auto-managed, fast, limited size.	Ch 1
Heap	Free-store region for dynamic allocations; manually managed via <code>malloc/free</code> .	Ch 1
BSS	Block Started by Symbol; holds uninitialized global and static variables (zero-filled).	Ch 1
Data segment	Holds initialized global and static variables.	Ch 1
Code (text) segment	Read-only region containing the compiled machine instructions.	Ch 1

4. Pointer Bugs

Term	Definition	Chapter
Dangling pointer	A pointer that still holds the address of freed or returned memory.	Ch 1
Memory leak	Allocated heap memory that is never freed; the pointer is lost or overwritten.	Ch 1
Buffer overflow	Writing past the end of an array, corrupting adjacent memory.	Ch 1

5. Operators and Keywords

Term	Definition	Chapter
<code>sizeof</code>	Operator that returns the size in bytes of a type or expression (compile-time).	Ch 1
<code>&</code> (address-of)	Returns the memory address of its operand.	Ch 1
<code>*</code> (dereference)	Accesses the value at the address held by a	Ch 1

	pointer.	
--	----------	--

6. Pointer Qualifiers and Types

Term	Definition	Chapter
<code>const int* p</code>	Pointer to a const int; <code>*p</code> cannot be modified, but <code>p</code> can be reassigned.	Ch 1
<code>int* const p</code>	Const pointer to int; <code>p</code> cannot be reassigned, but <code>*p</code> can be.	Ch 1
<code>const int* const p</code>	Const pointer to const int; neither <code>p</code> nor <code>*p</code> can be modified.	Ch 1
<code>void*</code>	Generic pointer type; can point to any data type; cannot be dereferenced directly.	Ch 1
<code>(type*) cast</code>	Explicit conversion from <code>void*</code> to a typed pointer (required in C++, optional in C).	Ch 1
<code>typedef</code>	Creates an alias for an existing type; used to simplify pointer/struct declarations.	Ch 4

7. Struct Memory Concepts

Term	Definition	Chapter
Struct padding	Compiler-inserted bytes after struct members to satisfy alignment requirements.	Ch 4
Alignment	Requirement that data be stored at addresses that are multiples of its size.	Ch 4
<code>struct Node</code>	User-defined composite type; each instance can contain pointers to other instances (linked list nodes).	Ch 4

8. Composite Types

Term	Definition	Chapter
<code>union</code>	A type where all members share the same memory location; size = largest member.	Ch 5
<code>enum</code>	User-defined integer type with named constants (e.g., <code>enum Color { RED, GREEN };</code>).	Ch 5
Bit fields	Struct members with a specified bit width; used to pack flags tightly.	Ch 5

9. Bitwise Operators

Term	Definition	Chapter
<code>&</code> (bitwise AND)	Sets each bit to 1 only if both operands have 1	Ch 5

	at that position.	
\ (bitwise OR)	Sets each bit to 1 if either operand has 1 at that position.	Ch 5
^ (bitwise XOR)	Sets each bit to 1 if exactly one operand has 1 at that position.	Ch 5
~ (bitwise NOT)	Flips every bit (0 becomes 1, 1 becomes 0).	Ch 5
<< (left shift)	Shifts bits left by n positions; fills with zeros on the right.	Ch 5
>> (right shift)	Shifts bits right by n positions; behavior for signed types is implementation-defined.	Ch 5

\newpage